

SOMMAIRE

SOMMAIRE	3
LISTE DES FIGURES.....	9
INTRODUCTION GENERALE	18
CHAPITRE 1	21
CADRE GENERAL DU PROJET	21
I. INTRODUCTION	22
II. PRESENTATION DE L'ORGANISME D'ACCUEIL :	22
III. CAHIER DES CHARGES	23
1. CONTEXTE DU PROJET	23
IV. OBJECTIFS DU PROJET :	25
V. PROBLEMATIQUE :	25
VI. SOLUTIONS PROPOSEES :	26
VII. SPECIFICATION DES BESOINS FONCTIONNELS :	26
VIII. SPECIFICATION DES BESOINS NON FONCTIONNELS :	27
IX. CONCLUSION	28
CHAPITRE 2 :	30
CONCEPTS DE BASE	30
NOTION DEVOPS	30
I. INTRODUCTION :	31
II. QU'EST-CE QUE SDLC ?	31
III. POURQUOI DEVOPS :	32
IV. QU'EST-CE QUE DEVOPS?	33
V. QUAND ADOPTER DEVOPS ?	34
1. Grandes Applications Distribuées :	34
2. Applications Hébergées sur une Plateforme Cloud :	35
VI. Les principes essentiels de DevOps:	35
VII. PIPELINE ET CHAINE D'OUTILS DEVOPS :	36
VIII. Conclusion	37
CHAPITRE 3 :	38
ETAT DE L'ART.....	38
I. INTRODUCTION :	39
II. CI/CD : QU'EST-CE QUE C'EST ?	39

1.	L'INTEGRATION CONTINUE :	40
➤	DES OUTILS D'INTEGRATION CONTINUE :	41
2.	LE DEPLOIEMENT CONTINU :	41
➤	Des outils déploiement continu:	41
3.	LES AVANTAGES DU CI/CD :	42
III.	CONTROLE DU CODE SOURCE / BUILD :	43
1.	VERSION CONTROL SYSTEMS	44
2.	ARTIFACT REPOSITORIES	44
IV.	ARCHITECTURE DES MICROSERVICES :	45
V.	CONCEPTION D'ARCHITECTURE MICROSERVICE	46
1.	QUE SONT LES MICROSERVICES ?	47
VI.	AVANTAGES DES MICROSERVICES	48
-	Agilité :	48
VII.	TEST CONTINU :	49
VIII.	CLOUD ET CONTENEURISATION :	50
1.	Cloud Infrastructures	50
2.	Conteneurisation	51
3.	Orchestration des conteneurs	52
IX.	OUTILS DE GESTION DE CONFIGURATION ET AUTOMATISATION :	52
X.	MONITORING ET ALERTING :	53
XI.	Conclusion :	54
Chapitre 4 : Méthodes Agiles pour la Gestion de Projet		55
I.	Introduction :	56
II.	Principes de l'Agilité :	56
III.	Méthodologie Scrum :	56
IV.	AVANTAGES DE SCRUM POUR LA GESTION DE PROJET :	57
V.	APPLICATION DE SCRUM DANS LE PROJET :	58
VI.	UTILISATION DE JIRA DANS LE PROJET DE GESTION DES INSCRIPTIONS :	58
VII.	SPECIFICATION DES BESOINS :	59
1.	IDENTIFICATION DES ACTEURS	59
2.	MODELISATION DES BESOINS	61
VIII.	BACKLOG DE PRODUIT :	63
IX.	PLANIFICATION DES SPRINTS :	68
X.	L'ARCHITECTURE DE L'APPLICATION EN MICROSERVICES :	69
XI.	CONCLUSION :	71

CHAPITRE 5 :	73
Sprint 0 Mise en place du projet	73
I. INTRODUCTION :	74
1. OBEJECTIF SPRINT 0.....	74
2. BACKLOG DU SPRINT 0 :	74
3. TACHES A ACCOMPLIR :	75
II. Conclusion :	76
CHAPITRE 6 : SPRINT 1.....	77
I. Introduction :	78
II. SPRINT GOAL DU SPRINT 1 :	78
III. BACKLOG DU SPRINT 1 :	78
IV. ANALYSE FONCTIONNEL DE MICROSERVICES DE AUTHENTIFICATION :	79
1. MICROSERVICE D'AUTHENTIFICATION POUR LES ETUDIANTS :	80
2. MICROSERVICE D'AUTHENTIFICATION POUR LES ADMINISTRATEURS :	80
V. CONCEPTION DE MICROSERVICES DAUTHENTIFICATION :	80
1. Diagrammes de cas d'utilisations de sprint 1 :	81
2. Diagramme de séquence de cas « S'authentifier »	82
3. Diagramme de classe pour sprint 1 :	82
VI. LES ETAPES DE CREATION DU PIPELINE POUR MICROSERVICES AUTHENTIFICATION : 84	
1. PLAN :	84
2. ACCES AU CODE SOURCE SUR GITHUB :	84
3. EXECUTER LES TESTS :	85
4. VERIFIER LA QUALITE DU CODE :	86
5. CREATION DES BUILD :	86
6. CREATION DES IMAGES :	87
7. Déploiement sur kubernetes :	88
8. Monitoring des l'application :	89
9. Technologies utilisées :	91
10. CI/CD.....	92
VII. REALISATION DU SPRINT 1:	93
1. INTERFACE D'AUTHENTIFICATION :	93
2. INTERFACE D'INSCRIPTION DES ETUDIANTS :	93
3. INTERFACE DE MISE A JOUR PROFIL.....	94
VIII. MICROSERVICES 2 GESTION DES ETUDIANTS.....	95

1.	ANALYSE FONCTIONNEL DE MICROSERVICES DE GESTION DES ETUDIANTS	95
2.	LES ETAPES DE CREATION DU PIPELINE POUR MICROSERVICES GESTION DES ETUDIANTS :	95
IX.	REALISATION DE GESTION DES ETUDIANTS :	100
1.	INTERFACE LISTER LES ETUDIANTS :	100
2.	INTERFACE AJOUTER UN NOUVEAU ETUDIANT :	101
X.	Conclusion :	101
CHAPITRE 7 : SPRINT 2.....		102
I.	Intoduction :	103
II.	SPRINT GOAL DU SPRINT 2 :	103
III.	Backlog du sprint 2 :	103
IV.	ANALYSE FONCTIONNEL DE MICROSERVICES DE GESTION DES FORMATIONS :	105
V.	CONCEPTION DE MICROSERVICES GESTION DES FORMATIONS :	106
1.	Diagramme de cas utilisation pour admin Gérer les formations	106
2.	Diagramme de cas utilisation pour étudiant consulter les formations.....	107
3.	Diagramme de séquence ajouter formations :	108
4.	Diagramme de séquence Modifier formations :	109
5.	diagramme de classe de sprint 2 :	109
VI.	LES ETAPES DE CREATION DU PIPELINE POUR MICROSERVICES GESTION DES FORMATIONS.....	110
1.	Plan :	110
2.	accès au code source sur GitHub :	111
3.	Exécuter les tests :	111
4.	vérifier la qualité du code :	112
5.	CREATION DES IMAGES :	113
6.	DEPLOIEMENT SUR KUBERNETES :	113
7.	Monitoring des l'application :	114
8.	JENKINS :	115
VII.	REALISATION :	116
1.	INTERFACE GERER GESTION DES FORMATIONS :	116
2.	INTERFACE AJOUTER UN NOUVEAU FORMATION :	117
3.	INTERFACE CONSULTER LISTE DES FORMATIONS :	118
4.	INTERFACE DE FORMULAIRE DE PARTICIPATION :	118
5.	INTERFACE DE CONFIRMATION DE PARTICIPATION :	119
VIII.	MICROSERVICES 3 GESTION DES EVENEMENTS	120

1.	ANALYSE FONCTIONNEL DE MICROSERVICES DE GESTION DES EVENEMENTS : ..	120
2.	CONCEPTION DE GESTION DES EVENEMENTS	121
3.	LES ETAPES DE CREATION DU PIPELINE POUR MICROSERVICES GESTION DES EVENEMENTS :	121
IX.	REALISATION DES MICRO SERVICES DE GESTION EVENEMENTS :	128
1.	INTERFACE DE GERER DES EVNEMENTS.....	128
2.	INTERFACE DAJOUTER NOUVEL EVENEMENTS :	129
3.	INTERFACE DE CONSULTER LES EVNEMENTS POUR ETUDIANTS	130
X.	Conclusion :	130
	CHAPITRE 8.....	131
	SPRINT 3	131
I.	Introduction :.....	132
II.	OBJECTIFS DU SPRINT 3 :	132
III.	BACKLOG SPRINT 3 :	132
IV.	REALISATION PARTIE FRONTEND	133
1.	Création d'un Dockerfile	133
2.	Création d'un Jenkinsfile	135
3.	Tests Selenium et SonarQube	135
4.	Déploiement avec Kubernetes	136
V.	CONCLUSION :	137
	CONCLUSION GENERALE	139
	NETOGRAPHIE	142
	Perspective et Intégrations Technologiques	143

LISTE DES FIGURES

Figure 1la societe itQAN	22
Figure 2 SDLC	31
Figure 3 DEVELOPPEURS VS PERATIONS	32
Figure 4 DEVELOPPEURS AVEC OPERATIONS	33
Figure 5: objectif DevOps.....	34
Figure 6:chaîne de pipiliene.....	36
Figure 7:description CI CD.....	40
Figure 8 : les outils CI.....	41
Figure 9: les outils CD	42
Figure 10: pipeline CI CD	43
Figure 11: les outils de version de controle	44
Figure 12: les outils de build des artrfacts	45
Figure 13:LES OUTILS D D'ARTEFACT REPOSITORIES	45
Figure 14:ARCHITECTURE MONOLITHIQUE VS ARCHITECTURE DE MICROSERVICES	46
Figure 15:ARCHITECTURE DE MICROSERVICES.....	47
Figure 16:LES OUTILS DE TEST CONTENU	50
Figure 17:LES OUTILS DE CLOUD.....	51
Figure 18:LES OUTILS DE CONTENEURISATION.....	52

Figure 19:LES OUTILS DE CONTENEUR	52
Figure 20:LES OUTILS D’AUTOMATISATION	53
Figure 21:LES OUTILS MONOTORING	54
Figure 22:TEAM MEMBRES	Erreur ! Signet non défini.
Figure 23:DIAGRAMME DE CAS UTILISATION GLOBAL	62
Figure 24:ARCHITECTURE MICROSERVICES DE MON PROJET	71
Figure 25:DIAGRAMME DE CAS D’UTILISATIONS POUR SPRINT 1	81
Figure 26:DIAGRAMME DE CAS D’UTILISATIONS POUR ETUDIANT SPRINT 1	Erreur ! Signet non défini.
Figure 27:DIAGRAMME DE SEQUENCE DE CAS « S’AUTHENTIFIER »	82
Figure 28:DIAGRAMME DE CLASSE.....	83
Figure 29:INTERFACE JIRA SPRINT 1	84
Figure 30:INTERFACE GITHUB POUR MICROSERVICE AUTHENTIFICATION	85
Figure 31:CAPTURE DE TEST SONARQUBE POUR MICROSERVICE AUTHENTIFICATION	86
Figure 32:CAPTURE DE CREATION DES BUILD	87
Figure 33:CAPTURE DE CONNEXION DOCKER HUB.....	87
Figure 34:CAPTURE DE TAG IMAGE DOCKER	88
Figure 35:CAPTURE DE POUSSER L’IMAGE VERS DOCKER HUB.....	88
Figure 36:CAPTURE DE DOCKER HUB.....	88

Figure 37:CAPTURE DE DEPLOYMENTS	89
Figure 38:CAPTURE DE COLLECTER METRICS	90
Figure 39:CAPTURE DE PROMETHEUS.....	91
Figure 40:CAPTURE DE GRAFANA	91
Figure 41:INTEFRACE DAUTHENTIFIER.....	93
Figure 42:INTERFACE D'INSCRIPTION DES ETUDIANTS	94
Figure 43:INTERFACE UPDATE PROFILE ETUDIANTS	94
Figure 44:INTERFACE GITHUB POUR MICROSERVICE GESTION ETUDIANTS	96
Figure 45:CAPTURE DE JTEST POUR MICROSERVICE GESTIONN ETUDIANTS	96
Figure 46:CAPTURE DE TEST SONARQUBE.....	97
Figure 47:CAPTURE DE CREATION DES BUILD	97
Figure 48:CAPTURE DE CREATION IMAGE.....	98
Figure 49:CAPTURE EXECUTION PODS.....	98
Figure 50:INTERFACE DE CONTENEUR MYSQL.....	98
Figure 51:INTERFACE DE CONTENEUR MYSQL.....	99
Figure 52:INTERFACE PROMTHEUS	99
Figure 53:CAPTURE DASHOBAORD GRAFANA	100
Figure 54:INTERFACE GESTION DES ETUDIANTS	100

Figure 55:INTERFACE AJOUTER ETUDIANT.....	101
Figure 56:DIAGRAMME DE CAS UTILISATION POUR ADMIN GERER LES FORMATIONS	106
Figure 57:DIAGRAMME DE CAS UTILISATION POUR ETUDIANT CONSULTER LES FORMATIONS	107
Figure 58:Diagramme de séquence ajouter formations	108
Figure 59:.Diagramme de séquence Modifier formations	109
Figure 60:DIAGRAMME DE CLASSE SPRINT 2	110
Figure 61:INTEFACE IJRA POUR SPRINT 2	110
Figure 62:GITHUB POUR GESTION DES FORMATION Sprint 2	111
Figure 63:CAPTURE DE JTEST POUR MICROSERVICE GESTIONN DES FORMATION	112
Figure 64:CAPTURE DE TEST SONARQUBE.....	112
Figure 65:CAPTURE EXECUTION PODS.....	113
Figure 66:INTERFACE DE CONTENEUR MYSQL.....	113
Figure 67:INTERFACE DE CONTENEUR MYSQL.....	114
Figure 68:INTERFACE DE SERVICE.....	114
Figure 69:INTERFACE DE PROMETHEUS	115
Figure 70:INTERFACE DE GRAFANA	115
Figure 71:INTERFACE DE JENKINS.....	116
Figure 72:INTERFACE DE GESTION DES FORMATION POUR ADMIN	117

Figure 73:INTERFACE D’AJOUTER FORMATION POUR ADMIN	117
Figure 74:INTERFACE DE CONSULTER DES FORMATION POUR ETUDIANT	118
Figure 75:INTERFACE DE PARTICIPER A UNE FORMATION POUR ETUDIANT	119
Figure 76:INTERFACE DE CONFIRMATION A UNE FORMATION POUR ETUDIANT	120
Figure 77:DIAGRAMME DE CAS UTILISATION GERER DES EVENEMENTS	121
Figure 78:GITHUB GERER DES EVENEMENTS.....	122
Figure 79:CAPTURE DE JTEST	123
Figure 80:CAPTURE 1 DE TEST SONARQUBE.....	124
Figure 81:CAPTURE 2 DE TEST SONARQUBE.....	124
Figure 82:capture execution deployments	125
Figure 83:CAPTURE EXECUTION PODS.....	125
Figure 84:CAPTURE EXECUTION LOGS	125
Figure 85:INTERFACE DE CONTENEUR MYSQL.....	126
Figure 86:INTERFACE DE CONTENEUR MYSQL.....	126
Figure 87:INTERFACE DE METRICS.....	127
Figure 88:INTERFACE DE GARFANA	127
Figure 89:INTERFACE DE GARFANA	128

Figure 90:INTERFACE DES LISTE DES EVNEMENTS POUR ADMIN	129
Figure 91:INTERFACE AJOUTER EVNEMENTS POUR ADMIN.....	129
Figure 92:INTERFACE CONSULTER DES EVNEMENTS POUR ETUDIANTS	130
Figure 93:INTERFACE DE GESTION DES VERSION GITHUB	133
Figure 94:INTERFACE DE CREATION IMAGE POUR FRONTEND	134
Figure 95:INTERFACE IMAGE POUR FRONTEND	134
Figure 96:INTERFACE JENKINS PIPILINE POUR FRONTEND	135
Figure 97:INTERFACE TEST SONARQUEB POUR FRONTEND	136
Figure 98:INTERFACE TEST SELNUM POUR FRONTEND	136
Figure 99:: INTERFACE DEPLOYMENT POUR FRONTEND PODS	137
Figure 100:INTERFACE DEPLOYMENT POUR FRONTEND SERVICES.....	137
Figure 101:INTERFACE EXECUTION DEPLOYMENT POUR FRONTEND	137

LISTE DES TABLES

TABLEAU 1 : LISTE DES ACTEURS.....	60
TABLEAU 2 : IDENTIFICATION DES ACTEURS ET DES CAS UTILISATIONS.....	61
TABLEAU 3 : PRODUCT BACKLOG.....	70
TABLEAU 4 : PLANNING DES SPRINTS.....	71
TABLEAU 5 : BACKLOG DE SPRINT 0.....	77
TABLEAU 6 : BACKLOG DE SPRINT 1.....	81
TABLEAU 7 : BACKLOG DE SPRINT 2.....	108
TABLEAU 8 : BACKLOG DE SPRINT 3.....	136

INTRODUCTION GENERALE

Dans le paysage informatique en constante évolution, les approches devops ont profondément transformé le développement et le déploiement de logiciels. En adoptant une culture et des pratiques devops, les organisations peuvent non seulement accélérer la mise sur le marché de leurs produits, mais aussi améliorer significativement la qualité des logiciels tout en augmentant la satisfaction des utilisateurs. Ce rapport se focalise sur l'implémentation réussie de devops au sein du centre 4C de l'iset Tozeur, en mettant particulièrement l'accent sur l'optimisation de la plateforme de gestion des inscriptions aux formations et événements.

L'architecture monolithique, autrefois dominante, montre aujourd'hui ses limites face à la complexité croissante des applications. En réponse, les microservices émergent comme une solution prometteuse, permettant une gestion plus agile et évolutive des systèmes. Cette transition vers une architecture microservices impacte directement la conception et la gestion des processus d'inscription et de participation des étudiants aux activités du centre 4C.

La conteneurisation et l'orchestration deviennent des piliers essentiels de cet écosystème, facilitant un déploiement rapide et fiable des microservices tout en assurant la cohérence des pratiques devops tout au long du cycle de vie des applications. Ces technologies permettent une gestion efficiente des services, minimisant les interruptions et garantissant une expérience utilisateur optimale.

Ce rapport est structuré en plusieurs chapitres, chacun explorant en profondeur un aspect spécifique de notre approche devops dans le contexte du centre 4C :

Chapitre 1 : Cadre général du projet Ce chapitre établit les fondations du projet en définissant les objectifs de la plateforme de gestion des inscriptions, ainsi que ses exigences fonctionnelles et non fonctionnelles.

Chapitre 2 : Concepts fondamentaux et devops Exploration approfondie des principes fondamentaux de devops, avec un accent sur l'impact des microservices et leur adoption dans l'environnement éducatif du centre 4C.

Chapitre 3 : État de l'art et CI/CD Revue des meilleures pratiques en matière d'intégration continue (CI), de déploiement continu (CD), de conteneurisation et d'orchestration, cruciales pour une gestion efficace des microservices.

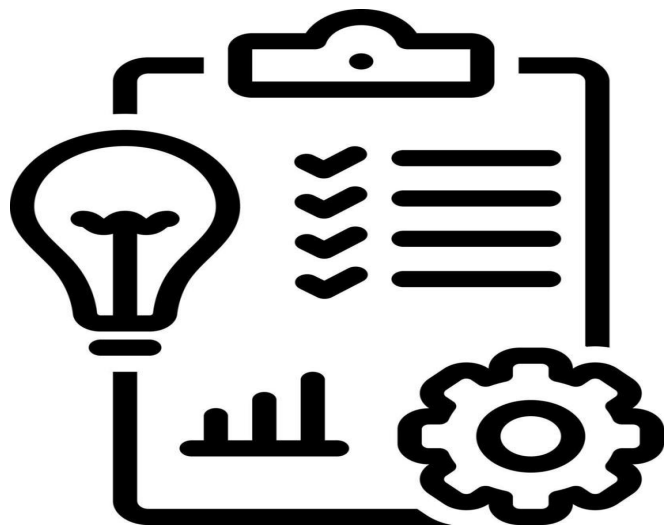
Chapitre 4 : Méthodes agiles pour la gestion de projet Application des méthodologies agiles, telles que Scrum, pour une gestion itérative et incrémentale du projet, mettant l'accent sur l'adaptabilité et la réactivité aux besoins évolutifs.

Chapitre 5 : Implémentation des microservices et sprints Analyse détaillée de la conception, du développement et du déploiement des microservices pour la gestion des inscriptions et événements, illustrée par des études de cas pratiques et les défis surmontés.

Chaque chapitre contribue à offrir une compréhension claire et complète de notre approche devops, mettant en lumière les avantages tangibles d'une gestion modernisée et agile pour les initiatives éducatives et événementielles du centre 4C.

CHAPITRE 1

CADRE GENERAL DU PROJET



I. INTRODUCTION

Dans ce chapitre, j'exposerai le contexte général du projet. Pour cela, je présenterai tout d'abord l'organisme d'accueil. Ensuite, j'énoncerai le contexte du projet ainsi que les différents facteurs qui ont suscité le besoin de mon projet et la solution proposée. Puis, je spécifierai l'ensemble des besoins fonctionnels et non fonctionnels.

II. PRESENTATION DE L'ORGANISME D'ACCUEIL :

J'ai eu l'occasion de préparer mon projet de fin d'étude intitulé « implémentation DEVOPS pour évolution développement d'une application web de gestion de formations ». ITQAN, est une entreprise Tunisienne de service et ingénierie informatique fondée par des ingénieurs expérimentés dans la capitale technologique de San Francisco.



Figure 11a société itQAN

ITQAN, fournit des services de première classe à ses clients sur la scène nationale et internationale, notamment le développement et la conception de sites Web, la gestion de contenu numérique et le référencement SEO. En plus, elle comprend une équipe créative de graphistes et d'éditeurs de contenu. Dans une logique de performance et de proximité client, ITQAN, associe de nombreux services à sa gamme logicielle pour simplifier le quotidien de ses clients, de leur faire gagner du temps, et accompagner le développement de leur entreprise.

III. CAHIER DES CHARGES

1. CONTEXTE DU PROJET

Le Centre de Carrières et de Certification des Compétences (4C) de l'ISET Tozeur est un centre éducatif dynamique qui joue un rôle crucial dans l'enrichissement des compétences des étudiants. Il organise régulièrement divers projets, formations et événements qui visent à améliorer l'employabilité des étudiants et à les préparer pour le marché du travail. Cependant, malgré l'importance et la fréquence de ces activités, le processus actuel de gestion des inscriptions présente plusieurs limitations et défis qui entravent l'efficacité et la qualité de ces initiatives. Voici une analyse détaillée des contextes et des facteurs qui justifient le besoin de développer une nouvelle plateforme de gestion des inscriptions :

a. Inefficacité du Processus Manuel :

Actuellement, la gestion des inscriptions se fait principalement de manière manuelle, ce qui engendre plusieurs problèmes :

- Erreurs humaines : La saisie manuelle des données est sujette à des erreurs fréquentes, telles que des fautes de frappe, des omissions ou des doublons.
- Temps et ressources : Le traitement manuel des inscriptions est chronophage et mobilise beaucoup de ressources humaines, réduisant ainsi le temps disponible pour d'autres tâches importantes.
- Difficulté de suivi : Suivre l'état des inscriptions et des participations manuellement est complexe et peut conduire à des confusions ou à des pertes d'informations.

b. Manque de Centralisation

Les informations relatives aux inscriptions ne sont pas centralisées :

- Accès dispersé aux données : Les données des inscrits sont souvent dispersées dans différents fichiers ou documents, rendant leur accès et leur gestion difficiles.
- Mise à jour inefficace : La mise à jour des informations sur les participants, les formations et les événements est laborieuse et peut entraîner des incohérences.
- Absence de visibilité globale : Les gestionnaires n'ont pas une vue d'ensemble claire et instantanée des inscriptions, ce qui complique la planification et la prise de décision.

c. Absence de Suivi Automatisé et de Notifications

Le processus actuel manque d'automatisation pour le suivi et les notifications :

- Communication limitée : Les participants ne reçoivent pas de notifications automatiques concernant l'état de leur inscription, les rappels d'événements ou les mises à jour importantes.
- Engagement des participants : L'absence de suivi automatisé réduit l'engagement et la satisfaction des participants, qui peuvent se sentir mal informés ou négligés.

d. Gestion Complexe des Périodes de Forte Affluence

Pendant les périodes de forte affluence, la gestion des inscriptions devient particulièrement complexe :

- Surcharge administrative : La hausse du nombre d'inscriptions lors de certains événements peut entraîner une surcharge administrative et des délais dans le traitement des inscriptions.
- Risque accru d'erreurs : Le volume élevé d'inscriptions augmente le risque d'erreurs et de pertes de données.

e. Sécurité des Données

La sécurité des données des participants est une préoccupation majeure :

- Protection insuffisante : Les méthodes actuelles de gestion manuelle ne garantissent pas une protection adéquate des données personnelles et sensibles des participants.
- Conformité aux normes : Il est difficile de se conformer aux normes de sécurité et de confidentialité des données (comme le GDPR) avec un système manuel et dispersé.

2. OBJECTIFS DU PROJET :

- Gestion centralisée des formations : Créer un catalogue centralisé des formations et événements accessible à tous les utilisateurs, incluant les étudiants, l'équipe 4C, et l'administration.
- Inscription et suivi des participants : Simplifier le processus d'inscription et permettre aux participants de suivre facilement leurs inscriptions et formations.
- Alléger la gestion administrative : Automatiser et sécuriser le processus de gestion des formations pour augmenter l'efficacité administrative.
- Architecture DevOps : Mettre en place une architecture DevOps pour un déploiement continu et une gestion efficace de l'infrastructure.
- Outils d'évaluation et de feedback : Intégrer des outils permettant d'évaluer les formations en ligne pour en mesurer l'efficacité et améliorer la qualité.
- Reporting et analyses avancées : Fournir des rapports détaillés et des requêtes personnalisées sur les aspects quantitatifs et qualitatifs des formations et événements.
- Convivialité de la plateforme : Développer une interface conviviale pour les étudiants et les administrateurs.
- Sécurité des données : Assurer la sécurité des données et des transactions effectuées sur la plateforme.

3. PROBLEMATIQUE :

Actuellement, la gestion des inscriptions aux formations et événements au centre 4C ISET Tozeur est laborieuse et peu sécurisée. Les méthodes manuelles et les outils disparates utilisés entraînent des inefficacités, des erreurs et des retards dans le traitement des inscriptions. De plus, l'absence de centralisation complique la gestion et

le suivi des participants, et il est difficile de garantir la sécurité des données des étudiants.

4. SOLUTIONS PROPOSEES :

- Développement d'une plateforme intégrée : Créer une plateforme unique où toutes les formations et événements sont listés et où les étudiants peuvent s'inscrire en ligne de manière sécurisée.
- Authentification sécurisée : Mettre en place un système d'authentification utilisant le numéro de carte d'identité nationale (CIN) et un mot de passe, avec des options pour l'authentification multi-facteurs.
- Automatisation et gestion administrative : Développer des outils pour importer les étudiants via des fichiers CSV, gérer les formations (création, lecture, mise à jour, suppression), valider les inscriptions et notifier les étudiants par e-mail.
- Fonctionnalités avancées : Intégrer des fonctionnalités avancées pour les statistiques des formations, la classification des formations par thème et la génération de rapports détaillés sur les inscriptions et les participations.
- Architecture DevOps : Utiliser des technologies DevOps comme Docker, Kubernetes, et Jenkins pour assurer un déploiement continu, une gestion efficace de l'infrastructure et la scalabilité de la plateforme.
- Sécurité des données : Implémenter des mécanismes de sécurité robustes incluant le chiffrement des données sensibles, la gestion des accès et des autorisations, et la conformité aux normes de sécurité telles que le GDPR et le HIPAA.
- Interface conviviale : Développer une interface utilisateur intuitive et facile à utiliser pour les étudiants et les administrateurs, avec des guides et une documentation complète pour faciliter la prise en main.

5. SPECIFICATION DES BESOINS FONCTIONNELS :

- Authentification :

Les utilisateurs doivent pouvoir s'authentifier en utilisant leur numéro de carte d'identité nationale (CIN) et un mot de passe.

- Fonctionnalités pour les étudiants :
 - Consulter la liste des formations disponibles.
 - Participer à des formations.
 - S'inscrire à de nouvelles formations.
- Fonctionnalités pour les administrateurs :
 - Importer des étudiants dans la base de données à partir de fichiers CSV ou d'autres formats compatibles.
 - Gérer les formations (CRUD : création, lecture, mise à jour, suppression).
 - Afficher la liste des inscrits à chaque formation.
 - Valider les inscriptions des étudiants aux formations.
 - Consulter la liste des étudiants ayant suivi une formation après validation.
 - Notifier les étudiants par e-mail des confirmations d'inscription et d'autres informations importantes.
- Fonctionnalités avancées :
 - Rubrique statistique pour les formations, incluant des données sur le nombre d'inscriptions, les taux de participation, etc.
 - Liste des formations disponibles pour une période donnée.
 - Liste des formations classées par thème.
 - Liste des étudiants ayant suivi le plus grand nombre de formations, par sexe.

6. SPECIFICATION DES BESOINS NON FONCTIONNELS :

Les besoins non fonctionnels représentent les règles à respecter afin d'assurer une bonne qualité de l'application à achever le bon fonctionnement du futur système, parmi ses besoins nous allons noter :

- Ergonomie :
 - La plateforme doit être intuitive et facile à utiliser pour les étudiants et les administrateurs, même pour ceux ayant peu de connaissances informatiques.
 - L'interface utilisateur doit être claire, cohérente et esthétique.
 - La navigation doit être simple et fluide.
 - Les formulaires d'inscription et de gestion doivent être concis et faciles à remplir.

- Des messages d'aide et des tutoriels doivent être disponibles pour guider les utilisateurs.
- Sécurité :
 - Respecter les normes de sécurité les plus strictes pour protéger les données des utilisateurs.
 - Authentification robuste basée sur des mots de passe forts ou une authentification multi-facteurs.
 - Chiffrement des données sensibles en transit et au repos.
 - Mécanismes d'autorisation stricts pour contrôler l'accès à la plateforme.
- Documentation :
 - Fournir une documentation complète et à jour couvrant les aspects techniques et fonctionnels de la plateforme.
 - Documentation claire, concise et facile à comprendre, incluant des guides d'installation, des tutoriels, des exemples de code et des références à l'API.
 - Accessible aux utilisateurs finaux et aux administrateurs.
- Performance : La plateforme doit être performante et capable de gérer un grand nombre d'utilisateurs simultanément.
- Disponibilité : La plateforme doit être disponible 24 heures sur 24, 7 jours sur 7.
- Maintenabilité : La plateforme doit être facile à maintenir et à mettre à jour.
- Évolutivité : La plateforme doit être évolutive pour répondre aux besoins futurs.

IV. CONCLUSION

Ce chapitre a présenté le projet "Implémentation DevOps pour l'évolution du développement d'une application web de gestion de formations", en introduisant ITQAN, le contexte, les objectifs, la problématique, les solutions proposées et les besoins. Le projet vise à améliorer la gestion des formations au centre 4C ISET Tozeur grâce à une plateforme web et une approche DevOps, améliorant ainsi efficacité, sécurité et flexibilité. Le chapitre suivant discutera des concepts et principes

fondamentaux du DevOps et de leur impact sur le développement logiciel et l'infrastructure informatique.

CHAPITRE 2 :

CONCEPTS DE BASE

NOTION DEVOPS



I. INTRODUCTION :

La mise en œuvre de CI CD Pipeline ou le logiciel Continuous Integration/Continuous Déploiement est l'épine dorsale de l'environnement DevOps moderne. Vous pouvez trouver l'exigence de compétences en intégration continue et déploiement continu dans divers rôles tels que Data Engineer, Cloud Architect, Data Scientist, etc. CI/CD comble le fossé entre les équipes de développement et d'exploitation en automatisant la génération, le test et le déploiement d'applications. Dans ce blog, nous saurons Qu'est-ce que le pipeline CI CD et comment il fonctionne. Avant de passer au fonctionnement du pipeline CI CD, commençons par comprendre Dans cette partie, nous allons spécifier les besoins auxquels notre architecture doit répondre. Une telle analyse nous permet de définir les objectifs à atteindre et nous oriente sur les axes d'évaluation et de tests de notre architecture.

II. QU'EST-CE QUE SDLC ?

SDLC ou Software Development Life Cycle est la méthodologie cadre pour développer des logiciels de haute qualité, efficaces et efficaces, et les déployer dans les plus brefs délais. Le cadre SDLC implique des processus et des phases prédéfinis, et chaque étape de la phase SDLC produit les livrables requis pour la phase suivante du cycle de vie.

Un cycle de vie de développement logiciel typique comprend les phases suivantes



Figure 2 SDLC

Une approche dite "traditionnelle" attend généralement du client une expression

détaillée et validée du besoin en entrée de réalisation, laissant peu de place au changement.

La réalisation dure le temps qu'il faut et le rendez vous est repris avec le client pour la recette. Cet effet tunnel peut être très néfaste et conflictuel, on constate souvent un déphasage entre le besoin initial et l'application réalisée¹

III. POURQUOI DEVOPS :

DevOps est né de la nécessité de combler le fossé entre les équipes de développement et d'opérations dans le domaine du logiciel. Historiquement, les développeurs étaient axés sur la livraison rapide de nouvelles fonctionnalités, tandis que les équipes d'opérations se concentraient sur la stabilité et la disponibilité des systèmes. Cela créait une tension entre les deux, les développeurs souhaitant des déploiements fréquents et rapides, tandis que les Ops étaient réticents en raison des risques associés. DevOps aligne les objectifs en encourageant la collaboration et l'automatisation, permettant ainsi de livrer des fonctionnalités de manière continue tout en maintenant la stabilité et la fiabilité des systèmes.

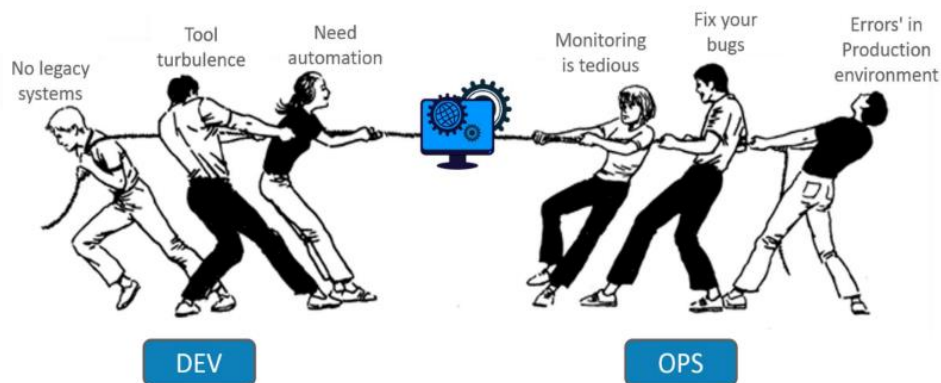


Figure 3 DEVELOPPEURS VS PERATIONS

¹ Cours monsieur haithem hafsi Culture devops

IV. QU'EST-CE QUE DEVOPS?

Le DevOps est une approche de collaboration entre le développement et l'informatique opérationnelle, visant à automatiser et à rendre reproductibles la production et le déploiement de logiciels. Cette méthode accélère la livraison d'applications et de services logiciels, permettant aux organisations de mieux servir leurs clients et de rester compétitives sur le marché. En bref, le DevOps se définit comme l'alignement du développement et de l'informatique opérationnelle grâce à une communication et une collaboration améliorée.



Figure 4 DEVELOPPEURS AVEC OPERATIONS

- L'objectif du DevOps est de casser cette barrière et le fonctionnement en silo. Cette collaboration passe par de nombreux moyens mais l'objectif est toujours le même : faire collaborer développeurs et Ops pour atteindre des objectifs communs.
- Combinant développement (Dev) et opérations (Ops), DevOps est l'union des personnes, des processus et des technologies destinés à fournir continuellement de la valeur aux clients.

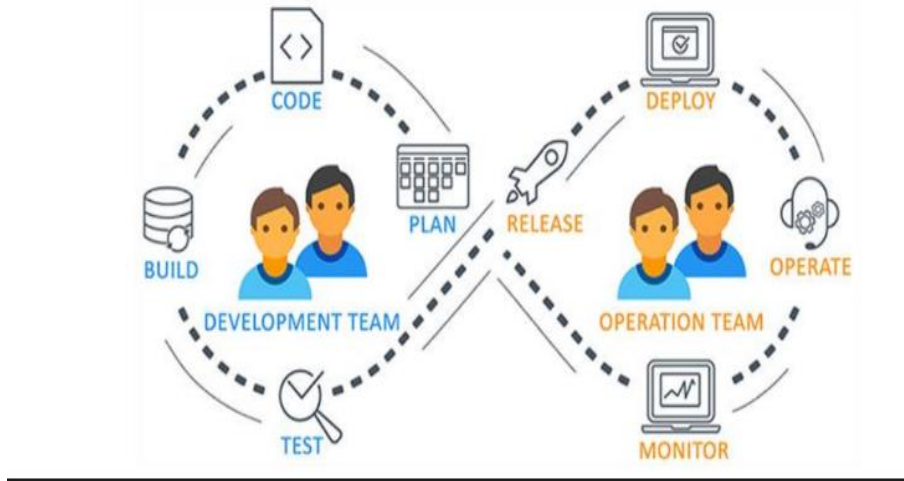


Figure 5: objectif DevOps

V. QUAND ADOPTER DEVOPS ?

DevOps, une approche qui combine le développement logiciel (Dev) et les opérations informatiques (Ops), est particulièrement utile pour certains types de projets et environnements. Voici une analyse détaillée des scénarios où l'adoption de DevOps est particulièrement avantageuse :

1. Grandes Applications Distribuées :

- Amélioration de la Coordination : Facilite la collaboration entre les équipes de développement et d'exploitation.
- Automatisation des Processus : Réduit les erreurs et augmente l'efficacité.
- Intégration et Livraison Continues (CI/CD) : Assure des déploiements rapides et fréquents.
- Sites de Commerce Électronique :
 - Scalabilité : Gère les pics de trafic efficacement.
 - Sécurité : Intègre les pratiques de sécurité dès le début du développement.
 - Amélioration de l'Expérience Utilisateur : Permet des mises à jour rapides et continues.

2. Applications Hébergées sur une Plateforme Cloud :

- Infrastructure en tant que Code (IaC) : Gère les ressources de manière cohérente et reproductible.
- Flexibilité : Exploite la flexibilité du cloud pour des déploiements rapides.
- Optimisation des Coûts : Utilise les ressources cloud de manière optimale pour réduire les coûts.

VI. Les principes essentiels de DevOps:

Les principes essentiels de DevOps sont fondamentaux pour son adoption réussie :

- Action centrée sur le client : Priorité aux actions visant à améliorer les produits et services pour répondre aux besoins des clients.
- Responsabilité de bout en bout : L'équipe DevOps assure un support complet depuis la conception jusqu'à la fin de vie des produits, renforçant ainsi la responsabilité et la qualité.
- Amélioration continue : Une culture de DevOps favorise l'amélioration constante pour réduire le gaspillage et accélérer l'évolution des produits ou services.
- Automatisation complète : L'automatisation est au cœur du processus DevOps, englobant non seulement le développement logiciel mais aussi l'ensemble de l'infrastructure.
- Collaboration d'équipe : Les membres de l'équipe, tels que les concepteurs, développeurs et testeurs, travaillent ensemble dans une collaboration totale.
- Surveillance et tests complets : Des procédures de surveillance et de test robustes sont nécessaires à une équipe DevOps pour garantir la qualité et la fiabilité des produits ou services.

VII. PIPELINE ET CHAÎNE D'OUTILS DEVOPS :

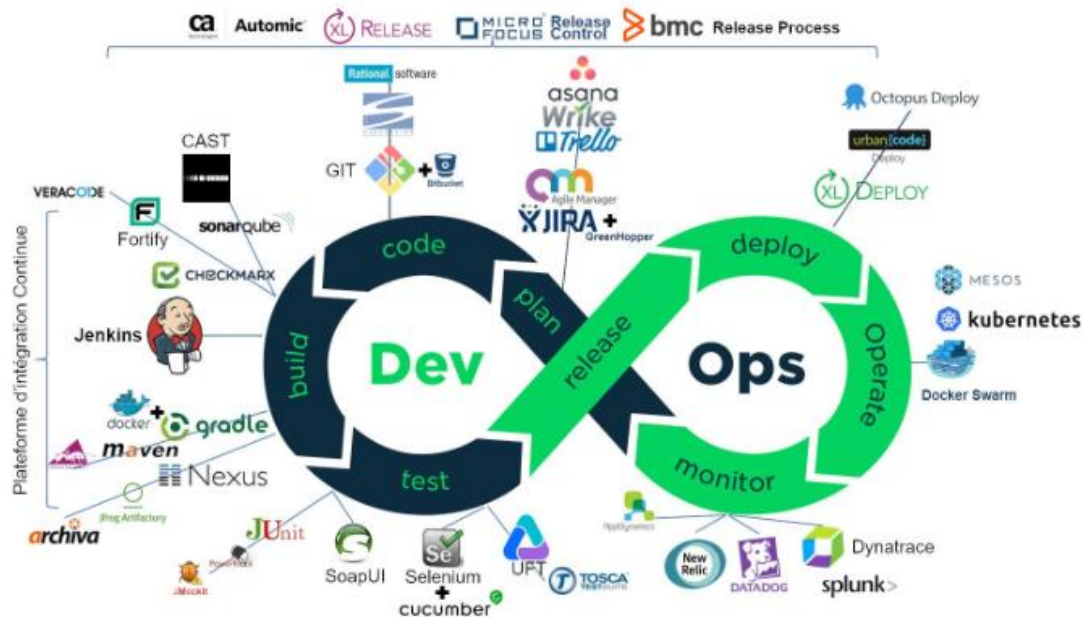


Figure 6: chaîne de pipeline

- Planifier : au début d'un projet, d'un sprint/itération ou d'une journée on fait la planification la hiérarchisation. Cela permet de s'assurer que tous les membres de l'équipe comprennent les objectifs actuels et de définir le travail attendu.
- Coder : les développeurs créent et soumettent des modifications ou des ajouts au code qui répondent aux tâches définies pour l'itération en cours. Cette étape comprend également la révision du code et les clarifications ou ajustements nécessaires.
- Build : compilation, test et mise en package du code .
- Tester : les builds réussis sont soumis à une série de tests unitaires pour garantir la fonctionnalité, la qualité et souvent la sécurité. Idéalement, les tests les plus rapides ou les plus critiques sont effectués en premier:
- si un test échoue, la modification du code à l'origine de l'échec peut être renvoyée aux développeurs instantanément, et moins d'efforts seront gaspillés.
- Si tous les tests réussissent, le build est transmis à la livraison et les modifications sont fusionnées dans la branche de code principale.

- Livrer : on conserve la dernière version réussie est conservée et mise à disposition pour le déploiement.
- Déployer : un build peut être déployé dans un environnement de test pour des tests supplémentaires (ex : tests d'acceptation par l'utilisateur). Il peut également être livré à des environnements de production ou de mise à disposition pour le déploiement.
- Opérer/configurer l'infrastructure : Les Ops construisent ou maintiennent une infrastructure évolutive, en utilisant l'IaC (Infrastructure as a Code) et vérifient les problèmes de sécurité en plus de la gestion des journaux.
- Monitoring : après la mise en production, l'application est contrôlée pour intervenir en cas de problèmes détectés.

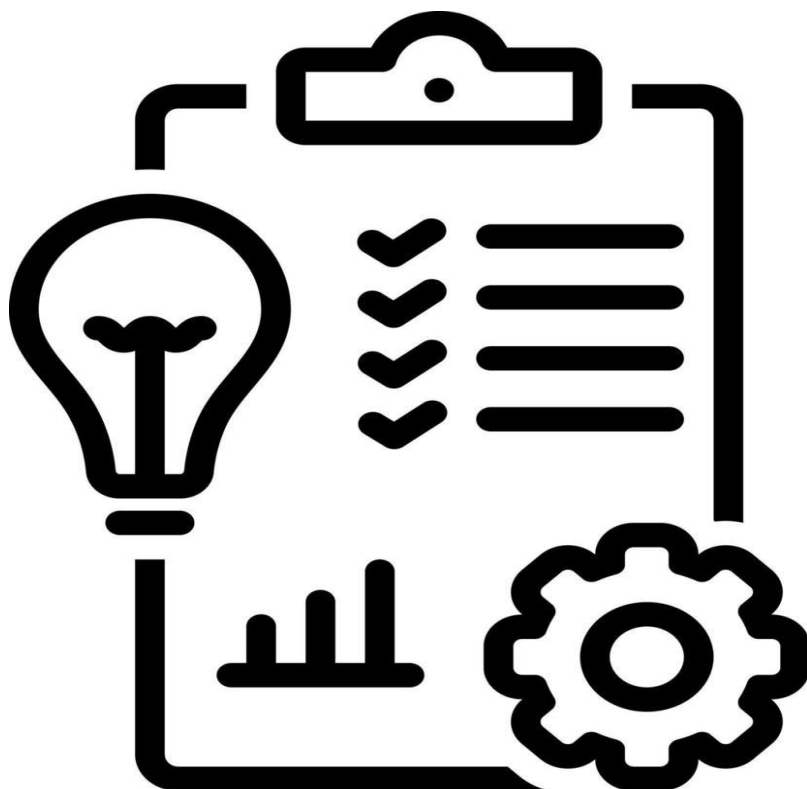
VIII. Conclusion

DevOps automatise les processus entre les équipes de développement et IT, accélérant et fiabilisant le développement, les tests et les livraisons de logiciels. Cette approche favorise la collaboration, améliore la confiance et assure une meilleure qualité des produits.

Le CI/CD en DevOps utilise des pipelines automatisés pour l'intégration, les tests et le déploiement continus. Les tendances incluent l'automatisation, l'Infrastructure as Code, la conteneurisation, la surveillance avancée et l'intégration de la sécurité (DevSecOps). Ces pratiques facilitent des livraisons agiles, améliorent la qualité et la fiabilité des applications et renforcent la collaboration entre équipes.

CHAPITRE 3 :

ETAT DE L'ART



I. INTRODUCTION :

Dans le domaine du développement logiciel, les pratiques et technologies évoluent rapidement pour répondre aux besoins croissants en termes d'efficacité, de qualité et de rapidité de livraison. L'un des paradigmes les plus influents de ces dernières années est le DevOps, qui vise à unifier le développement et les opérations pour améliorer la collaboration et la productivité. Au cœur de cette philosophie se trouvent les pratiques de CI/CD (Continuous Integration/Continuous Delivery et/ou Continuous Deployment) ainsi que des architectures modernes telles que les microservices. Ce chapitre explore l'état de l'art de ces technologies et pratiques, en mettant en lumière leurs avantages, leurs outils associés et leurs impacts sur la gestion du cycle de vie des applications.

II. CI/CD : QU'EST-CE QUE C'EST ?

CI/CD, pour continuous integration / continuous delivery et/ou continuous deployment est aujourd'hui une pratique courante dans le monde du développement logiciel. Outil DevOps, il permet de livrer/déployer de manière continue et automatisée un produit, et de simplifier la relation entre le monde du développement et celui de l'exploitation.

Pour rentrer un peu plus dans les détails, le CI/CD permet de builder, tester, livrer et/ou de déployer de manière automatique un produit développé. Cela fait donc le lien entre les équipes de développement et d'exploitation, chargée respectivement de créer le produit, et d'exploiter et de déployer ce produit. C'est en cela une pratique DevOps

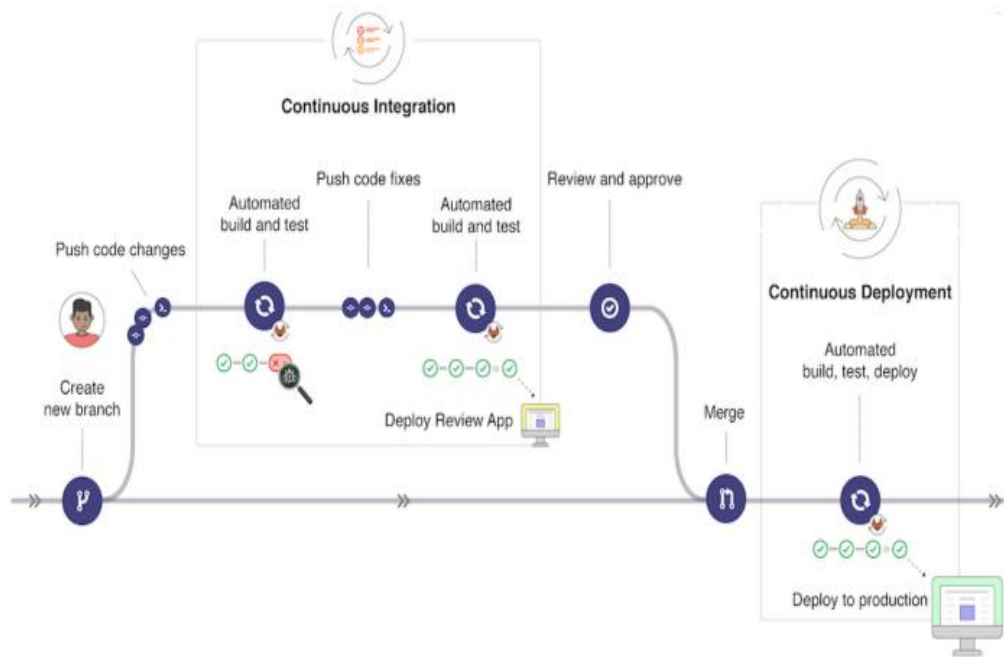


Figure 7: description CI CD

1. L'INTEGRATION CONTINUE :

L'intégration continue consiste à commit et merge le code des développeurs sur une même branche le plus régulièrement possible. L'idée est ici de s'assurer que la fusion des codes ne provoque pas de breaking changes, en forçant les merges une ou plusieurs fois par jour.

Le but du CI est de se rendre compte au plus tôt lorsqu'il y a bug, des régressions ou des conflits, et ainsi de pouvoir les résoudre au plus vite pour éviter la dette technique.

L'intégration continue comprend, et c'est là aussi un point clé, une phase de tests en amont. Nous pourrions découper le workflow de la façon suivante :

- Les tests (d'intégration, notamment) sont lancés en local sur la machine du développeur avant ou au moment du commit ;
- Le code est compilé sur le serveur, ce qui permet de détecter les éventuelles erreurs de merge ;
- Le code est de nouveau soumis aux tests sur le serveur pour s'assurer qu'il n'y ait pas de problème.

➤ DES OUTILS D'INTEGRATION CONTINUE :

Les outils d'intégration continue les plus couramment utilisés incluent Jenkins, GitLab CI, Travis CI, CircleCI et Bamboo.

Dans le cadre de mon projet, j'ai choisi Jenkins comme outil d'intégration continue. Jenkins a été sélectionné en raison de sa flexibilité, de sa robustesse et de sa large adoption dans l'industrie du développement logiciel. Que voir dans chapitre 5 réalisation



Figure 8 : les outils CI

2. LE DEPLOIEMENT CONTINU :

Le déploiement continu, ou continuous deployment, est très proche de la livraison continue : on crée là aussi un livrable. La différence, c'est que le déploiement est cette fois automatique. En reprenant notre exemple d'application Android, nous pourrions imaginer qu'elle soit uploadée et envoyée aux bêta testeurs sans action humaine.

➤ Des outils déploiement continu:

Voici une liste de certains outils de Continuous Delivery (livraison continue) et de déploiement continu (CD) :



Figure 9: les outils CD

3. LES AVANTAGES DU CI/CD :

Maintenant que nous savons ce qu'est, dans le principe, le CI/CD, parlons un peu des avantages et inconvénients de cette méthode DevOps.

❖ Les avantages de la méthode

Si le concept de CI/CD convainc de plus en plus d'équipes de développement, ce n'est pas pour rien ! Cette méthode présente en effet des avantages certains :

- Les bugs et régressions sont détectés presque immédiatement ;
- Une version prête à l'emploi et à jour est disponible sans interruption ;
- Les utilisateurs finaux ayant accès très régulièrement aux dernières versions, ils peuvent faire des feedbacks en temps réel.

En cela, l'utilisation de CI/CD rend le développement plus performant, et les cycles de construction/release plus rapides, plus sûrs.

Tous ces éléments, notamment la mise en place du pipeline et le développement des tests, font que le CI/CD peut être difficile à intégrer. Certaines entreprises ont des contraintes budgétaires et/ou temporelles qui limitent l'utilisation de ces méthodes (bien que sur le long terme, elles peuvent faire gagner du temps et de l'argent).

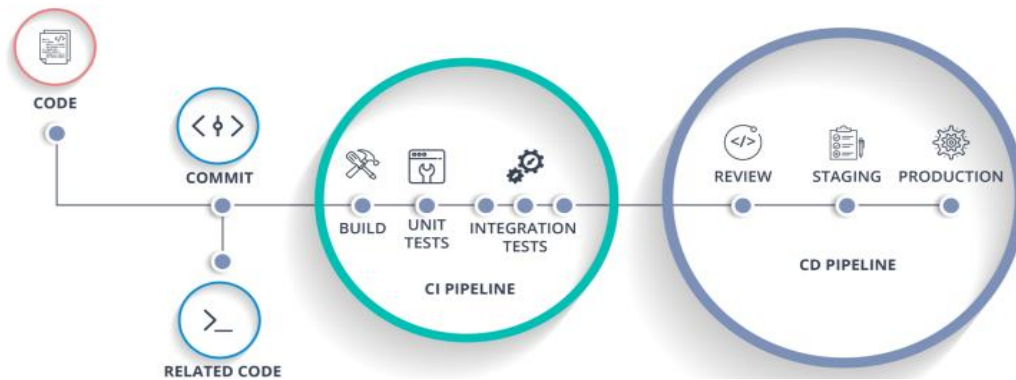


Figure 10: pipeline CI CD

III. CONTROLE DU CODE SOURCE / BUILD :

La gestion du code source est le processus de suivi des modifications apportées au code source, permettant aux développeurs de conserver un historique de ces modifications. Avant son introduction, diverses approches étaient utilisées pour éviter les conflits, telles que la communication entre les développeurs pour coordonner les travaux sur des fonctionnalités spécifiques. Cependant, ces méthodes ne permettaient pas de suivre efficacement les modifications et rendaient difficile la correction des bugs.

Les outils modernes de gestion du code source intègrent souvent le concept de ramification, permettant aux développeurs de créer des branches pour travailler sur des fonctionnalités distinctes. Par exemple, un développeur peut travailler sur la "page d'atterrissage" tandis qu'un autre travaille sur le "tableau de bord de l'utilisateur". Chaque développeur travaille sur sa branche dédiée et fusionne son code dans la branche principale après avoir testé que les fonctionnalités fonctionnent comme prévu..²

- Pourquoi la gestion du code source est-elle essentielle ?
 - Amélioration de la collaboration
 - Ils donnent une version complète de l'historique
 - Aide à l'automatisation du flux de travail

² <https://geekflare.com/fr/source-code-management-tools/>

- Sauvegarde du code
- Amélioration de la communication

1. VERSION CONTROL SYSTEMS

- (SCM – Source Code Management ou VCS – Version Control Systems)
- Le contrôle du code source est un moyen pour garder trace et historiser les changements apportés à un code source et de collaborer sur ce dernier.
- C'est un élément essentiel pour le développement en général et indispensable pour le DevOps en particulier. Le contrôle du code source permet d'avancer rapidement et intelligemment. Il protège notamment le code source des dommages soudains, des erreurs humaines et des différents conflits
- Git est un contrôleur de code source gratuit et open source. Il est nettement plus rapide que beaucoup d'autres logiciels de contrôle du code source.
- Git est utilisé souvent avec des services d'hébergement de répertoire. Ce sont des serveurs qui exécutent Git. Ils permettent de suivre et de partager vos projets de contrôle de version Git en dehors de votre ordinateur/serveur local.
- Les plus populaires sont : GitHub, GitLab, Beanstalk, AWS CodeCommit et Bitbucket.



Figure 11: les outils de version de controle

2. ARTIFACT REPOSITORIES

- Maven et Gradle sont deux outils de Build des Artifacts



Figure 12: les outils de build des artrfacts

- Git est dédié au fichier texte, dans ce cas il n'est pas la meilleure option. Il existe des outils plus spécifiques appelés Artifact Repositories.
- Ces outils stockent les artefacts binaires avec les métadonnées qui les décrivent et quelques informations supplémentaires comme les dépendances, leurs versions, etc.

Exemple d'Artifact Repositories : Sonatype Nexus, JFrog Artifactory, npm



Figure 13:LES OUTILS D D'ARTEFACT REPOSITORIES

IV. ARCHITECTURE DES MICROSERVICES :

L'architecture des microservices est un style architectural qui crée des applications en décomposant celles-ci en services indépendants plus petits, chacun étant responsable de ses propres tâches. Contrairement à l'architecture monolithique où toutes les fonctionnalités sont développées dans une seule pile technologique nécessitant une coordination étroite entre les développeurs, l'architecture des microservices permet à chaque service d'utiliser une pile technologique différente. Cela permet aux développeurs de se concentrer sur leurs propres microservices sans affecter le code des autres.

Les microservices offrent plusieurs avantages, notamment la facilité d'évoluer en fonction du trafic, la possibilité de déployer des services sans affecter les autres, et une meilleure isolation des composants. Dans une architecture monolithique, tous les composants partagent les mêmes ressources hôtes, ce qui peut entraîner des

ralentissements ou des plantages si l'un des composants est surchargé. Toutefois, l'architecture des microservices présente également des inconvénients, comme la gestion complexe due à l'augmentation exponentielle du nombre de services.

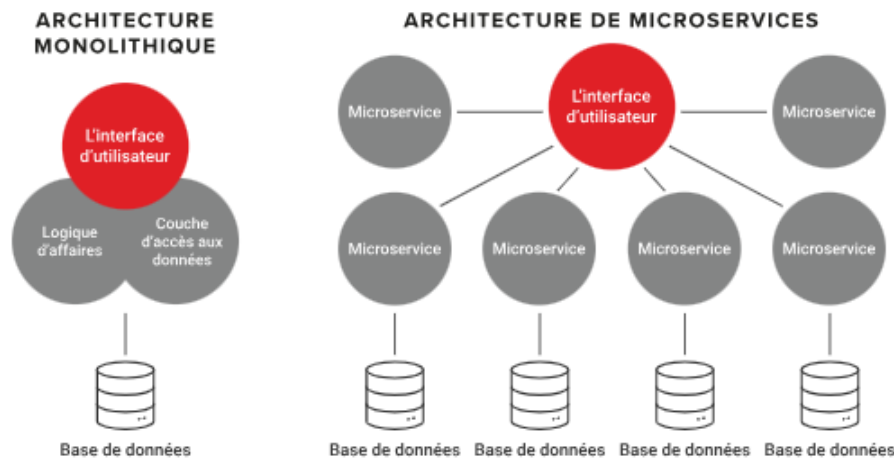


Figure 14: ARCHITECTURE MONOLITHIQUE VS ARCHITECTURE DE MICROSERVICES

V. CONCEPTION D'ARCHITECTURE MICROSERVICE

Les microservices sont un style architectural répandu pour générer des applications résilientes, hautement scalables, capables d'évoluer rapidement et pouvant être déployées indépendamment. Mais une architecture de microservices réussie nécessite une approche différente pour concevoir et générer des applications.

Une architecture de microservices se compose d'un ensemble de petits services autonomes. Chaque service est autonome et doit implémenter une fonctionnalité unique dans un contexte borné. Un contexte borné est une division naturelle au sein d'une entreprise qui fournit une délimitation explicite dans laquelle un modèle de domaine existe.

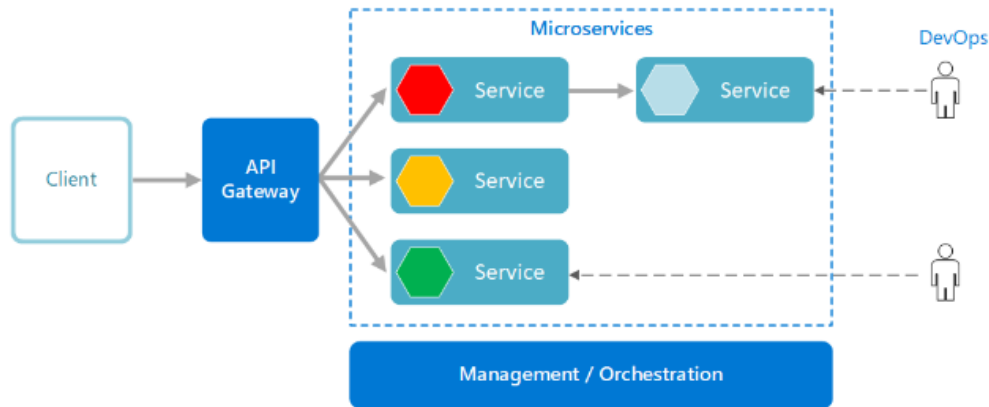


Figure 15: ARCHITECTURE DE MICROSERVICES

1. QUE SONT LES MICROSERVICES ?

- Les microservices sont de petite taille, indépendants et faiblement couplés. Une équipe réduite de développeurs peut écrire un service et en assurer la maintenance.
- Chaque service est un code base distinct, qui peut être géré par une petite équipe de développement.
- Les services peuvent être déployés de manière indépendante. Une équipe peut mettre à jour un service existant sans avoir à recréer et redéployer l'application entière.
- Les services sont responsables de la persistance de leurs propres données ou de leur propre état externe. Ils se distinguent en cela du modèle traditionnel dans lequel une couche de données distincte gère la persistance des données.
- Les services communiquent entre eux à l'aide d'API bien définies. Les détails de l'implémentation interne de chaque service sont cachés aux autres services.
- Prend en charge la programmation polyglotte. Par exemple, les services n'ont pas besoin de partager une pile de technologies, des bibliothèques ou des frameworks identiques.

Outre les services proprement dits, une architecture de microservices type est constituée d'autres composants :

- Gestion/orchestration. Ce composant est chargé de placer les services sur des nœuds, d'identifier les défaillances, de rééquilibrer les services entre les nœuds, etc. En général, ce composant est une technologie prête à l'emploi comme Kubernetes, plutôt qu'un élément personnalisé.
- Passerelle d'API : la passerelle d'API est le point d'entrée des clients. Au lieu d'appeler directement les services, les clients appellent la passerelle d'API, qui transfère l'appel aux services appropriés sur le back-end.

L'utilisation de la passerelle d'API offre les avantages suivants :

- Il dissocie les clients des services. Les services peuvent être versionnés ou refactorisés sans avoir à mettre à jour tous les clients.
- Les services peuvent utiliser des protocoles de messagerie qui ne sont pas compatibles avec le web, comme AMQP.
- La passerelle d'API peut exécuter d'autres fonctions transversales, telles que l'authentification, la journalisation, la terminaison SSL et l'équilibrage de charge.
- Stratégies prêtes à l'emploi, telles que la limitation, la mise en cache, la transformation ou la validation.

VI. AVANTAGES DES MICROSERVICES

- Agilité :

- Déploiement indépendant des microservices, facilitant la gestion des correctifs et nouvelles fonctionnalités.
- Permet des mises à jour sans redéployer toute l'application, avec la possibilité de restaurer en cas de problème.
- Évite les blocages en production causés par des bogues dans les applications traditionnelles.

Équipes spécialisées de taille restreinte :

- Un microservice est assez petit pour être géré par une seule équipe, favorisant la flexibilité.

- Les petites équipes sont plus productives grâce à une communication plus rapide et moins de gestion.
- Base de code de petite taille :
 - Minimisation des dépendances de code, facilitant l'ajout de nouvelles fonctionnalités sans modifications étendues.
 - Chaque microservice gère son propre code et ses propres données, réduisant les interdépendances.
- Pot-pourri de technologies :
 - Les équipes peuvent choisir la technologie la mieux adaptée à chaque service, utilisant diverses piles technologiques.
- Isolation des erreurs :
 - Les défaillances d'un microservice individuel n'affectent pas l'ensemble de l'application.
 - Utilisation de modèles comme le disjoncteur et la communication asynchrone pour gérer les défaillances.
- Extensibilité :
 - Mise à l'échelle indépendante des services, permettant de distribuer les ressources efficacement.
 - Utilisation d'orchestrateurs comme Kubernetes pour optimiser l'utilisation des ressources.
- Isolation des données :
 - Mises à jour de schéma facilitées, ne concernant qu'un seul microservice.
 - Réduction des risques associés aux modifications de schéma dans les applications monolithiques où les données sont partagées..

VII. TEST CONTINU :

Les tests continus sont cruciaux pour le développement et la maintenance des applications, garantissant que le logiciel fonctionne correctement et répond aux exigences de performance, de fiabilité et de stabilité. Cette stratégie permet aux équipes de développement d'identifier et de corriger les problèmes précocement, réduisant ainsi les risques de défaillance en production.

Les tests continus se divisent en trois catégories principales, chacune utilisant des outils spécifiques pour garantir la qualité du logiciel :

- Tests fonctionnels : Vérifient que chaque fonctionnalité du logiciel fonctionne comme prévu. Utilisent JUnit pour les tests unitaires et Selenium pour les tests d'acceptation et d'intégration.
- Tests non fonctionnels : Évaluent les aspects tels que la performance et la sécurité. Emploient JMeter pour évaluer les performances et Snort pour tester la sécurité.

Tests de maintenance : Assurent la qualité continue lors des mises à jour et modifications du logiciel. Utilisent Cucumber pour rédiger des scénarios de tests en langage naturel, vérifiant que les modifications n'introduisent pas de nouvelles erreurs.



Figure 16: LES OUTILS DE TEST CONTENU

VIII. CLOUD ET CONTENEURISATION :

1. Cloud Infrastructures

Avec l'essor de l'informatique en nuage, nous pouvons gérer des services à distance, fournis par un fournisseur de cloud, sans avoir à nous soucier de la maintenance du matériel et des centres de données.

Nous pouvons facilement adapter nos ressources en fonction de la charge de travail, voire l'automatiser, et garantir une haute disponibilité ainsi qu'une tolérance aux pannes.

Dans le cloud computing, les fournisseurs de cloud et les clients partagent la responsabilité de la gestion de leurs ressources. Ce modèle est appelé la responsabilité partagée, qui varie selon le type de service.

Ces services sont IaaS (Infrastructure as a Service), PaaS (Platform as a Service) et SaaS (Software as a Service). Et au fur et à mesure que nous passons d'un service à un autre, le fournisseur de cloud joue un rôle plus important dans la gestion de notre service.

Les cloud provider qui ont la plus grande part du marché sont : AWS (Amazon Web Services), Microsoft Azure, GCP (Google cloud Provider)



Figure 17: LES OUTILS DE CLOUD

2. Conteneurisation

Si nous utilisons une architecture de microservices, il est très probable que nous adoptions la conteneurisation, surtout dans les applications modernes.

La conteneurisation est le processus qui consiste à regrouper le code source de l'application et toutes les dépendances nécessaires dans un conteneur, de sorte qu'elles puissent fonctionner dans n'importe quel environnement.

En ce qui concerne les avantages des conteneurs par rapport aux machines virtuelles, les conteneurs sont réputés pour être légers car ils partagent le noyau du système d'exploitation de l'hôte, ce qui les rend moins gourmands en CPU et en RAM que les machines virtuelles.

Ils offrent également une isolation. Les conteneurs sont isolés les uns des autres, de sorte que tout problème ou vulnérabilité dans un conteneur n'affectera pas les autres.

Bien que les fichiers de machines virtuelles et les fichiers de conteneurs soient tous deux portables et puissent être déplacés vers n'importe quelle machine, la principale différence réside dans leur taille. Les fichiers de conteneurs sont beaucoup plus légers, ce qui les rend plus rapides à déplacer.

- Technologies : Docker (leader absolue), Rkt, LXD



Figure 18: LES OUTILS DE CONTENEURISATION

3. Orchestration des conteneurs

À mesure que le nombre de conteneurs augmente, leur gestion devient difficile et exige un effort considérable. Il est donc nécessaire de trouver un moyen d'automatiser ce travail, et c'est là que l'orchestration des conteneurs intervient.

L'orchestration des conteneurs nous aide à automatiser le cycle de vie des conteneurs. Elle offre diverses fonctionnalités telles que l'automatisation des déploiements, la mise à l'échelle et les capacités d'auto-réparation.

Solutions : Kubernetes, Docker Swarm, Nomad, Apache Mesos.



Figure 19: LES OUTILS DE CONTENEUR

IX. OUTILS DE GESTION DE CONFIGURATION ET AUTOMATISATION :

L'automatisation permet d'éliminer les tâches répétitives des équipes Devops.

Plusieurs types d'automatisation en Devops existent :

- Mettre en place des configurations automatiques sur les serveurs
- Automatiser les actions des serveurs

Plusieurs outils existent en fonction de l'infrastructure existante et des besoins de l'entreprise :

- Terraform : provisioning d'infrastructure ;
- Ansible : gestion de la configuration des serveurs esclaves ;
- Puppet : gestion de la configuration des serveurs esclaves ;³



Figure 20: LES OUTILS D'AUTOMATISATION

X. MONITORING ET ALERTING :

Les outils de monitoring et alerting permettent d'avoir une vue d'ensemble sur son infrastructure, de résoudre les problèmes qui surviennent et d'améliorer les performances.

L'application open source Prometheus et le service Grafana permettent de monitorer les clusters Kubernetes. En couplant trois outils, ELK (Elasticsearch, Logstash et Kibana) est une solution d'analyse de logs performante. On peut jouer sur les performances de chaque outil individuellement et les adapter à ses besoins : Logstash pour la normalisation/envoi des logs, Elasticsearch pour le stockage et Kibana pour la visualisation. ELK permet de faire de l'analyse de logs (forensics) et de l'agrégation (dashboard).

- Outils de monitoring : Sensu, Splunk, Prometheus, Grafana, ELK, Nagios, Cacti

³ https://www.padok.fr/blog/outils-devops#Automatisation_et_gestion_de_configuration



Figure 21: LES OUTILS MONITORING

XI. Conclusion :

Le chapitre 3 explore les fondements de DevOps en mettant l'accent sur CI/CD, les microservices, la conteneurisation et les outils de monitoring. CI/CD permet une livraison continue automatisée, tandis que les microservices offrent une architecture flexible. La conteneurisation simplifie le déploiement avec Docker, et les outils comme Prometheus assurent une surveillance efficace des performances.

Ces pratiques et technologies renforcent la flexibilité, la rapidité et l'efficacité du développement logiciel, malgré les défis qu'elles impliquent. En les adoptant, les entreprises peuvent améliorer leur collaboration interne et répondre plus efficacement aux exigences du marché.

Le prochain chapitre se concentrera sur les méthodologies agiles, complétant ainsi l'approche DevOps pour optimiser les cycles de développement.

Chapitre 4 : Méthodes Agiles pour la Gestion de Projet



I. Introduction :

Dans le monde dynamique d'aujourd'hui, les entreprises et les organisations sont constamment à la recherche de moyens plus efficaces pour gérer leurs projets. Les méthodologies agiles, telles que Scrum et Kanban, ont gagné en popularité ces dernières années, offrant une approche flexible et adaptative pour la gestion de projets complexes.

Ce chapitre se concentrera sur Scrum, une méthodologie agile largement utilisée, et explorera son intégration avec Jira, un outil de gestion de projet puissant. Nous examinerons les rôles, les artefacts et les événements clés de Scrum, ainsi que les avantages de son utilisation avec Jira.

II. Principes de l'Agilité :

- Manifeste Agile :
 - Individus et interactions plutôt que processus et outils
 - Logiciel fonctionnel plutôt que documentation exhaustive
 - Collaboration avec le client plutôt que négociation de contrat
 - Réponse au changement plutôt que suivi d'un plan
- Valeurs et principes agiles :
 - Réactivité : Capacité à s'adapter rapidement aux changements
 - Collaboration : Travail en équipe et avec les parties prenantes
 - Amélioration continue : Évaluation régulière et amélioration des processus et produits

III. Méthodologie Scrum :

Scrum est un cadre de gestion de projet agile qui favorise une approche itérative et incrémentale pour le développement de produits. Il est basé sur l'idée de diviser le projet en courtes périodes de travail appelées sprints, d'une durée généralement de 1 à 4 semaines. À la fin de chaque sprint, une version fonctionnelle du produit est livrée, permettant aux parties prenantes de fournir des commentaires et d'orienter le développement futur.

- Rôles Scrum

Scrum définit trois rôles clés :

- Scrum Master : Facilite les processus Scrum, supprime les obstacles
- Product Owner : Responsable de la maximisation de la valeur du produit
- Équipe de développement : Groupe auto-organisé qui crée l'incrément de produit

- Événements Scrum :

Scrum utilise également trois artefacts principaux pour suivre l'avancement et la transparence du projet :

- Sprint : Cycle de développement de 2 à 4 semaines
- Daily Stand-up : Réunion quotidienne pour synchroniser les activités
- Sprint Review : Réunion à la fin du Sprint pour examiner l'incrément de produit
- Sprint Retrospective : Réunion pour évaluer et améliorer le processus de travail

- Artefacts Scrum :

- Product Backlog : Liste ordonnée des besoins du produit
- Sprint Backlog : Ensemble des éléments à développer pendant le Sprint
- Increment : Version utilisable et potentiellement livrable du produit

IV. AVANTAGES DE SCRUM POUR LA GESTION DE PROJET :

- Amélioration de la communication et de la collaboration: Scrum favorise une communication ouverte et transparente entre tous les membres de l'équipe et les parties prenantes.
- Adaptabilité et flexibilité: Scrum permet de s'adapter rapidement aux changements de priorités ou aux nouveaux besoins.
- Délivraison rapide de valeur: Scrum permet de livrer des fonctionnalités fonctionnelles et précieuses fréquemment.

- Amélioration de la qualité du produit: Scrum met l'accent sur la livraison d'incréments de produit de haute qualité.
- Augmentation de la motivation et de l'engagement de l'équipe: Scrum favorise un environnement de travail collaboratif et responsabilisant.

V. APPLICATION DE SCRUM DANS LE PROJET :

- Mise en place des Sprints pour le développement de la plateforme :
 - Planification des Sprints pour structurer le développement
- Gestion des tâches et des priorités via les User Stories :
 - Création et gestion des User Stories pour définir les fonctionnalités et les priorités
- Collaboration et communication au sein de l'équipe :
 - Utilisation des événements Scrum pour assurer une communication et une collaboration efficaces

VI. UTILISATION DE JIRA DANS LE PROJET DE GESTION DES INSCRIPTIONS :

Dans le cadre du projet de développement de la plateforme de gestion des inscriptions pour le centre 4C ISET Tozeur, l'utilisation de Jira s'est avérée être un pilier crucial pour garantir une gestion efficace et transparente du projet. En exploitant les fonctionnalités de Jira, j'ai pu structurer le travail, suivre l'avancement des tâches et des Sprints, et favoriser une communication claire et continue au sein de l'équipe.

-Création et Gestion des User Stories, Tâches et Bugs

-Suivi de l'Avancement des Sprints :

-Utilisation des Tableaux Kanban et Scrum

-Gestion des Tâches Quotidiennes

-Reporting et Analyse des Performances

En somme, l'utilisation de Jira a été essentielle pour assurer la réussite du projet de gestion des inscriptions. Cet outil m'a permis de gérer efficacement le travail, de suivre l'avancement du projet et d'encourager une collaboration transparente et productive au sein de l'équipe.

VII. SPECIFICATION DES BESOINS :

1. IDENTIFICATION DES ACTEURS

Le système de gestion des inscriptions aux formations et événements du centre 4C ISET Tozeur dispose de deux types d'acteurs principaux : les administrateurs et les étudiants, subdivisés en étudiants inscrits et étudiants passagers.

TABLEAU 1 : LISTE DES ACTEURS

Acteurs	Rôles	Interactions
Administrateur 	- Gérer les formations (création, mise à jour, suppression)	- Utilise principalement l'interface back-office pour gérer les contenus (formations) et les utilisateurs (étudiants)
	- Valider les inscriptions des étudiants	- Accède aux outils de reporting et d'analyse pour une gestion efficace et centralisée
	- Consulter les statistiques des formations et des participants	- Envoie des notifications par e-mail aux étudiants
	- S'inscrire aux formations proposées	- Accède à l'interface front-office pour parcourir le catalogue des formations disponibles, s'inscrire à celles

Étudiant Inscrit 		de son choix, et suivre les formations
	- Participer activement aux formations	- Peut consulter ses inscriptions actuelles et son historique de formation via cette interface conviviale
	- Consulter l'historique des formations suivies pour suivre son parcours et ses progrès	
Étudiant Passager 	- Consulter les formations disponibles sans possibilité d'inscription ou de participation	- Utilise l'interface front-office pour parcourir et consulter la liste des formations disponibles. Cette interaction est purement informative, permettant aux étudiants de découvrir les formations.

Les acteurs et les cas utilisation sont résumés dans le tableau suivant

TABLEAU 2 : IDENTIFICATION DES ACTEURS ET DES CAS UTILISATIONS

Acteurs	Cas d'utilisation
Administrateur	Gérer les formations
	Gérer les événements
	Gérer les inscriptions
	Gérer les participations
Étudiant Inscrit	Gérer profil
	Se désabonner
	Consulter les formations
	Participer à une formation
	Consulter les événements
	Contacter l'administrateur
Étudiant Passager	S'inscrire
	Consulter les formations
	Consulter les événements
	Contacter l'administrateur

2. MODELISATION DES BESOINS

J'ai donc identifié nos acteurs. Maintenant, il s'agit de définir plus en détail le besoin de chaque acteur en répondant à la question : QUI devra faire QUOI ? Le diagramme de cas d'utilisation représente les fonctionnalités du système, c'est-à-dire l'ensemble des actions que devront réaliser nos acteurs.

a. DIAGRAMME DE CAS UTILISATION GLOBAL

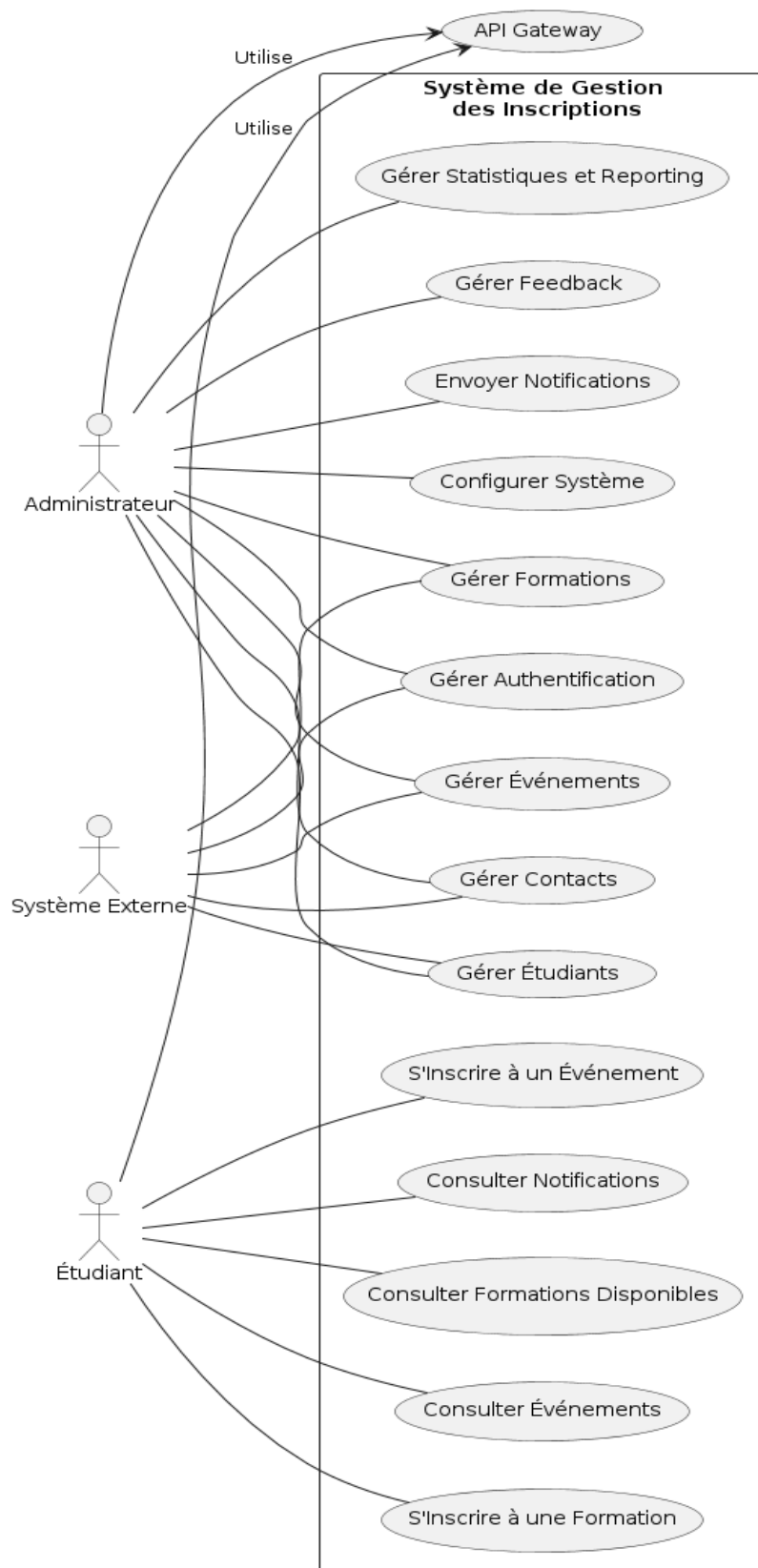


Figure 22:DIAGRAMME DE CAS UTILISATION GLOBAL

Ces cas d'utilisation décrivent les interactions principales entre les étudiants et la plateforme, aidant à définir les fonctionnalités essentielles pour répondre aux besoins des administrateurs et des étudiants.

b. Diagramme de séquence global voir annexe

VIII. BACKLOG DE PRODUIT :

Le Backlog de produit correspond à une liste priorisée des besoins et des exigences du produit. Les éléments du Backlog de produit, appelé aussi les histoires utilisateurs, sont formulés en une ou deux phrases décrivant de manière claire et précises la fonctionnalité désirée par l'utilisateur, généralement, écrit sous la forme suivante : en tant que X, je veux Y, afin de Z.

Dans ce tableau, je présente les histoires utilisateurs classées par priorité et les détails associés. Les besoins et exigences du produit sont listés selon un ordre croissant de priorité.

- ✓ Identifiant : c'est un nombre unique et auto-incrémenté pour chaque User Story.
- ✓ Priorité : c'est l'importance attribuée par le Product Owner à cette tâche. Pour que le tableau soit lisible et clair, nous allons nommer la "très haute priorité" par l'abréviation "TH", la "haute priorité" par l'abréviation "H", la "moyenne priorité" par l'abréviation "M", et la "basse priorité" par l'abréviation "B".
- ✓ Tâches : la tâche à réaliser.
- ✓ Estimation : la durée nécessaire pour réaliser une tâche (en jours).
- ✓ Sprint Assigné : le sprint au cours duquel la tâche est planifiée pour être réalisée.

TABLEAU 3 : PRODUCT BACKLOG

SPRINTS	ID	USER STORY	PRIORITE	STORY POINT (j)
SPRINT 0 1 mois	1	En tant qu'administrateur du projet, je dois m'assurer que l'environnement de développement est correctement configuré.	Haute	7j
	2	En tant qu'administrateur, je dois suivre une formation sur Node.js pour acquérir les compétences nécessaires au développement de la plateforme.	Haute	30j
	3	En tant qu'administrateur je veux rechercher et développer sur la culture, l'historique et les outils DevOps.	Moyenne	4j
	4	En tant qu'administrateur, je veux préparer cahier de charge et fixer	Haute	3j

		architecture micro services		
	5	En tant que admin je veux créer base des données avec MYSQL	Moyenne	2
Sprint 1	6	En tant que admin je veux développer microservices 1 gestion des étudiants CRUD	Haute	7
	7	En tant que admin je veux développer services authentification	Haute	7
	8	En tant que admin je veux créer interface pour admin et l'étudiant avec angular	Haute	7
	9	En tant que admin je veux créer interface des consulter les information pour les étudiants et les interface pour admin et push a GitHub par version	Moyenne	4
	10	En tant que admin je veux faire test unitaire JTEST et SONARQUEB lorsque termine micro	Moyenne	3
Sprint 2	11	En tant que admin je veux développer micro2 gestion des formations	Haute	4

	12	En tant que admin je veux préparer interface pour admin pour gérer les formations	Haute	2
	13	En tant que admin je veux créer interface pour consulter les informations	Moyenne	2
	14	En tant que admin je veux faire gestion des événements	Moyenne	2
	15	En tant qu'Administrateur je veux faire les interface des consulter événements et gérer événements pour admin	Moyenne	2
	16	En tant que admin je veux mettre branche a GitHub	Faible	1
	17	En tant que admin je veux faire test unitaire JTEST et SONARQUEB lorsque termine micro	Faible	3
	18	En tant que admin je veux créer microservices de contact	Faible	2
	19	En tant que admin je veux faire interface contact	Faible	2

Sprint 3	20	En tant que admin je veux corriger les erreur dans partie frontend et push a GitHub version	Haute	5
	21	En tant que admin je veux faire dockerfile création image pour frontend	Moyenne	2
	22	En tant que admin je veux faire jenkinsfile pour frontend	Moyenne	3
	23	En tant que admin je veux faire test SELENUMM ET SONARQUEBE pour frontend	Moyenne	4
	24	Entant que admin je veux faire déploiement avec kubernet pour frontend et push autre version a GitHub	Faible	5
Sprint 4	25	En tant qu'Amin je veux faire déploiement pour chaque micro services	Haute	7
	26	Entant que admin je faire jenkinsfile pour chaque microservice	Haute	7
	27	En tant que admin je veux corriger bugs	Moyenne	7
	28	en tant que admin je veux faire monotoring	Moyenne	7

		grafana et promtheus pour chaque micro		
	29	En tant que admin je veux préparer rapport de pfe	Moyenne	
	30	En tant que admin je veux Préparer une présentation PowerPoint	Faible	

IX. PLANIFICATION DES SPRINTS :

La clé la plus importante pour la réussite d'un projet est une bonne planification. En effet, le planning aide à bien subdiviser le travail et sépare les tâches à effectuer, il offre une meilleure estimation et gestion de temps nécessaire pour accomplir ces tâches. Dans notre projet, j'estime réaliser notre application dans un laps de temps d'environ 4 mois. Le tableau ci-dessous montre le calendrier que nous avons ajusté pour réaliser les différentes parties du projet.

TABLEAU 4 : PLANNING DES SPRINTS

Sprint	User Story	Début	FIN
Sprint 0	1-5	22-01-2024	22-02-2024
Sprint 1	6-10	22-02-2024	22-03-2024
Sprint 2	11-17	22-03-2024	22-04-2024
Sprint 3	18- 24	22-02-2024	22-05-2024
Sprint 4	25-30	23-05-2024	20-06-2024

Cette planification des sprints permet de structurer efficacement le développement du projet, avec des objectifs clairs pour chaque sprint, facilitant ainsi le suivi et la gestion du projet.

X. L'ARCHITECTURE DE L'APPLICATION EN MICROSERVICES :

L'architecture en microservices pour le système de gestion des inscriptions aux formations et événements du centre 4C ISET Tozeur se compose de plusieurs services indépendants, chacun responsable d'une fonction spécifique. Voici une vue d'ensemble de l'architecture proposée :

▪ MICROSERVICES PRINCIPAUX :

- Service d'Authentification :
 - Rôle : Gérer l'authentification des utilisateurs (administrateurs et étudiants) via le CIN et un mot de passe.
 - Technologies : Node.js pour la gestion des sessions.
- Service de Gestion des étudiants
 - Rôle : Gérer les informations des utilisateurs (administrateurs, étudiants inscrits, étudiants passagers), y compris l'importation des étudiants depuis des fichiers CSV.
 - Technologies : Node.js, MySQL
- Service de Gestion des Formations
 - Rôle : Gérer le cycle de vie des formations (création, mise à jour, suppression), et l'inscription des étudiants aux formations.
 - Technologies : Node.js, Sequelize (ORM), MySQL.
- Service de Gestion des Événements :
 - Rôle : Gérer le cycle de vie des événements proposés par le centre 4C ISET Tozeur, y compris la création, la mise à jour et la suppression des événements.
 - Technologies : Node.js, Sequelize (ORM), MySQL.
- Service de Gestion des Contacts :
 - Rôle : Gérer les contacts et les informations de communication avec les étudiants et autres parties prenantes.

- Technologies : Node.js ,mysql pour le stockage des données de contact.
 - Service de Notification
 - Rôle : Envoyer des notifications par e-mail aux étudiants pour confirmer les inscriptions et fournir d'autres informations importantes.
 - Technologies : Node.js, Nodemailer.
 - Service de Statistiques et Reporting
 - Rôle : Collecter et analyser des données statistiques sur les formations et les inscriptions des étudiants, et fournir des rapports détaillés.
 - Technologies : Node.js, Sequelize (ORM), MySQL, Grafana.
 - MICROSERVICES AVANCES
 - SERVICE DE FEEDBACK
 - Rôle : Recueillir et analyser les feedbacks des étudiants sur les formations.
 - Technologies : Node.js, Sequelize (ORM), MySQL.
- Infrastructure et Déploiement
- API Gateway
 - Rôle : Agir comme point d'entrée unique pour toutes les demandes des utilisateurs et distribuer les demandes aux services appropriés.



Figure 23: ARCHITECTURE MICROSERVICES DE MON PROJET

Explications

- API Gateway : Centralise toutes les requêtes et les redirige vers les microservices appropriés.
- Microservices : Chaque microservice gère une fonction spécifique, permettant une modularité et une scalabilité optimales.
- Infrastructure : Des outils comme Docker et Kubernetes facilitent le déploiement, la gestion, et l'orchestration des conteneurs. Prometheus et Grafana surveillent les performances et la santé des services.

Cette architecture modulaire permet d'assurer une gestion efficace, une mise à l'échelle facile, et une maintenance simplifiée du système de gestion des inscriptions aux formations et événements du centre 4C ISET Tozeur

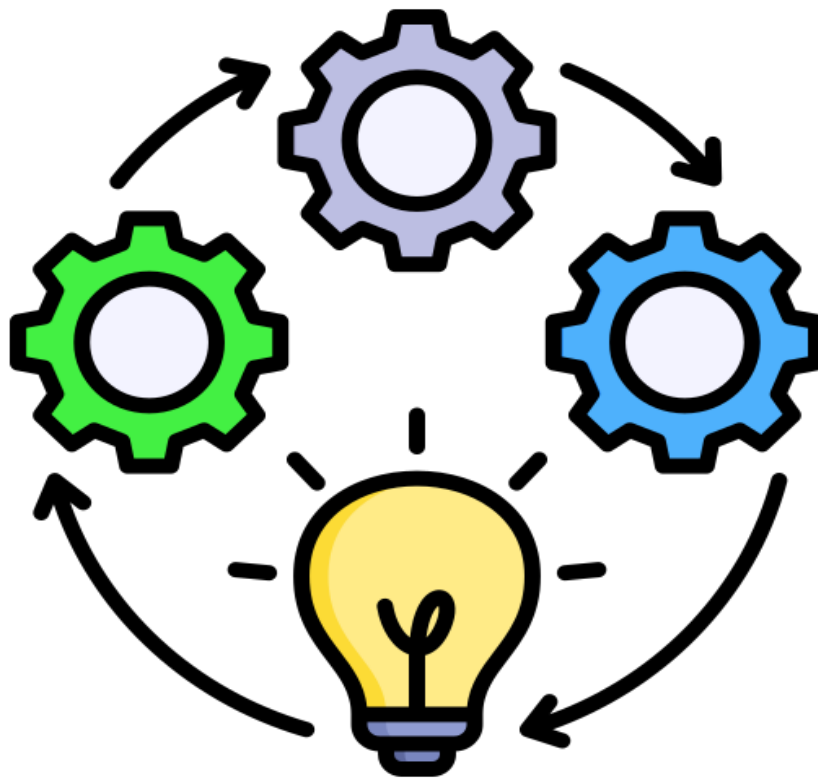
XI. CONCLUSION :

L'approche méthodique d'analyse et de spécification des besoins, ainsi que la planification rigoureuse du projet et le choix judicieux des technologies garantissent la

réussite du développement d'une plateforme de gestion des inscriptions performante, évolutive et répondant aux besoins du centre 4C ISET Tozeur.

CHAPITRE 5 :

Sprint 0 Mise en place du projet



I. INTRODUCTION :

Ce document détaille la mise en place et le développement du projet, en se concentrant sur les différents sprints et leurs objectifs. Le Sprint 0 est crucial pour préparer l'environnement de développement, les dépôts de code, les environnements de test et les outils de communication. L'objectif est d'assurer que tout est en place pour démarrer efficacement le développement au Sprint 1, y compris la préparation du backlog initial.

1. OBEJECTIF SPRINT 0

Le but du Sprint 0 est de préparer l'environnement de développement, les dépôts de code, les environnements de test, et les outils de communication, afin que l'équipe puisse démarrer efficacement le travail de développement dès le Sprint 1. Cela inclut également la préparation du backlog initial pour s'assurer que les user stories prioritaires sont prêtes pour le premier sprint de développement.

2. BACKLOG DU SPRINT 0 :

TABLEAU 5 : BACKLOG DE SPRINT 0

ID	CAS	USER STORY	PRIORITE	STORY POINT (J)
1	M'assurer Que L'environnement De Développement	En Tant Qu'administrateur Du Projet, Je Dois M'assurer Que L'environnement De Développement..	Haute	7
2	Dois Suivre Une Formation Sur Node.Js...	En Tant Qu'administrateur, Je Dois Suivre Une Formation Sur Node.Js...	Haute	30
3	Rechercher Et Développer Sur	En Tant Qu'administrateur Je Veux	Moyenne	7

	La Culture, L'histoire	Rechercher Et Développer Sur La Culture, L'histoire		
4	Préparer Cahier De Charge Et Fixer Architecture Microservices	En Tant Qu'administrateur, Je Veux Préparer Cahier De Charge Et Fixer Architecture Microservices	Haute	7
5	Créer Base Des Donnés Avec Mysql	En Tant Que Admin Je Veux Créer Base Des Donnés Avec Mysql	Moyenne	7

Ce tableau présente les user stories du Sprint 0 avec leur ID, leur cas, leur description, leur priorité et le nombre de story points estimé pour chacune. Cela fournit une vue synthétique du backlog, facilitant la gestion et la planification des tâches.

3. TACHES A ACCOMPLIR :

❖ M'assurer Que L'environnement De Développement Est Correctement Configuré

-Tâches détaillées :

- Installer et configurer Node.js
- Configurer un environnement de développement
- Installer et configurer MySQL
- Configurer Git et GitHub pour le projet
- Configurer Docker

❖ Suivre Une Formation Sur Node.js

-Tâches détaillées :

- Rechercher et sélectionner un cours de formation Node.js offert par une société de stage.

- Suivre les modules de base de la formation Node.js dispensée par la société de stage.
- Pratiquer des exercices et des projets Node.js recommandés dans le cadre de la formation.

❖ Rechercher Et Développer Sur La Culture, L'historique Et Les Outils DevOps

-Tâches détaillées :

- Rechercher des ressources sur la culture DevOps
- Étudier l'historique de DevOps
- Étudier les outils DevOps (Jenkins, Docker, Kubernetes, etc.)

❖ Préparer Cahier De Charge Et Fixer Architecture Microservices

-Tâches détaillées :

- Définir les besoins et les fonctionnalités du projet
- Écrire le cahier des charges
- Concevoir l'architecture microservices

❖ Créer Base Des Donnés Avec Mysql

-Tâches détaillées :

- Concevoir le schéma de la base de données
- Créer les tables et les relations dans MySQL
- Tester la base de données avec des exemples de données

II. Conclusion :

Le Sprint 0 est une phase de préparation essentielle qui garantit que tous les aspects techniques et organisationnels sont prêts avant le début du développement. Les tâches incluent la configuration de l'environnement de développement, la formation sur Node.js, la recherche sur les pratiques DevOps, la définition de l'architecture microservices et la création de la base de données. Cette préparation minutieuse est fondamentale pour le succès des phases de développement suivantes.

CHAPITRE 6

SPRINT 1

I. Introduction :

Ce chapitre décrit le Sprint 1, axé sur la mise en œuvre d'un microservice d'authentification sécurisé pour la plateforme d'inscription du 4C ISET Tozeur. L'objectif est de permettre aux étudiants et administrateurs de se connecter de manière sécurisée.

II. SPRINT GOAL DU SPRINT 1 :

Mettre en œuvre un microservice d'authentification sécurisé et efficace pour la plateforme d'inscription du 4C ISET Tozeur, permettant aux étudiants et aux administrateurs d'accéder de manière sécurisée au système. Cela inclut le développement des interfaces utilisateur, l'implémentation de l'authentification via le numéro d'identification national et le mot de passe, l'exécution de tests approfondis, ainsi que l'établissement d'un pipeline CI/CD robuste.

III. BACKLOG DU SPRINT 1 :

TABLEAU 6 : BACKLOG DE SPRINT 1

ID	Cas	USER STORY	Priorité	STORY POINT (j)
1	développer un microservice pour la gestion des étudiants	En tant qu'administrateur, je veux développer un microservice pour la gestion des étudiants	Haute	
2	développer un service d'authentification	En tant qu'administrateur, je veux développer un service d'authentification	Haute	
3		En tant qu'administrateur, je veux créer des		

		interfaces utilisateur avec Angular		
4	développer les interfaces de consultation des informations et les interfaces pour les administrateurs	En tant qu'administrateur, je veux développer les interfaces de consultation des informations et les interfaces pour les administrateurs		
5	effectuer des tests unitaires avec JUnit et SonarQube	En tant qu'administrateur, je veux effectuer des tests unitaires avec JUnit et SonarQube	Moyenne	

IV. ANALYSE FONCTIONNEL DE MICROSERVICES DE AUTHENTICATION :

L'analyse couvre les besoins en authentification des utilisateurs, les fonctionnalités de vérification des identifiants, la gestion des sessions, et l'authentification multi-facteurs pour les étudiants et administrateurs.

L'analyse fonctionnelle des microservices d'authentification dans le contexte de la plateforme de gestion des inscriptions du centre 4C ISET Tozeur implique de comprendre les besoins spécifiques liés à l'authentification des utilisateurs, tant du point de vue des étudiants que des administrateurs.

Voici une analyse fonctionnelle détaillée :

1. MICROSERVICE D'AUTHENTIFICATION POUR LES ETUDIANTS :

- Objectif : Permettre aux utilisateurs (étudiants et administrateurs) de s'identifier de manière sécurisée sur la plateforme.
- Fonctionnalités :
 - Vérification des identifiants : Le microservice doit valider l'identité des utilisateurs en vérifiant leurs numéros de carte d'identité nationale (CIN) et mots de passe.
 - Gestion des sessions : Il doit gérer les sessions d'authentification pour maintenir la connexion des utilisateurs pendant leur interaction avec la plateforme.
 - Authentification multi-facteurs (optionnel) : Soutenir l'authentification multi-facteurs pour renforcer la sécurité de l'accès.

2. MICROSERVICE D'AUTHENTIFICATION POUR LES ADMINISTRATEURS :

- Objectif : Assurer un accès sécurisé aux fonctionnalités administratives de la plateforme.
- Fonctionnalités :
 - Authentification sécurisée : Même principe que pour les utilisateurs, avec des autorisations supplémentaires pour les administrateurs.
 - Contrôle d'accès : Définir et gérer les niveaux d'accès des administrateurs aux différentes fonctionnalités de gestion.

V. CONCEPTION DE MICROSERVICES D'AUTHENTIFICATION :

Les diagrammes d'utilisation, de séquence, et de classe illustrent la mise en œuvre des fonctionnalités d'authentification.

1. Diagrammes de cas d'utilisations de sprint 1 :

Le Sprint 1 se concentre sur la mise en place de l'infrastructure de base et les fonctionnalités d'authentification essentielles pour permettre aux utilisateurs (étudiants et administrateurs) de se connecter à la plateforme. Cela comprend la création de l'interface de connexion, la mise en œuvre de l'authentification via CIN et mot de passe, ainsi que la configuration initiale des outils nécessaires pour le développement continu.

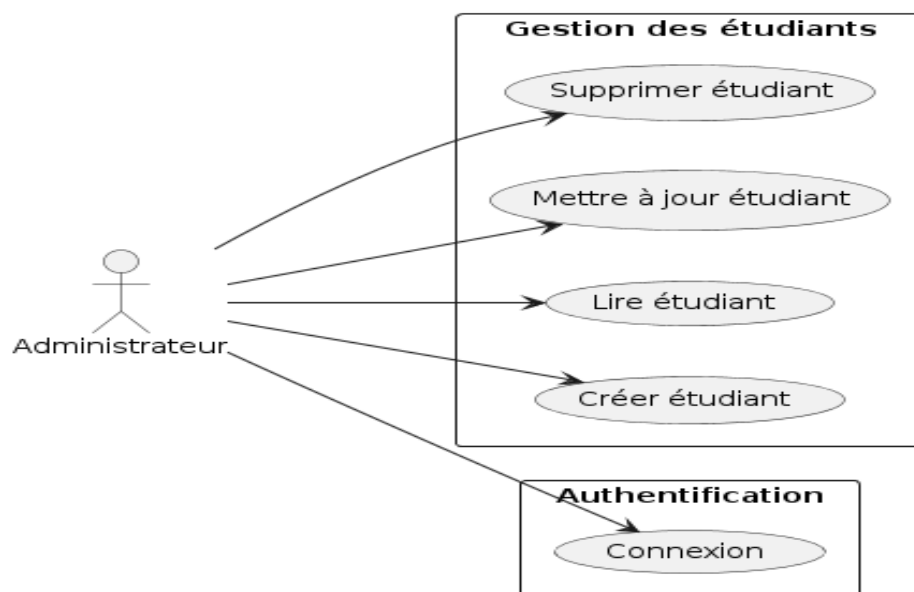


Figure 24:DIAGRAMME DE CAS D'UTILISATIONS POUR SPRINT 1

2. Diagramme de séquence de cas « S'authentifier »

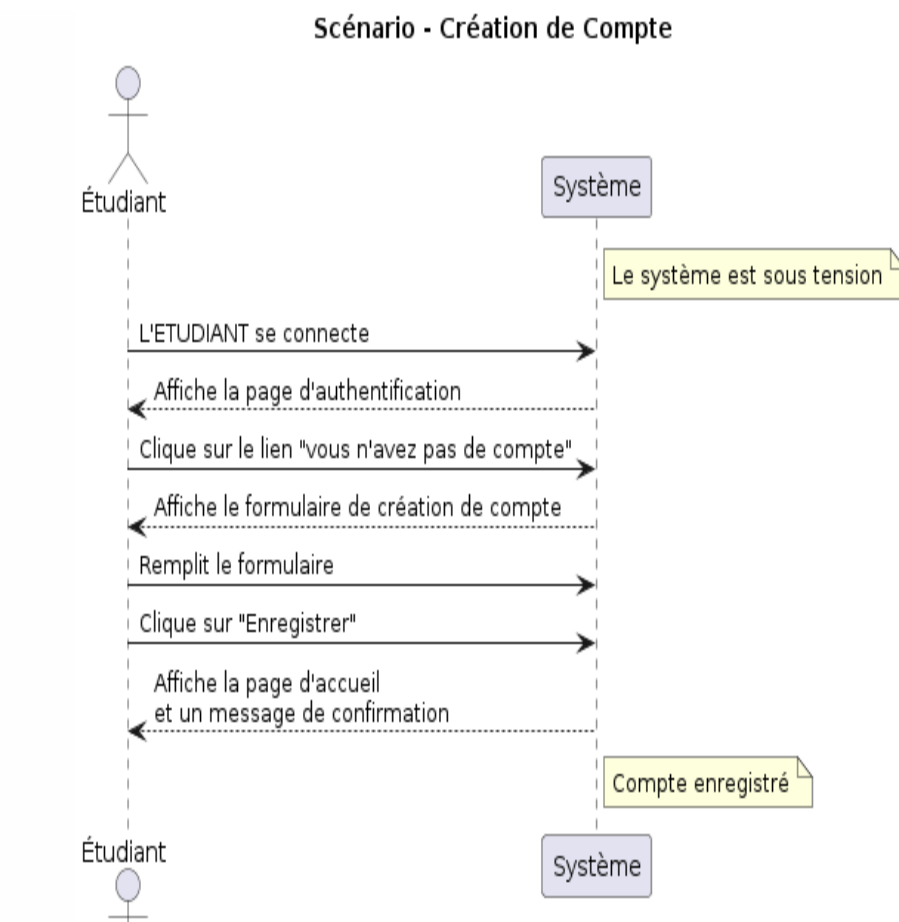


Figure 25:DIAGRAMME DE SEQUENCE DE CAS « S'AUTHTIFIER »

3. Diagramme de classe pour sprint 1 :

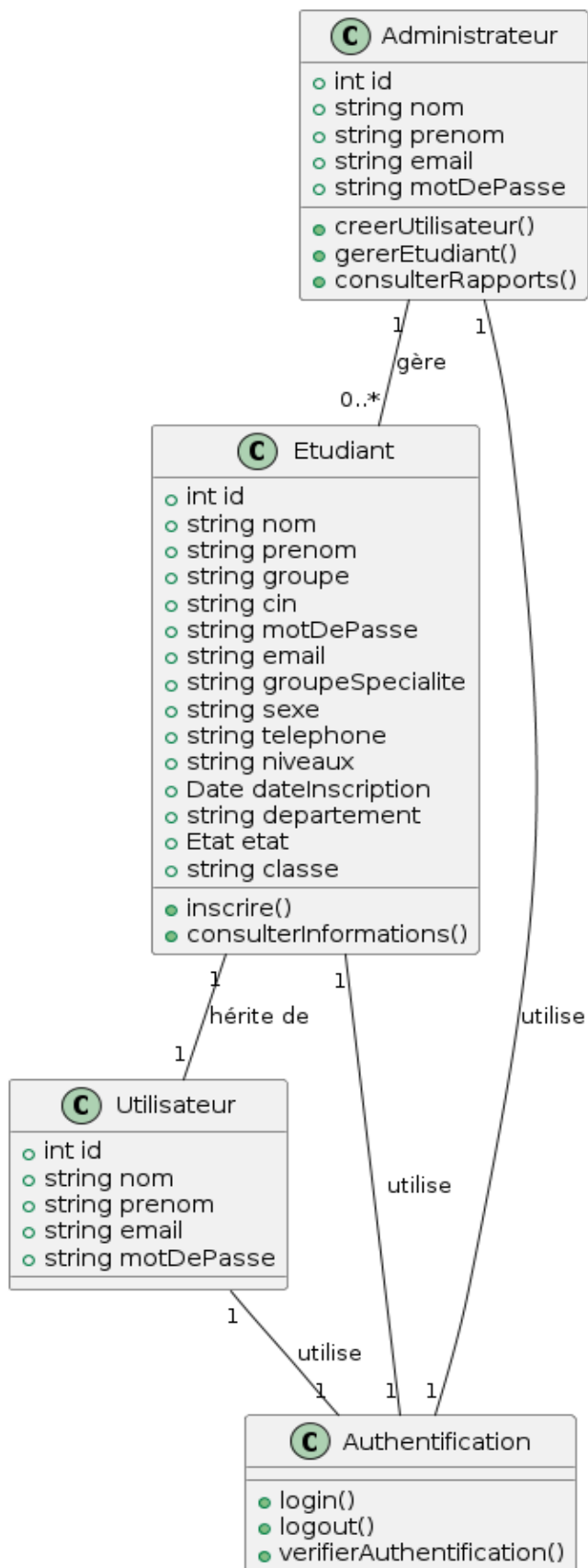


Figure 26: DIAGRAMME DE CLASSE

VI. LES ETAPES DE CREATION DU PIPELINE POUR MICROSERVICES AUTHENTIFICATION :

Les étapes incluent la planification dans Jira, l'accès au code source sur GitHub, l'exécution des tests avec JUnit et SonarQube, la création des builds avec Docker, le déploiement sur Kubernetes, et le monitoring avec Prometheus et Grafana.

1. PLAN :

Avant même de commencer la configuration du pipeline, la phase initiale consiste à planifier le projet dans Jira, un outil de gestion de projet populaire

Cette capture représente le plan du Sprint 1 dans Jira

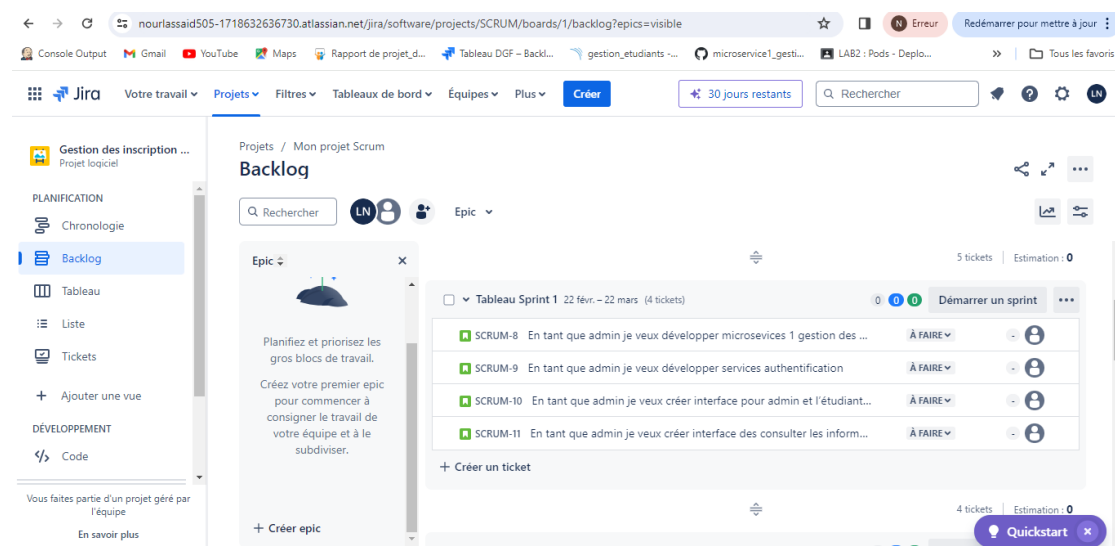


Figure 27:INTERFACE JIRA SPRINT 1

2. ACCES AU CODE SOURCE SUR GITHUB :

Cette capture représente l'intégration de GitHub comme outil de gestion de code source dans le pipeline de développement pour les microservices d'authentification. GitHub est choisi pour ses fonctionnalités de collaboration et d'intégration avec les pipelines CI/CD.

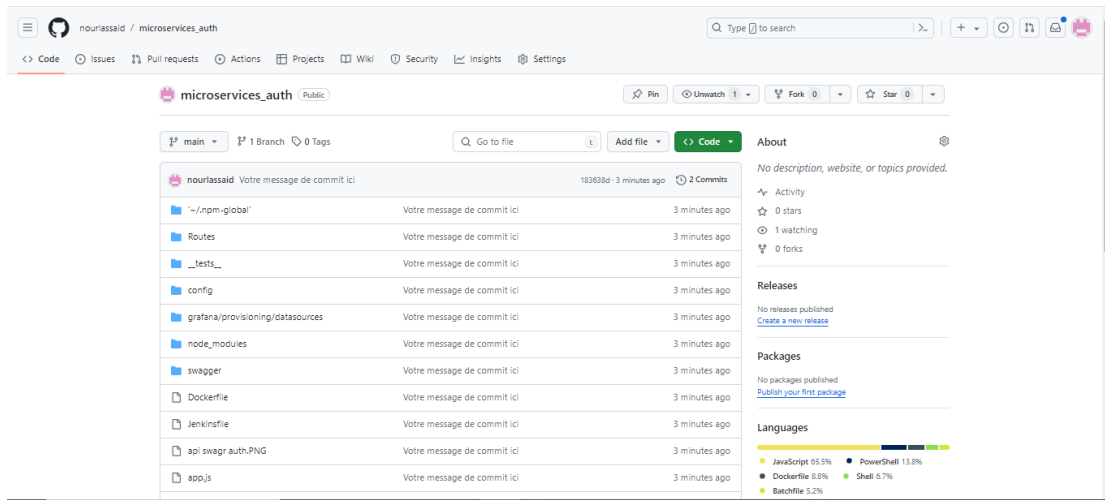


Figure 28: INTERFACE GITHUB POUR MICROSERVICE AUTHENTIFICATION

3. EXECUTER LES TESTS :

Cette capture montre l'utilisation de JTEST dans le pipeline pour l'exécution des tests. JTEST est un outil d'automatisation des tests adapté aux environnements Java, utilisé ici pour exécuter des tests unitaires, d'intégration ou de système afin de garantir la qualité du code.

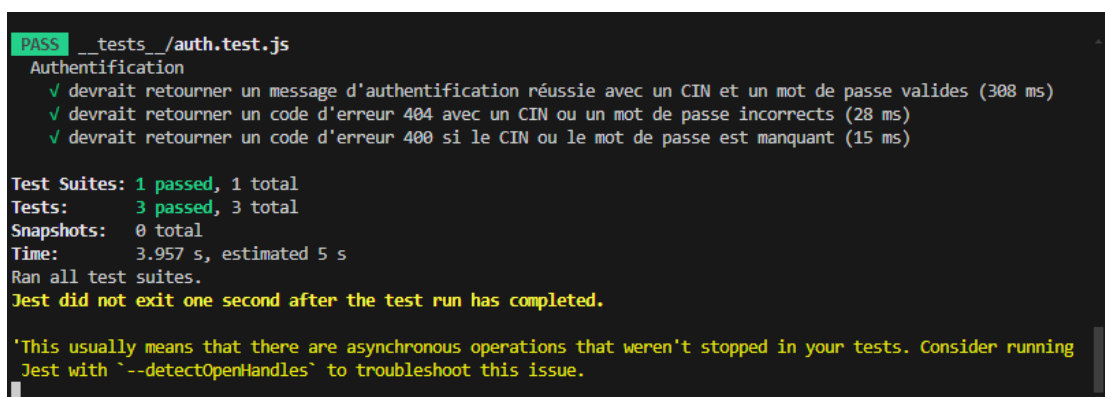


FIGURE 31 : CAPTURE DE JTEST POUR MICROSERVICE AUTHENTIFICATION

4. VERIFIER LA QUALITE DU CODE :

Cette capture illustre l'intégration de SonarQube dans le pipeline pour vérifier la qualité du code. SonarQube est utilisé pour l'analyse statique du code afin de détecter les bugs, les vulnérabilités de sécurité et les mauvaises pratiques de codage, assurant ainsi un code maintenable et performant.

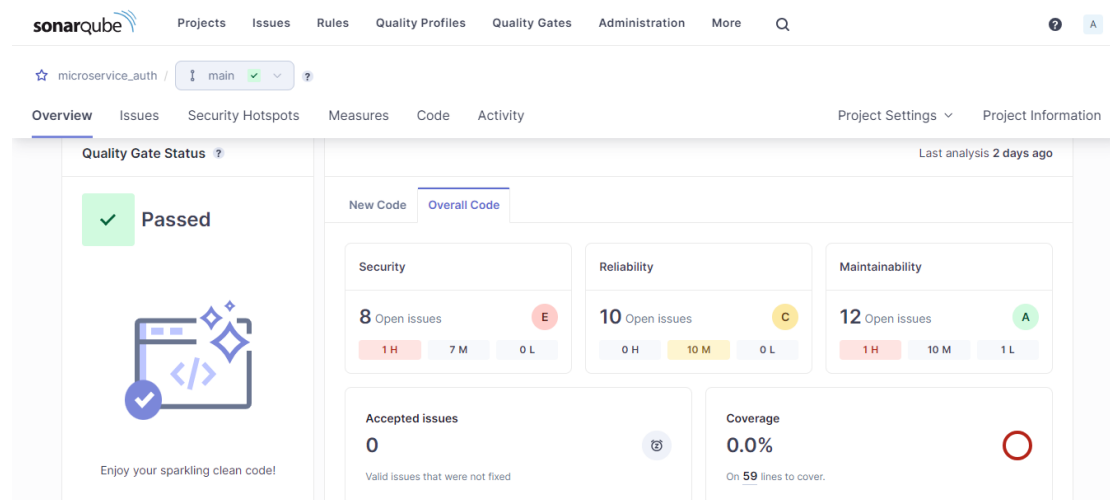


Figure 29: CAPTURE DE TEST SONARQUBE POUR MICROSERVICE AUTHENTICIDICATION

5. CREATION DES BUILD :

Un fichier Dockerfile est un script utilisé par Docker pour créer une image Docker. Cette image contient tout ce qui est nécessaire pour exécuter une application, y compris le code, les dépendances et les configurations. Le fichier que vous avez fourni est un Dockerfile multi-stage pour une application Node.js. Voici une explication détaillée de chaque section de votre Dockerfile

L'utilisation d'un Dockerfile multi-stage permet de réduire la taille de l'image finale. La première étape (build) inclut tout ce qui est nécessaire pour construire l'application (y compris les outils de développement et les dépendances). La deuxième étape (final) ne contient que ce qui est nécessaire pour exécuter l'application, ce qui entraîne une image plus petite et plus efficace.

```
C:\Users\MSAR\Backend_nour_Formations\micro1_authentication>docker build -t auth_service1 .
[+] Building 0.0s (0/0) docker:default
2024/06/15 23:05:20 http2: server: error reading preface from client //./pipe/docker_engine: file has already b
[+] Building 788.6s (13/13) FINISHED docker:default
=> [internal] load build definition from Dockerfile 0.3s
=> => transferring dockerfile: 605B 0.1s
=> [internal] load metadata for docker.io/library/node:20 9.5s
=> [auth] library/node:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore 0.4s
=> => transferring context: 2B 0.0s
=> [build 1/6] FROM docker.io/library/node:20@sha256:d0a9a2399581a9de1ff962a48a28b5cfe700678a6a5df8e3 537.0s
=> => resolve docker.io/library/node:20@sha256:d0a9a2399581a9de1ff962a48a28b5cfe700678a6a5df8e31a63aaa4 0.5s
=> => sha256:d0a9a2399581a9de1ff962a48a28b5cfe700678a6a5df8e31a63aaa47bebb923 1.21kB / 1.21kB 0.0s
=> => sha256:dc290380e80a773f08c0a382d736ef42069544c545f745bc03e56092e5880679 2.00kB / 2.00kB 0.0s
=> => sha256:d921256521a75ed3740a1854d592af87f1002d15ebb7e933b163681157adafec 7.41kB / 7.41kB 0.0s
=> => sha256:fea1432adf0957427b45b0ef8e37cbfe045b7cf8c15e3f43e48f2f613e214d16 49.58MB / 49.58MB 155.4s
=> => sha256:3873416e6a335157d669c6193a256dfb289331d669d87f200e4eed1f19f9ebb9 64.14MB / 64.14MB 247.3s
=> => sha256:5651b5803b186603909f6c77cdf7bddd4ba7ab8ca4ebccb5a6b0be9037b4e5b6 24.05MB / 24.05MB 135.0s
=> => sha256:8a142b8b0e695954ed154c13c098ede4e217dcb99aa7158e34361604191822bd 211.21MB / 211.21MB 420.9s
=> => sha256:e6762bf379915ae5a1fadf0446b0e676e49dcf5e2cf38b7f11f290fd335f6c14 3.34kB / 3.34kB 157.2s
=> => sha256:fea1432adf0957427b45b0ef8e37cbfe045b7cf8c15e3f43e48f2f613e214d16 38.1s
=> => sha256:dd3765d7cb0819a434a0184674e662399b15b6ce1d765c5f12b3efa1b083fe59 48.01MB / 48.01MB 271.4s
=> => extracting sha256:5651b5803b186603909f6c77cdf7bddd4ba7ab8ca4ebccb5a6b0be9037b4e5b6 9.3s
=> => sha256:2a8ec3295c5a9c1c38fb2b5eab0791f9fa9f86fd0b2ad05187115e6eb7fe0b12 1.25MB / 1.25MB 252.9s
=> => sha256:f89cb71e96e3a2cef8f1753eb631879fca12ca04ead932295ac2af07b1142f7b 454B / 454B 254.7s
=> => extracting sha256:3873416e6a335157d669c6193a256dfb289331d669d87f200e4eed1f19f9ebb9 40.6s
=> => sha256:8a142b8b0e695954ed154c13c098ede4e217dcb99aa7158e34361604191822bd 82.1s
=> => extracting sha256:e6762bf379915ae5a1fadf0446b0e676e49dcf5e2cf38b7f11f290fd335f6c14 0.0s
=> => sha256:dd3765d7cb0819a434a0184674e662399b15b6ce1d765c5f12b3efa1b083fe59 24.4s
=> => extracting sha256:2a8ec3295c5a9c1c38fb2b5eab0791f9fa9f86fd0b2ad05187115e6eb7fe0b12 0.5s
=> => extracting sha256:f89cb71e96e3a2cef8f1753eb631879fca12ca04ead932295ac2af07b1142f7b 0.0s
=> [internal] load build context 281.6s
=> => transferring context: 63.68MB 277.1s
```

Figure 30: CAPTURE DE CREATION DES BUILD

6. CREATION DES IMAGES :

- Description des Étapes pour Créer et Pousser une Image Docker

a. Connexion à Docker Hub

```
C:\Users\MSAR\Backend_nour_Formations\micro1_authentication>docker login
Authenticating with existing credentials...
Login Succeeded

C:\Users\MSAR\Backend_nour_Formations\micro1_authentication>
```

Figure 31: CAPTURE DE CONNEXION DOCKER HUB

Description : Cette commande vous permet de vous authentifier auprès de Docker Hub. Vous serez invité à entrer votre nom d'utilisateur et votre mot de passe Docker Hub.

b. Taguer l'Image Docker

```
C:\Users\MSAR\Backend_nour_Formations\micro1_authentification>docker tag auth_service1 nour0/auth_service1:latest  
C:\Users\MSAR\Backend_nour_Formations\micro1_authentification>
```

Figure 32: CAPTURE DE TAG IMAGE DOCKER

Cette commande attribue un tag à votre image Docker locale `auth_service1`. Le tag suit le format `<dockerhub_username>/<repository_name>:<tag>`, où `nour0` est votre nom d'utilisateur Docker Hub, `auth_service1` est le nom du dépôt, et `latest` est le tag.

c. Pousser l'Image vers Docker Hub

Cette commande pousse l'image Docker taguée vers votre dépôt Docker Hub. Assurez-vous que vous avez bien tagué votre image avant d'exécuter cette commande.

```
C:\Users\MSAR\Backend_nour_Formations\micro1_authentification>docker push nour0/auth_service1:latest  
The push refers to repository [docker.io/nour0/auth_service1]  
3ed4531dd397: Pushed  
ddc87a3452c4: Pushed  
5c6aa995425a: Mounted from library/node  
68c94c11bf87: Mounted from library/node  
a7f6e12671fb: Mounted from library/node  
e8c94ab423c9: Mounted from library/node  
3999ea91fb6e: Mounted from library/node  
43df359389fd: Mounted from library/node  
d3e8d42f967c: Mounted from library/node  
5d64de483bf5: Mounted from library/node  
latest: digest: sha256:1b482ff04dc6a5c1925a1e151853ecaa8034f0ee7b779276c552dce1182f47b4 size: 2423  
C:\Users\MSAR\Backend_nour_Formations\micro1_authentification>
```

Figure 33: CAPTURE DE POUSSER L'IMAGE VERS DOCKER HUB

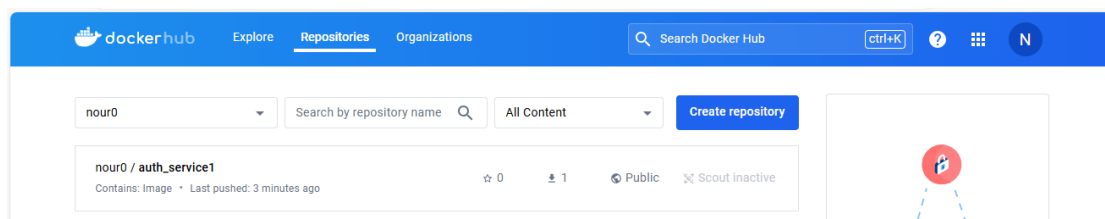


Figure 34: CAPTURE DE DOCKER HUB

7. Déploiement sur kubernetes :

Pour déployer une application sur Kubernetes, vous avez suivi plusieurs étapes :

1. Création des Images Docker : JAI créé des images Docker pour Nos services, comme illustré précédemment avec `auth_service1` et `mysql_auth`.
2. Définition des Déploiements Kubernetes :

Vous avez défini des déploiements Kubernetes pour chaque service. Cela inclut les spécifications telles que le nombre de réplicas, les images Docker utilisées, les variables d'environnement nécessaires, les montages de volume, etc.

Par exemple, pour MySQL, vous avez créé un déploiement avec un volume persistant et un ConfigMap pour l'initialisation des données.

2. Exposition des Services : Vous avez configuré des Services Kubernetes pour exposer vos déploiements. Cela inclut la définition des ports et, éventuellement, le type de service (NodePort, LoadBalancer, ClusterIP) en fonction de vos besoins
3. Vérification et Diagnostics : Vous avez utilisé des commandes comme `kubectl get pods` pour vérifier l'état de vos pods et `kubectl logs` pour inspecter les logs en cas d'erreurs ou de problèmes de démarrage.
4. Résolution des Problèmes : En cas d'erreurs comme des erreurs de connexion à la base de données, vous avez pris des mesures pour résoudre ces problèmes. Cela peut inclure la vérification des règles de pare-feu, la redémarrage des pods ou la vérification des adresses IP et des configurations de service.

```
C:\Users\MSAR\Backend_nour_Formations\micro1_authentification>kubectl get pods -n auth
NAME                                READY  STATUS   RESTARTS   AGE
backend-deployment-75b6b467f9-2km5j 1/1    Running  2 (26m ago) 36h
mysql-deployment-695d694cbf-k5xhj    1/1    Running  3 (26m ago) 2d4h

C:\Users\MSAR\Backend_nour_Formations\micro1_authentification>
```

Figure 35: CAPTURE DE DEPLOYMENTS

8. Monitoring des l'application :

Objectifs :

- Collecter des métriques de vos applications et services.

- Visualiser ces métriques à l'aide de Grafana.
- Configurer Prometheus pour collecter et stocker les métriques.

Cette capture d'écran montre l'interface principale de Prometheus avec les métriques collectées et quelques exemples de requêtes de métriques

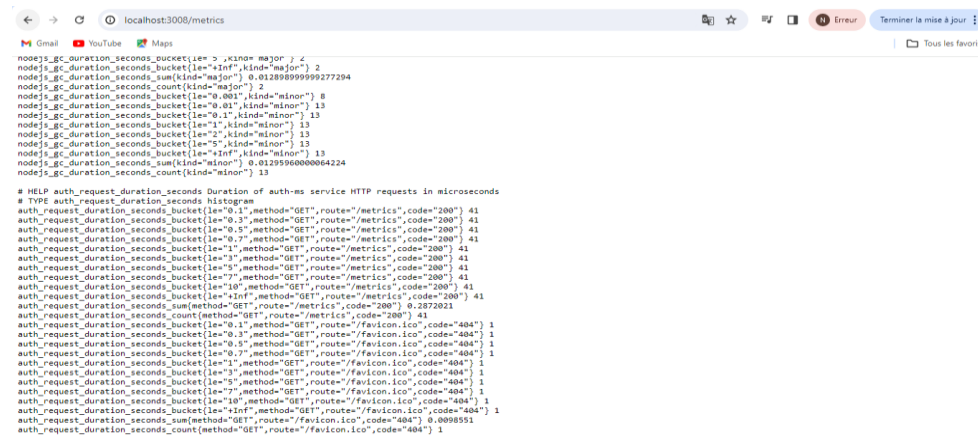


Figure 36: CAPTURE DE COLLECTER METRICS

❖ Étapes pour le Monitoring :

1. Installation de Prometheus et Grafana

Prometheus : Un système de monitoring et d'alerte open-source.

Grafana : Un outil de visualisation de données et de métriques.

Fichiers de Configuration :

prometheus.yml : Configuration de Prometheus pour scruter les endpoints exposés par les services.

datasource.yaml : Configuration des sources de données pour Grafana.

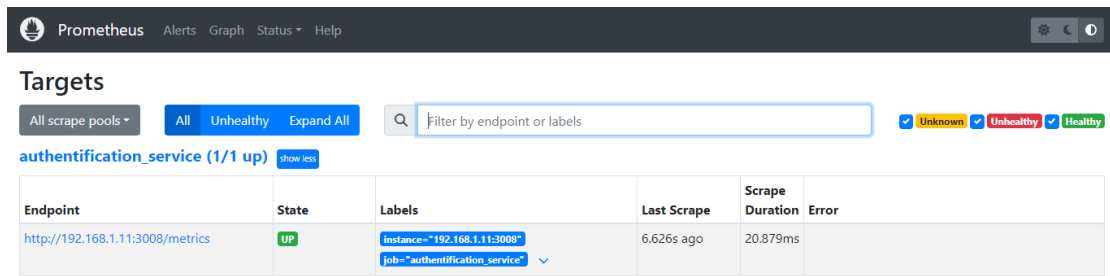


Figure 37: CAPTURE DE PROMETHEUS

Cette interface affiche Dashboard de garfana

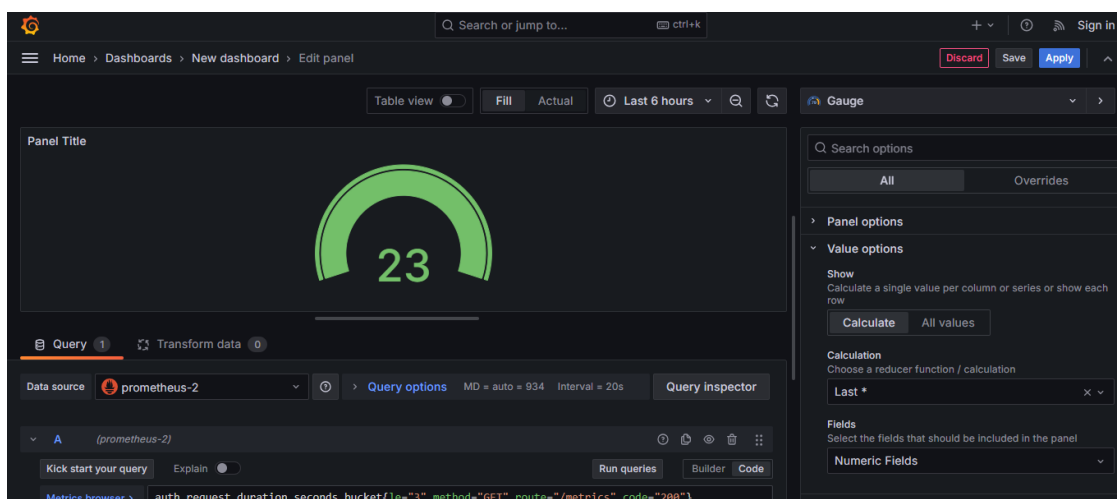


Figure 38: CAPTURE DE GRAFANA

9. Technologies utilisées :

Pour ce projet, plusieurs technologies ont été sélectionnées pour leur robustesse et leur capacité à répondre aux exigences du développement moderne. Voici un aperçu des technologies utilisées :

Angular : Utilisé pour le frontend, ce framework JavaScript de Google permet de développer une interface utilisateur dynamique avec une architecture modulaire et réactive.

Node.js : Employé pour le backend, Node.js offre une haute performance avec son modèle d'E/S non bloquant, idéal pour gérer des connexions simultanées et des opérations intensives.

MySQL : Choisi comme système de gestion de base de données, MySQL est reconnu pour sa fiabilité, ses performances et ses fonctionnalités avancées comme le support des transactions.

10.CI/CD

- Jenkins

VII. REALISATION DU SPRINT 1:

Cette section comporte des imprimes d'écran de quelques interfaces de l'application réalisée.

1. INTERFACE D'AUTHENTIFICATION :

Un étudiant doit saisir son CIN et son PASSWORD pour pouvoir accéder à sa propre session.

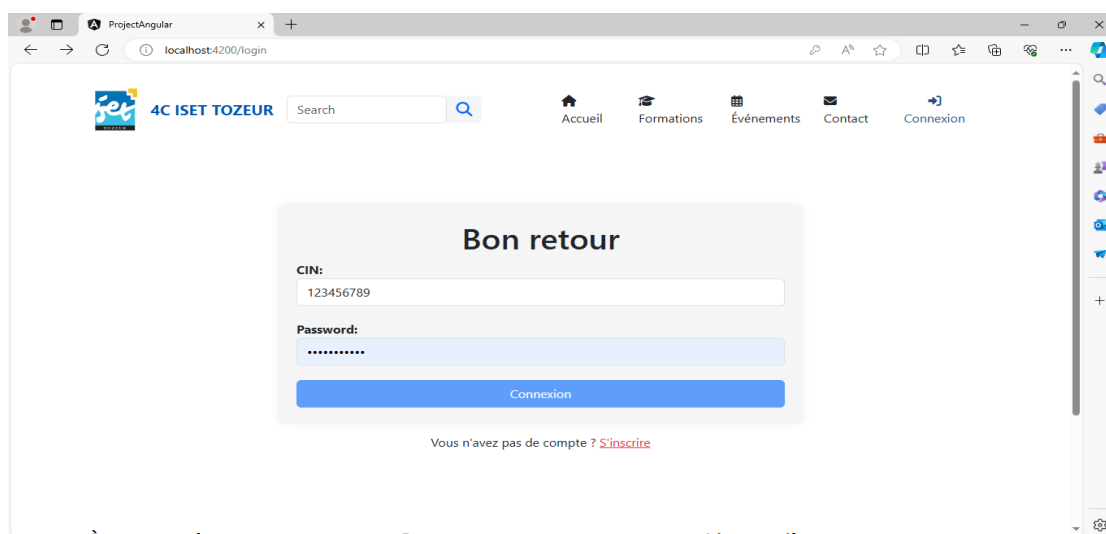


Figure 39:INTERFACE DAUTHENTIFIER

2. INTERFACE D'INSCRIPTION DES ETUDIANTS :

Cette interface permet à nos étudiants de s'inscrire en ligne.

ProjectAngular

localhost:4200/inscription-etudiant

4C ISET TOZEUR

Accueil Formations Événements Contact Connexion

Inscription étudiants

Nom :
djo

Prénom :
mohamed

Email :
mariam@gmail.com

CIN :
09123456

Mot de passe :

Figure 40:INTERFACE D'INSCRIPTION DES ETUDIANTS

3. INTERFACE DE MISE A JOUR PROFIL

Cette interface permet à nos étudiants de modifier coordonner.

ProjectAngular

localhost:4200/update-profile

Mise à jour du profil

Nom :
djo

Prénom :
djo

Email :
djo@gmail.com

CIN :

Mot de passe :

Téléphone :
78945620

Département :

Figure 41:INTERFACE UPDATE PROFILE ETUDIANTS

VIII. MICROSERVICES 2 GESTION DES ETUDIANTS

1. ANALYSE FONCTIONNEL DE MICROSERVICES DE GESTION DES ETUDIANTS

Le microservice d'authentification doit permettre aux utilisateurs de s'authentifier de manière sécurisée en utilisant leur numéro de carte d'identité nationale (CIN) et un mot de passe. Il doit également gérer les sessions et fournir des fonctionnalités de réinitialisation de mot de passe.

- Contexte et Objectif

Le service de gestion des étudiants est destiné à gérer les informations des utilisateurs, incluant les administrateurs, les étudiants inscrits et les étudiants passagers. Il doit également permettre l'importation des étudiants à partir de fichiers CSV. Ce service est essentiel pour maintenir une base de données à jour et pour faciliter l'administration des utilisateurs et des formations.

- Fonctionnalités du Service de Gestion des Étudiants

- Gestion des étudiants
- Importation des Étudiants
- Gestion des Étudiants Inscrits et Passagers

2. LES ETAPES DE CREATION DU PIPELINE POUR MICROSERVICES GESTION DES ETUDIANTS :

a. accès au code source sur GitHub :

Cette capture représente l'intégration de GitHub comme outil de gestion de code source dans le pipeline de développement pour les microservices de gestion des étudiants. GitHub a été choisi pour ses fonctionnalités avancées de collaboration et

d'intégration avec les pipelines CI/CD.

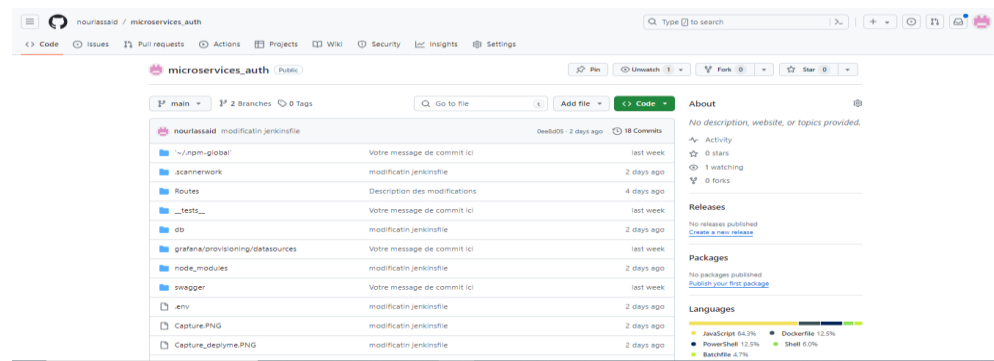


Figure 42: INTERFACE GITHUB POUR MICROSERVICE GESTION ETUDIANTS

b. exécuter les tests

J'utilise JUnit pour exécuter des tests unitaires afin de vérifier que le code fonctionne correctement et conforme aux spécifications.

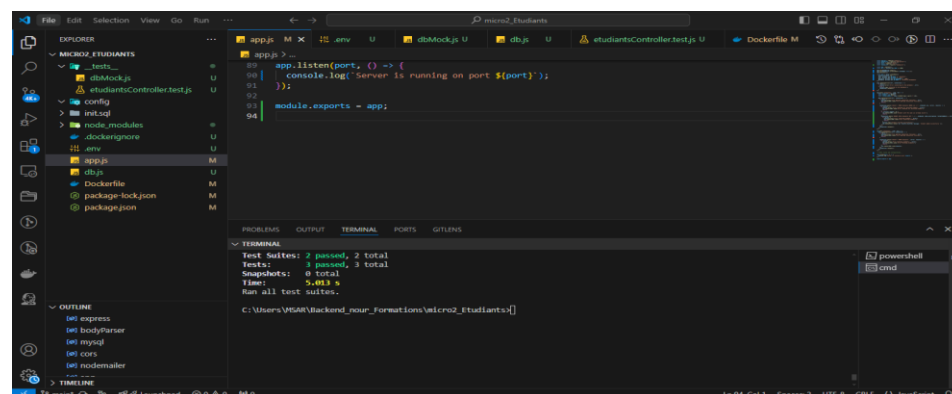


Figure 43: CAPTURE DE JTEST POUR MICROSERVICE GESTIONN ETUDIANTS

c. vérifier la qualité du code :

J'utilise SonarQube pour évaluer et améliorer la qualité du code, ainsi que pour détecter et corriger les bugs et les vulnérabilités.

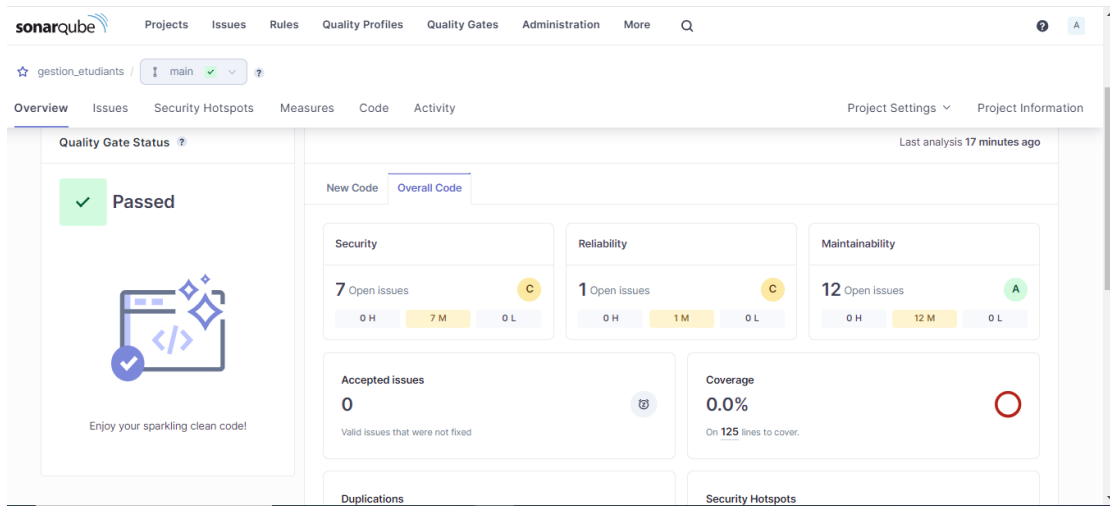


Figure 44: CAPTURE DE TEST SONARQUBE

d. création des build :

cette capture represente des build est essentielle pour enregistrer et documenter le processus de création des artefacts logiciels à chaque étape du développement.

```
C:\Users\MSAR\Backend_nour_Formations\micro2_Etudiants\db>docker build -t mysql_etudiant:latest .
[+] Building 2.5s (7/7) FINISHED
=> [internal] load build definition from Dockerfile                                docker:default
=> => transferring dockerfile: 369B                                              0.7s
=> [internal] load metadata for docker.io/library/mysql:latest                  0.4s
=> [internal] load .dockerignore                                                 0.0s
=> => transferring context: 2B                                                  0.6s
=> [internal] load build context                                                0.3s
=> => transferring context: 19.18kB                                             0.2s
=> [1/2] FROM docker.io/library/mysql:latest                                  0.1s
=> CACHED [2/2] COPY init.sql /docker-entrypoint-initdb.d/                    0.0s
=> exporting to image                                                            0.1s
=> => exporting layers                                                            0.0s
=> => writing image sha256:5472b439d4519e70d156fea5510f2e22da76ef3098d2bc62097286a43934348a 0.0s
```

Figure 45: CAPTURE DE CREATION DES BUILD

e. création des images :

Cette Capture d'écran illustrant la création d'une image Docker à partir d'un fichier Dockerfile et son étiquetage pour le téléversement ultérieur vers Docker Hub.

```
C:\Users\MSAR\Backend_nour_Formations\micro2_Etudiants\db>docker login
Authenticating with existing credentials...
Login Succeeded

C:\Users\MSAR\Backend_nour_Formations\micro2_Etudiants\db>docker tag mysql_etudiant nour0/ysql_etudiant:latest

C:\Users\MSAR\Backend_nour_Formations\micro2_Etudiants\db>docker push nour0/ysql_etudiant:latest
The push refers to repository [docker.io/nour0/ysql_etudiant]
a8239eec2213: Mounted from nour0/mysql1
5e4336bf306a: Mounted from nour0/mysql_auth
9a7bf98ce4cb: Mounted from nour0/mysql_auth
7db15ae0c4bc: Mounted from nour0/mysql_auth
193c79081b3f: Mounted from nour0/mysql_auth
04da47d8a5bf: Mounted from nour0/mysql_auth
b456f025514c: Mounted from nour0/mysql_auth
f64b5f1dbac0: Mounted from nour0/mysql_auth
ea99f328bc72: Mounted from nour0/mysql_auth
662cdc2f8e31: Mounted from nour0/mysql_auth
0e2e8dd6d8eb: Mounted from nour0/mysql_auth
latest: digest: sha256:8c997eb0ca370cf3cd21162cc41fa1e26eb85afacbb5deb823dfc58b7751f08d size: 2619

C:\Users\MSAR\Backend_nour_Formations\micro2_Etudiants\db>
```

Figure 46: CAPTURE DE CREATION IMAGE

f. DEPLOIEMENT SUR KUBERNETES :

Cette capture represente L'exécution de get pods ne devrait désormais afficher que les nouveaux pods :

NAME	READY	STATUS	RESTARTS	AGE
backend-deployment-576bf699bd-ffsz2	1/1	Running	0	14m
mysql-deployment-76864c54d5-xkqp6	1/1	Running	1 (5m3s ago)	8m31s

Figure 47: CAPTURE EXECUTION PODS

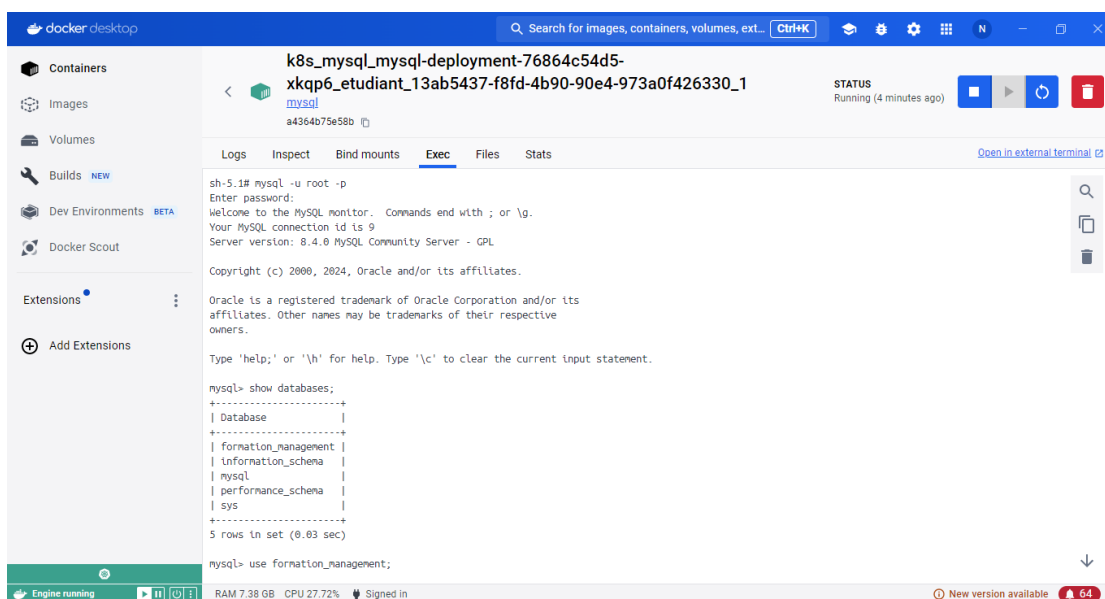


Figure 48: INTERFACE DE CONTENEUR MYSQL

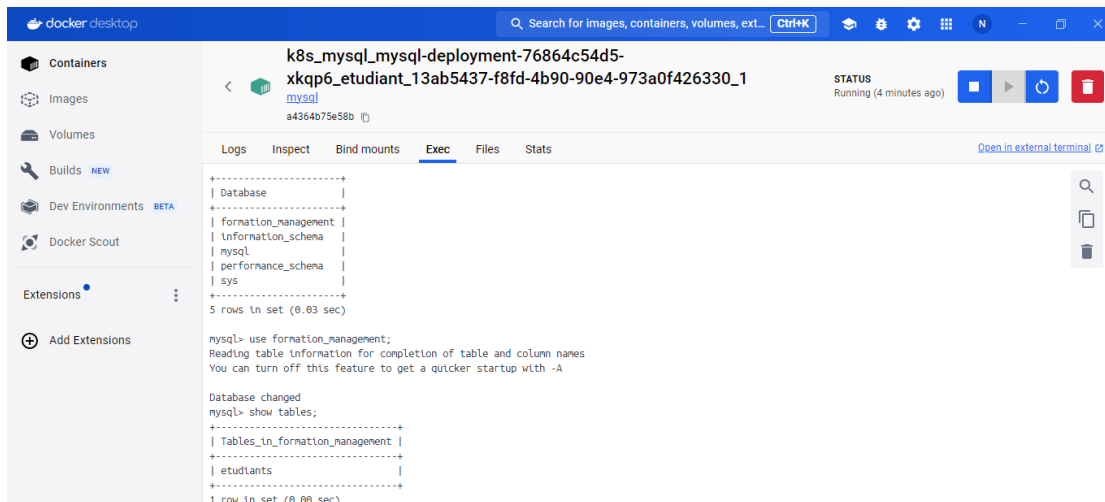


Figure 49:INTERFACE DE CONTENEUR MYSQL

g. MONITORING DES L'APPLICATION :

Cette capture illustre le monitoring des microservices de gestion des étudiants à l'aide de Grafana et Prometheus, mettant en avant la collecte et la visualisation des métriques essentielles pour assurer la performance et la fiabilité du système

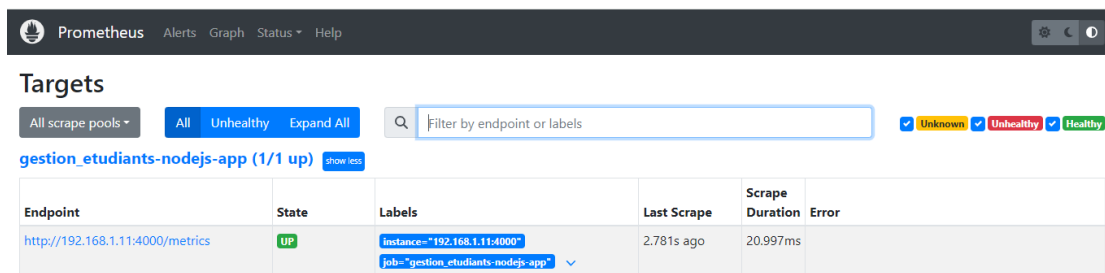


Figure 50:INTERFACE PROMTHEUS

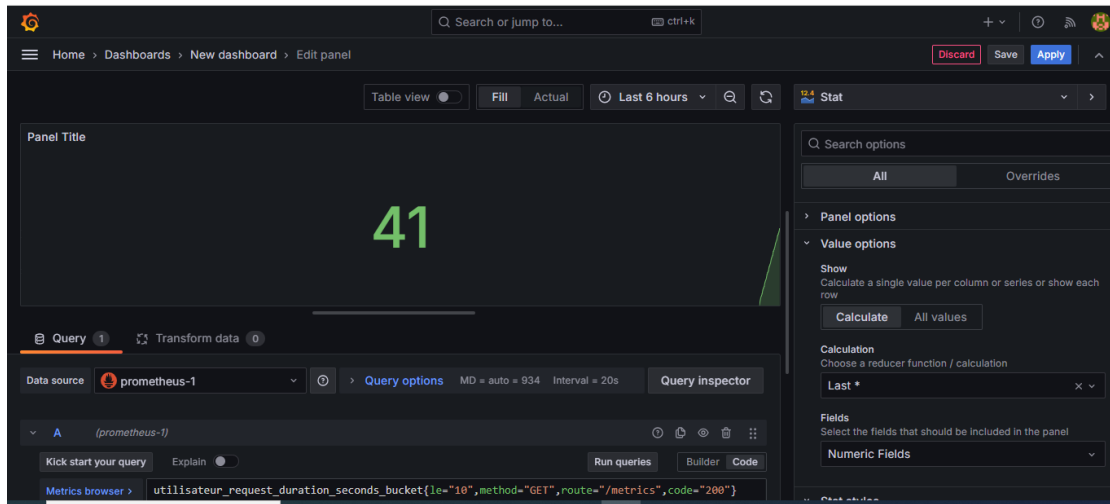


Figure 51: CAPTURE DASHBOAORD GRAFANA

IX. REALISATION DE GESTION DES ETUDIANTS :

Des interfaces permettent la gestion des étudiants, y compris l'ajout de nouveaux étudiants et la consultation de la liste des étudiants.

1. INTERFACE LISTER LES ETUDIANTS :

Cette interface permet au admin de consulter la liste des étudiants. En effet, il a la possibilité accepter ou refuser un étudiant.

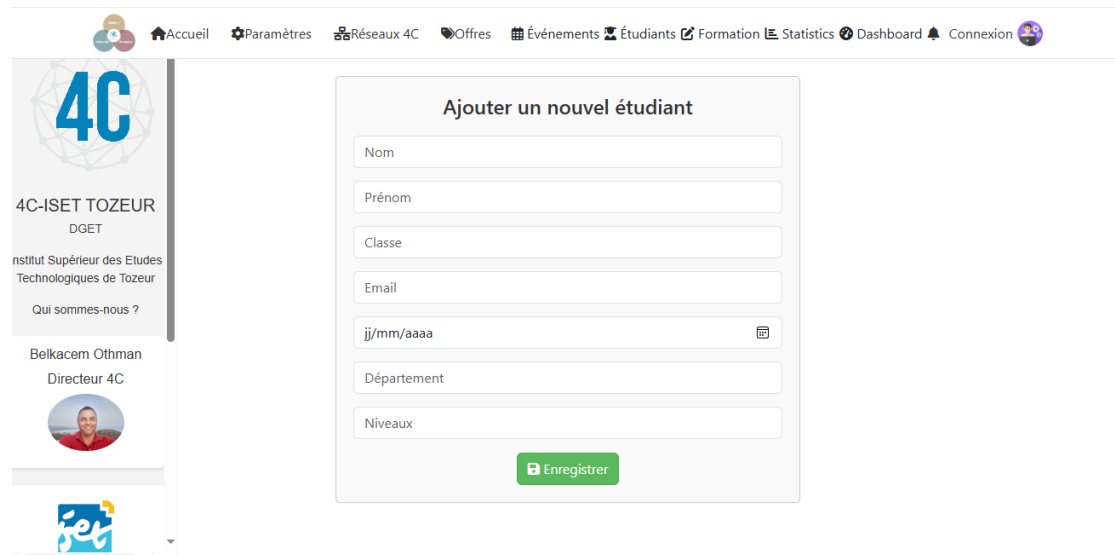
The image shows a web application interface for managing students. The header includes navigation links like 'Accueil', 'Paramètres', 'Réseaux 4C', 'Offres', 'Événements', 'Étudiants', 'Formation', 'Statistics', 'Dashboard', and 'Connexion'. The main content area has a table of students with columns: Nom, Prénom, Classe, Email, Date Inscription, Département, Niveaux, and Actions. The 'Actions' column contains 'Accepter' and 'Refuser' buttons.

Nom	Prénom	Classe	Email	Date Inscription	Département	Niveaux	Actions
Jane	Doe		jane.doe@example.com		Informatique	1	☒ Accepter ☒ Refuser
Doe	John		john.doe@example.com		CS	Undergraduate	☒ Accepter ☒ Refuser
Doe	John		john.doe@example.com		CS	Undergraduate	☒ Accepter ☒ Refuser
v	c		09720775		c	c	☒ Accepter ☒ Refuser
Doe	John		john.doe@example.com		CS	Undergraduate	☒ Accepter ☒ Refuser
Doe	John		john.doe@example.com		CS	Undergraduate	☒ Accepter ☒ Refuser
Doe	John		john.doe@example.com		CS	Undergraduate	☒ Accepter ☒ Refuser

Figure 52: INTERFACE GESTION DES ETUDIANTS

2. INTERFACE AJOUTER UN NOUVEAU ETUDIANT :

Pour ajouter un étudiant admin doit remplir les champs nécessaires puis cliquer sur le bouton " enregistrer "



The screenshot displays a web application interface for adding a new student. The top navigation bar includes links for Accueil, Paramètres, Réseaux 4C, Offres, Événements, Étudiants, Formation, Statistics, Dashboard, and Connexion. The left sidebar features the 4C logo, the institution's name (4C-ISET TOZEUR DGET), its full name (Institut Supérieur des Etudes Technologiques de Tozeur), a 'Qui sommes-nous ?' link, the name of the director (Belkacem Othman), and a profile picture. The main content area is titled 'Ajouter un nouvel étudiant' and contains a form with the following fields: Nom, Prénom, Classe, Email, a date field (jj/mm/aaaa), Département, and Niveaux. A green 'Enregistrer' button is located at the bottom of the form.

Figure 53:INTERFACE AJOUTER ETUDIANT

X. Conclusion :

Le Sprint 1 a réussi à mettre en place une authentification sécurisée et des interfaces utilisateur robustes, posant les bases pour les développements futurs. Le pipeline CI/CD garantit une intégration continue et un déploiement fiable des nouvelles fonctionnalités.

CHAPITRE

SPRINT 2

I. Introduction :

Ce chapitre détaille le déroulement du Sprint 2 dans le cadre du projet de microservices pour la gestion des formations au 4C ISET Tozeur. Ce sprint se concentre sur la mise en place des fonctionnalités de gestion des formations et des événements, ainsi que sur l'intégration continue et le déploiement des microservices développés.

II. SPRINT GOAL DU SPRINT 2 :

Au cours du Sprint 2 du projet microservices 3 gestion des formations, l'objectif est de mettre en place des composants et interfaces fondamentaux pour gérer et interagir avec les formations éducatives et les événements associés. Les principaux objectifs sont les suivants :

- Développer le Microservice de Gestion des Formations
- Préparer les Interfaces d'Administration pour la Gestion des Formations
- Créer des Interfaces pour Consulter les Informations des Formations
- Développer le Microservice de Gestion des Événements
- Développer les Interfaces pour Consulter et Gérer les Événements
- Brancher sur GitHub
- Intégrer les Tests JUnit et SonarQube
- Créer le Microservice de Contact
- Construire l'Interface de Contact

III. Backlog du sprint 2 :

TABLEAU 7 : BACKLOG DE SPRINT 2

ID	User Story	Cas	Priorité	Story Points
----	------------	-----	----------	--------------

1	Développer le Microservice de Gestion des Formations	Création CRUD avec Node.js	Haute	4
2	Préparer les Interfaces d'Administration pour Gérer les Formations	Préparer les interfaces avec Angular pour les administrateurs	Haute	2
3	Créer des Interfaces pour Consulter les Informations des Formations	Préparer des interfaces avec Angular pour les étudiants	Moyenne	2
4	Développer le Microservice de Gestion des Événements	Développer le microservice de gestion des événements avec Node.js	Moyenne	2
5	Développer les Interfaces pour Consulter et Gérer les Événements	Développer le frontend pour les administrateurs et les étudiants avec Angular	Moyenne	Non spécifié
6	Mettre en Branche sur GitHub	Mettre la version actuelle sur GitHub	Faible	1
7	Intégrer les Tests JUnit et SonarQube	Intégrer les tests JUnit et SonarQube	Faible	3

8	Créer le Microservice de Contact	Créer le microservice de contact	Faible	2
9	Créer l'Interface de Contact	Créer une interface de contact	Faible	2

Ce tableau présente de manière organisée les différentes user stories du Sprint 2, avec leurs cas associés, leur priorité et leur estimation en story points.

IV. ANALYSE FONCTIONNEL DE MICROSERVICES DE GESTION DES FORMATIONS :

✓ CRUD DES FORMATIONS :

- Création : Les administrateurs peuvent créer de nouvelles formations en fournissant les détails nécessaires tels que le titre, la description, la date, le lieu, etc.
- Lecture : Les utilisateurs peuvent consulter la liste des formations et événements disponibles avec des informations détaillées.
- Mise à Jour : Les administrateurs peuvent modifier les détails des formations existantes, par exemple, changer la date, mettre à jour la description, etc.
- Suppression : Les administrateurs peuvent supprimer des formations qui ne sont plus pertinentes ou annuler des événements.

✓ GESTION DES INSCRIPTIONS :

- Les étudiants peuvent s'inscrire aux formations disponibles via l'interface utilisateur.
- Les administrateurs peuvent consulter la liste des inscrits à chaque formation, gérer les inscriptions en attente et les confirmer.

✓ VALIDATION DES INSCRIPTIONS :

- Les administrateurs peuvent valider les inscriptions soumises par les étudiants, vérifiant ainsi la validité des informations fournies.

- Après validation, les étudiants reçoivent une notification par e-mail confirmant leur inscription avec les détails pertinents de la formation.

✓ NOTIFICATIONS PAR E-MAIL :

- Les notifications par e-mail sont envoyées automatiquement pour informer les utilisateurs des confirmations d'inscription, des changements d'horaires ou d'emplacement, et d'autres informations importantes liées aux formations et événements.
- Les e-mails doivent être configurables pour permettre une personnalisation en fonction des besoins spécifiques de chaque événement.

V. CONCEPTION DE MICROSERVICES GESTION DES FORMATIONS :

1. Diagramme de cas utilisation pour admin Gérer les formations

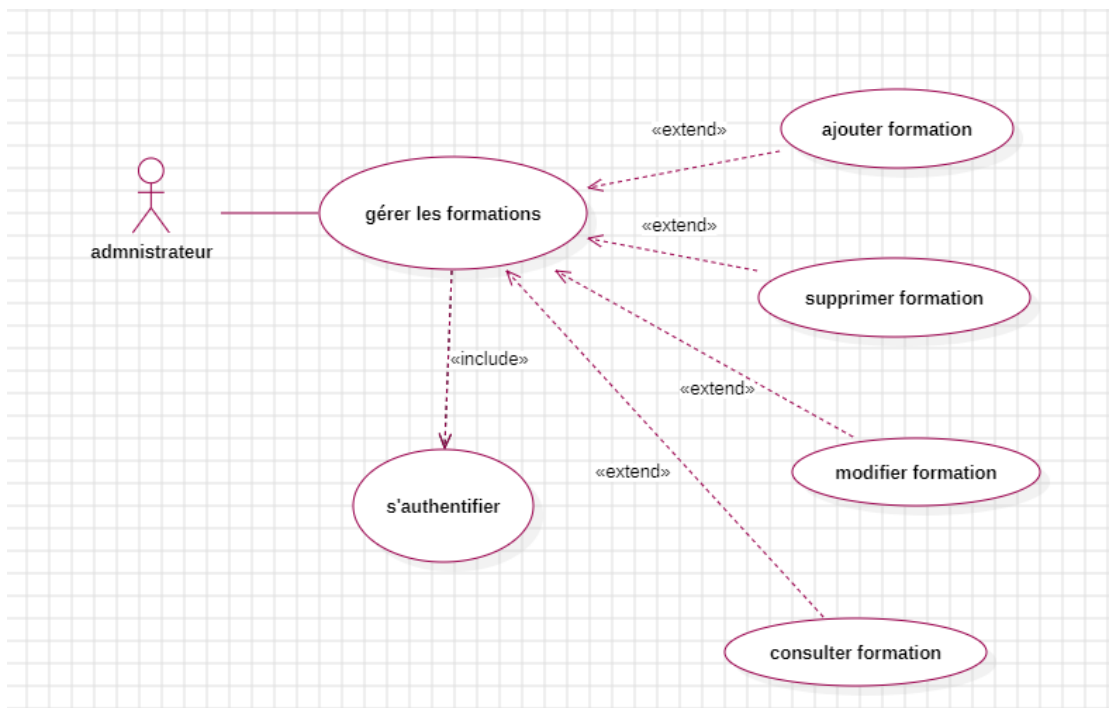


Figure 54:DIAGRAMME DE CAS UTILISATION POUR ADMIN GERER LES FORMATIONS

2. Diagramme de cas utilisation pour étudiant consulter les formations

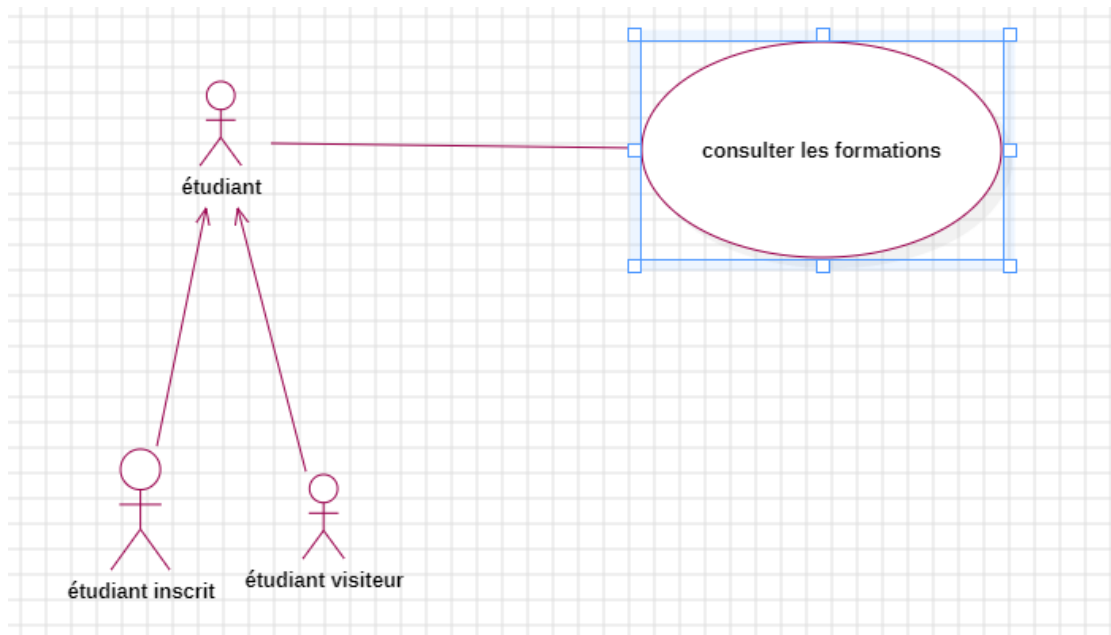


Figure 55:DIAGRAMME DE CAS UTILISATION POUR ETUDIANT CONSULTER LES FORMATIONS

3. Diagramme de séquence ajouter formations :

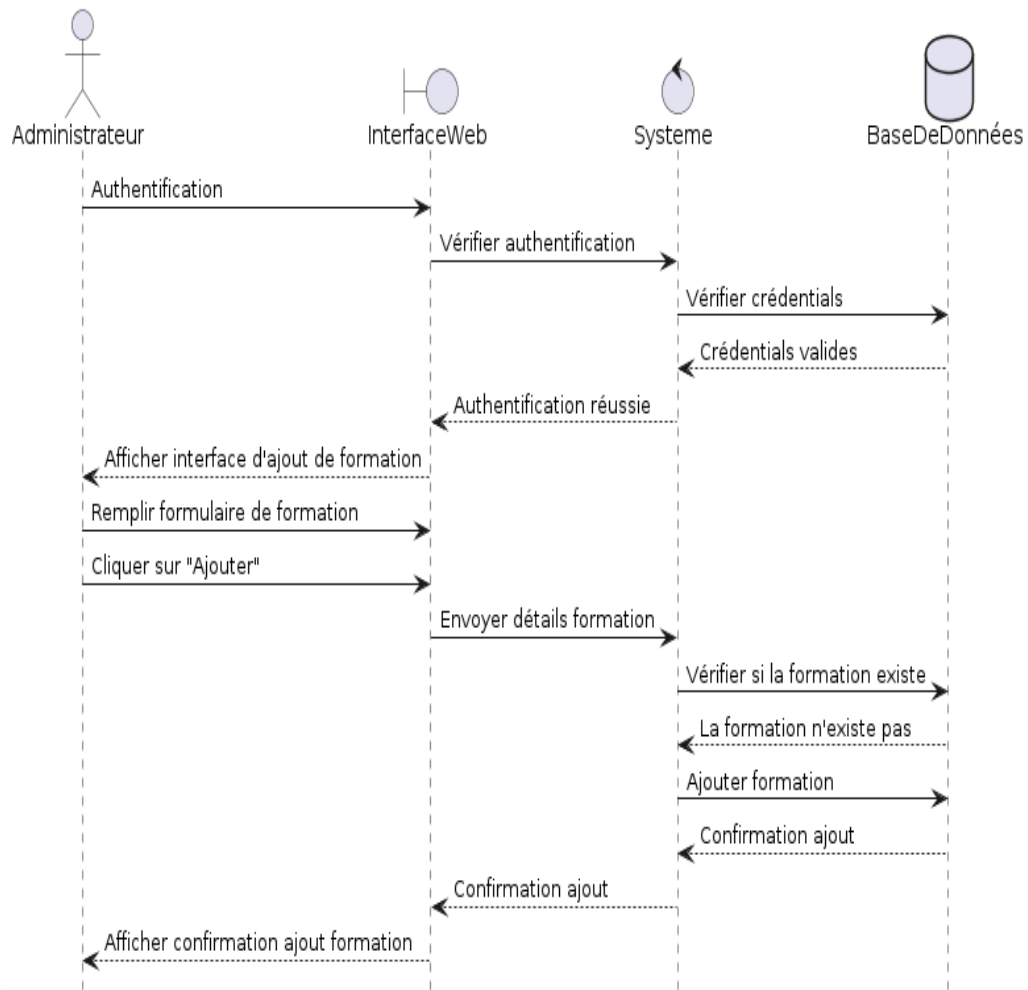


Figure 56: Diagramme de séquence ajouter formations

4. Diagramme de séquence Modifier formations :

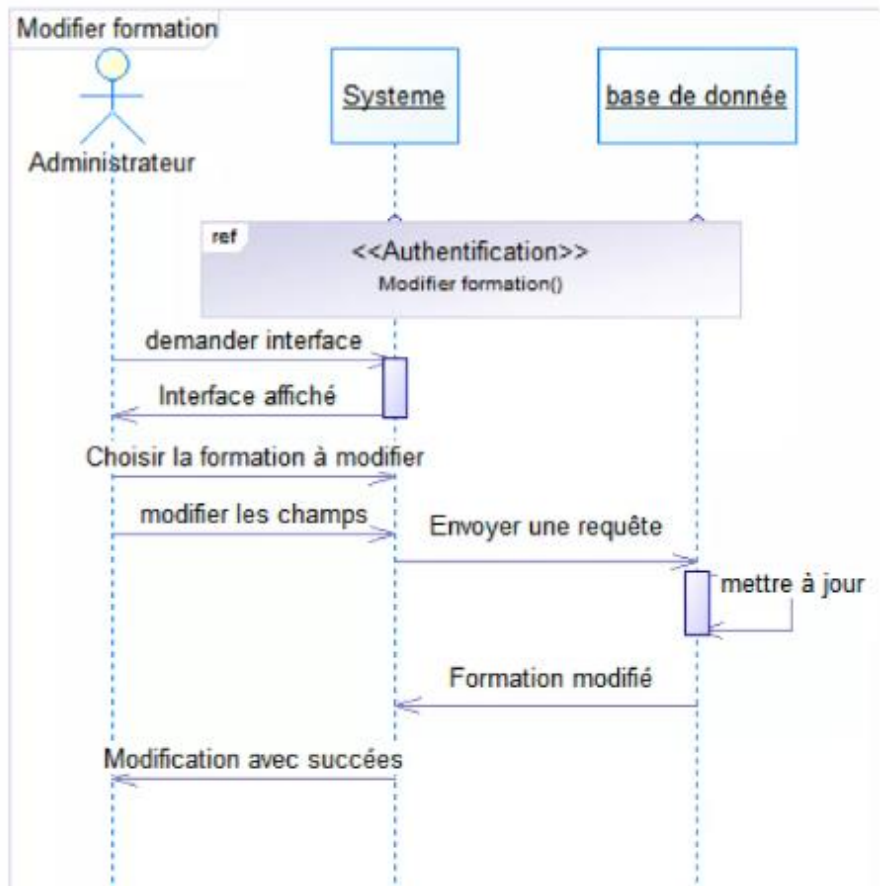


Figure 57: Diagramme de séquence Modifier formations

5. diagramme de classe de sprint 2 :

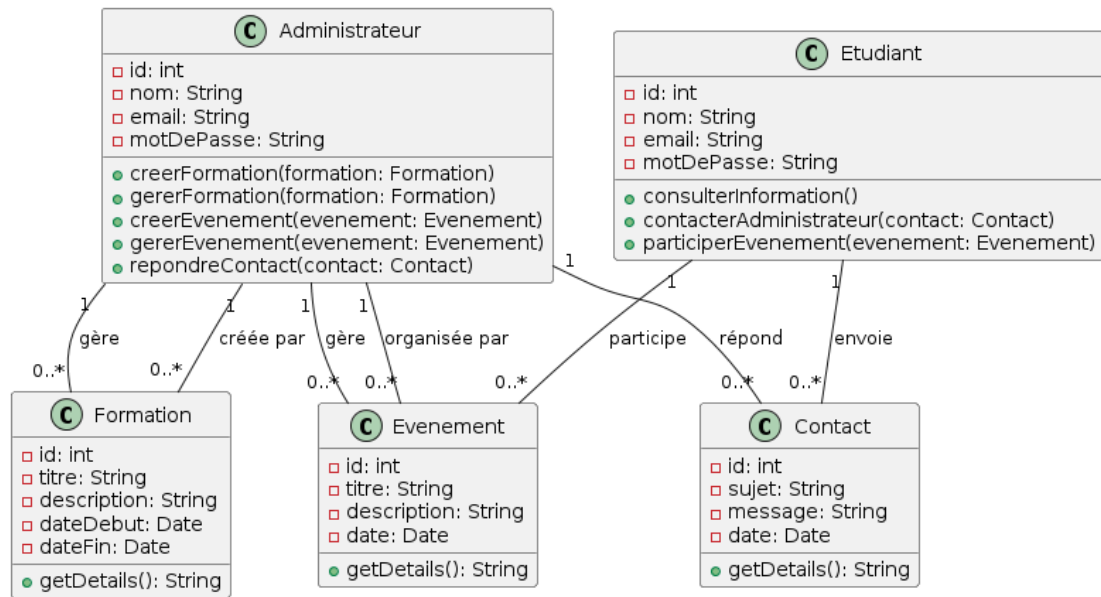


Figure 58:DIAGRAMME DE CLASSE SPRINT 2

VI. LES ETAPES DE CREATION DU PIPELINE POUR MICROSERVICES GESTION DES FORMATIONS

1. Plan :

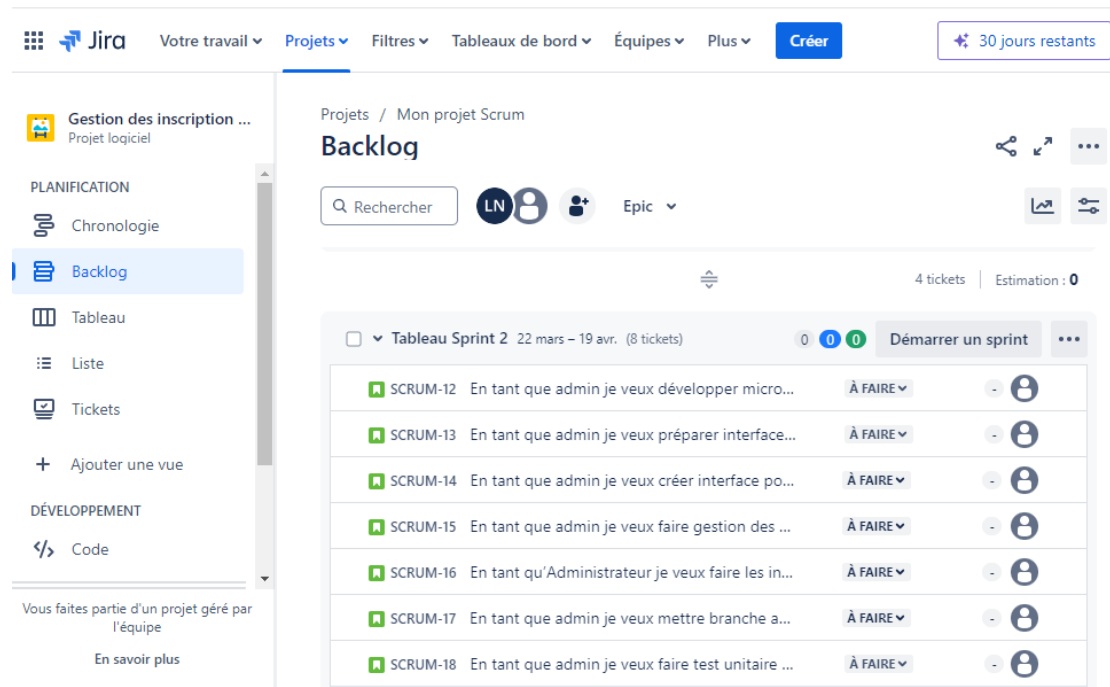


Figure 59:INTERFACE JIRA POUR SPRINT 2

2. accès au code source sur GitHub :

J'utilise Git pour cloner le repository GitHub contenant le code source des microservices de gestion des formations. Cela me permet d'accéder aux dernières versions du code et de collaborer efficacement avec l'équipe de développement.

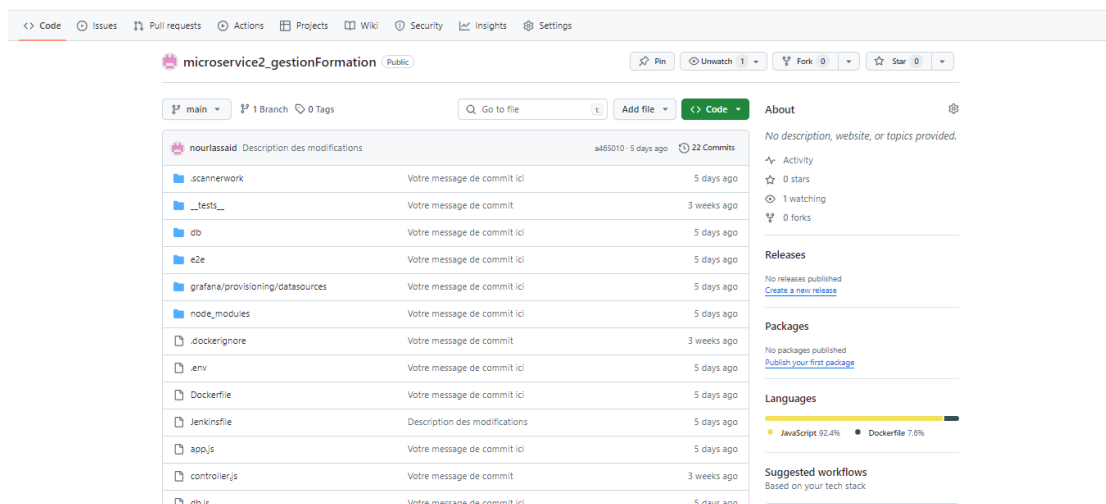


Figure 60: GITHUB POUR GESTION DES FORMATION Sprint 2

3. Exécuter les tests :

J'utilise JUnit pour automatiser l'exécution des tests unitaires, assurant ainsi que chaque composant du code fonctionne correctement selon les spécifications prévues.

```
C:\Users\MSAR\micro3_Formations>npm test

> micro3_ formations@1.0.0 test
> jest

PASS __tests__/_dbMock.js
PASS __tests__/_controller.test.js
A worker process has failed to exit gracefully and has been force exited. This is likely caused by tests leaking due to improper teardown. Try running with --detectOpenHandles to find leaks. Active timers can also cause this, ensure that .unref() was called on them.

Test Suites: 2 passed, 2 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        3.927 s
Ran all test suites.

C:\Users\MSAR\micro3_Formations>
```

Figure 61: CAPTURE DE JTEST POUR MICROSERVICE GESTIONN DES FORMATION

4. vérifier la qualité du code :

J'intègre SonarQube pour évaluer la qualité du code en détectant les erreurs, les bugs potentiels et les vulnérabilités, assurant ainsi une meilleure fiabilité et sécurité du logiciel.

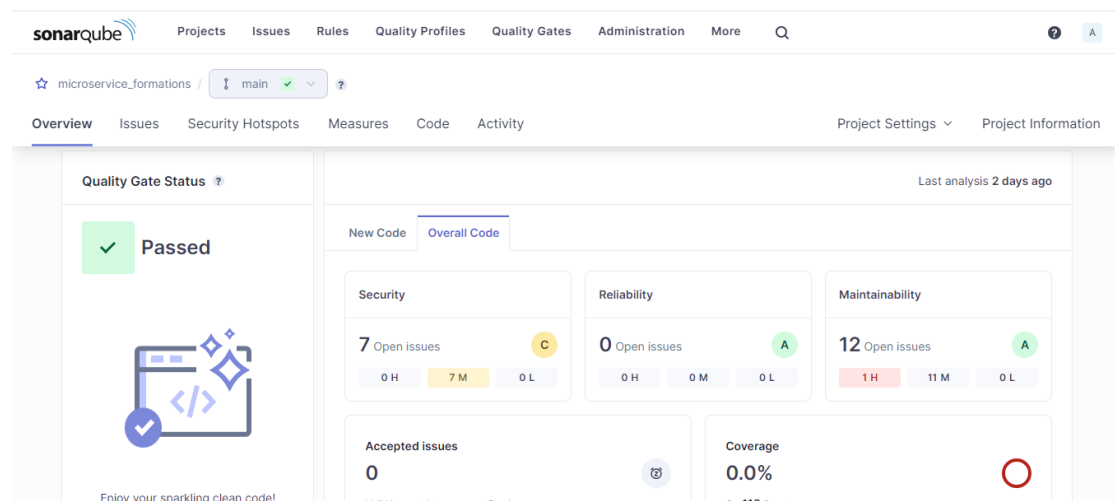


Figure 62: CAPTURE DE TEST SONARQUBE

5. CREATION DES IMAGES :

J'utilise Docker pour créer des images des microservices à partir des builds.

6. DEPLOIEMENT SUR KUBERNETES :

Je déploie les images Docker sur un cluster Kubernetes pour gérer le déploiement, la mise à l'échelle et la maintenance des applications containerisées.

- Outil Utilisé : Kubernetes

```
C:\Users\MSAR\micro3_Formations>kubectl get pods -n formation
NAME                                READY   STATUS    RESTARTS   AGE
backend-deployment-8595d6ccf-7lztw  1/1     Running   2 (8m30s ago)  29h
mysql-deployment-59545b959-v5fbl    1/1     Running   7 (8m30s ago)  3d3h
```

Figure 63: CAPTURE EXECUTION PODS

Cette figure represente base de donne dans contenuer mysql

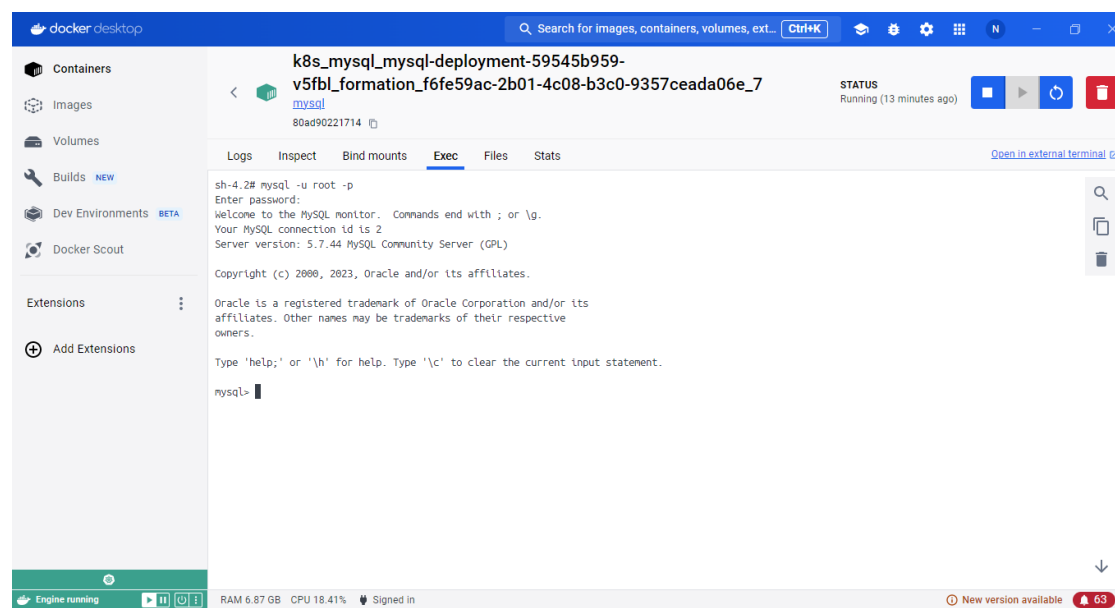


Figure 64: INTERFACE DE CONTENEUR MYSQL

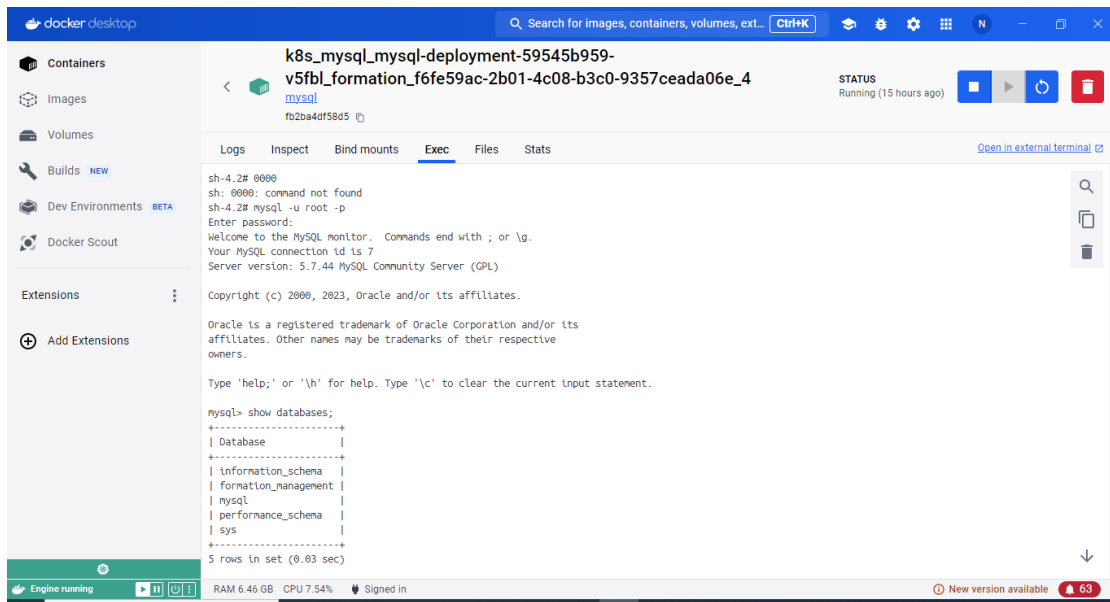


Figure 65:INTERFACE DE CONTENEUR MYSQL

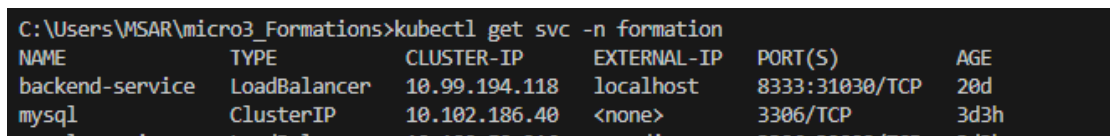


Figure 66:INTERFACE DE SERVICE

7. Monitoring des l'application :

J'utilise Prometheus et Grafana pour monitorer et analyser en temps réel les performances ainsi que la santé de l'application, assurant un suivi précis et efficace de ses métriques.

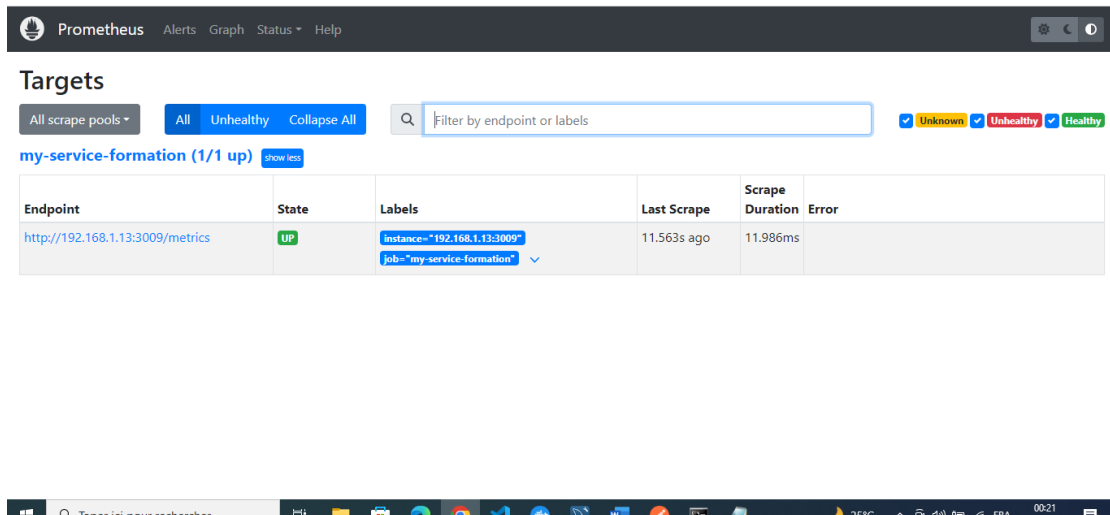


Figure 67:INTERFACE DE PROMETHEUS

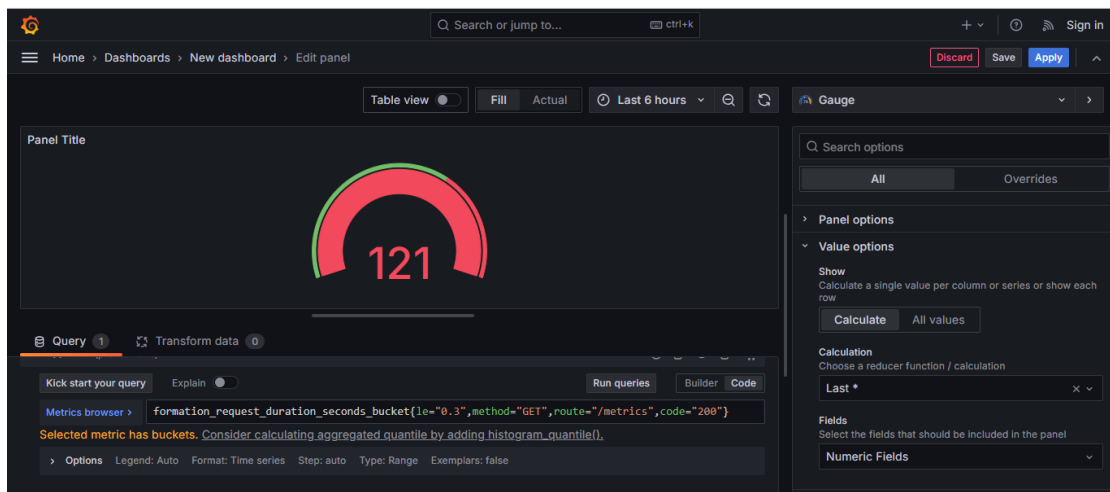


Figure 68:INTERFACE DE GRAFANA

8. JENKINS :

Utilisation de Jenkins pour orchestrer le pipeline de CI/CD.

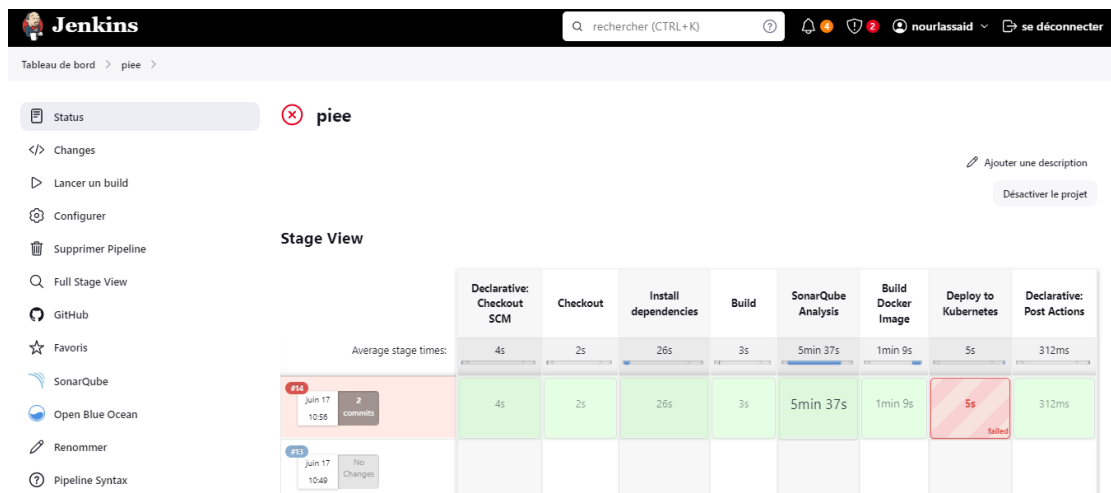


Figure 69:INTERFACE DE JENKINS

VII. REALISATION :

1. INTERFACE GERER GESTION DES FORMATIONS :

À travers cette interface admin peut gérer gestion des formations.



4C
ISET TOZEUR

DGET

Institut Supérieur des Etudes
Technologiques de Tozeur

Qui sommes-nous ?

Belkacem Othman
Directeur 4C




[Accueil](#)
[Paramètres](#)
[Réseaux 4C](#)
[Offres](#)
[Événements](#)
[Étudiants](#)
[Formation](#)
[Statistics](#)
[Dashboard](#)
[Connexion](#)

Ajouter Formation

Export to Excel

Nom	Description	Date début	Date fin	Lieu	Actions
Formation en développement web	Formation complète sur les technologies web modernes	2024-04-30T23:00:00.000Z	2024-06-29T23:00:00.000Z	En ligne	Éditer Supprimer
react	formation 1	2024-04-02T23:00:00.000Z	2024-04-18T23:00:00.000Z	tozeur	Éditer Supprimer
scrum		2024-04-18T23:00:00.000Z	2024-05-10T23:00:00.000Z	tozeur	Éditer Supprimer
cisco3	dd	2024-05-28T23:00:00.000Z	2024-06-08T23:00:00.000Z	tozeur	Éditer Supprimer

Figure 70:INTERFACE DE GESTION DES FORMATION POUR ADMIN

2. INTERFACE AJOUTER UN NOUVEAU FORMATION :

Pour ajouter une formation admin doit remplir les champs nécessaires puis cliquer sur le bouton " enregistrer "

Ajouter Formation

Export to Excel

Ajouter une nouvelle formation

big data

description

30/06/2024

02/07/2024

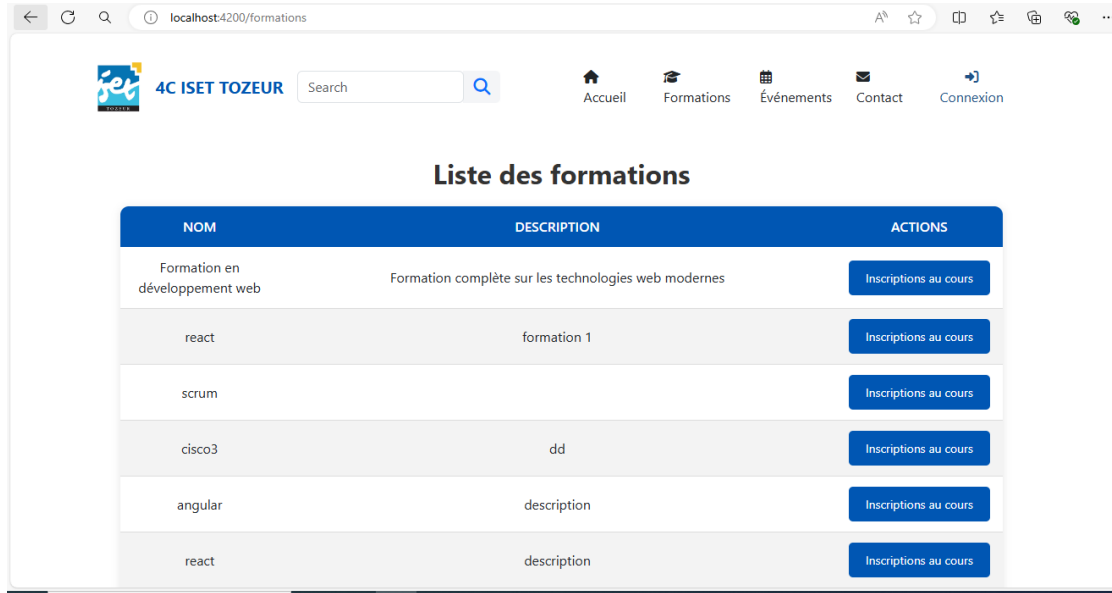
tozeur

Enregistrer

Figure 71:INTERFACE D'AJOUTER FORMATION POUR ADMIN

3. INTERFACE CONSULTER LISTE DES FORMATIONS :

La figure ci-dessous montre l'affichage des description des formations



NOM	DESCRIPTION	ACTIONS
Formation en développement web	Formation complète sur les technologies web modernes	Inscriptions au cours
react	formation 1	Inscriptions au cours
scrum		Inscriptions au cours
cisco3	dd	Inscriptions au cours
angular	description	Inscriptions au cours
react	description	Inscriptions au cours

Figure 72:INTERFACE DE CONSULTER DES FORMATION POUR ETUDIANT

4. INTERFACE DE FORMULAIRE DE PARTICIPATION :

Cette interface s'affiche formulaire lorsque étudiant demande participer formation.

Formulaire de Participation

Nom:
nour

Prénom:
lassaid

Email:
nourlassaid505@gmail.com

Envoyer

Figure 73:INTERFACE DE PARTICIPER A UNE FORMATION POUR ETUDIANT

5. INTERFACE DE CONFIRMATION DE PARTICIPATION :

Cette interface affiche confirmation de participation

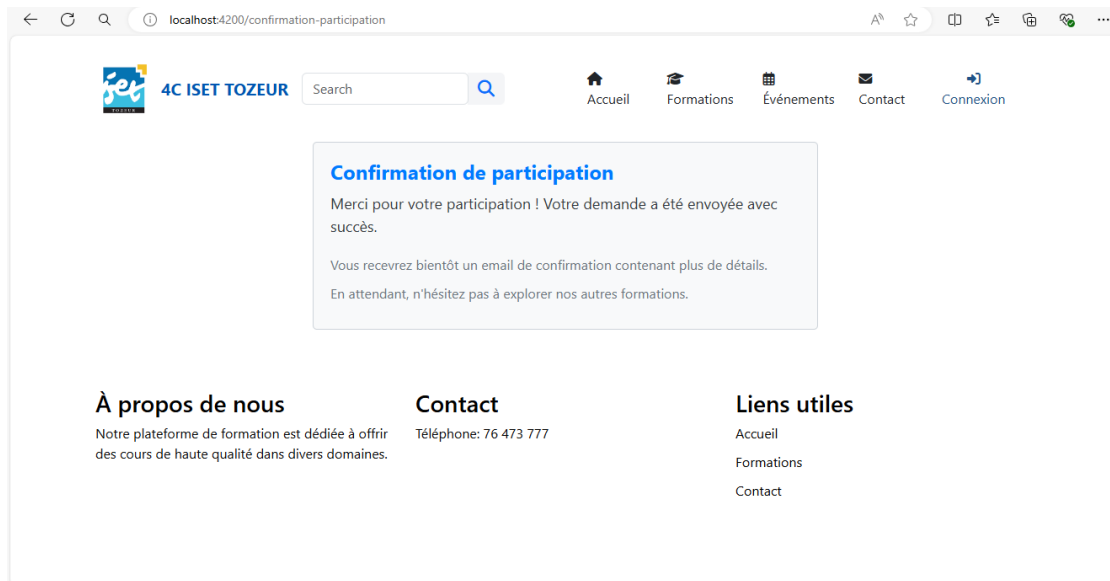


Figure 74: INTERFACE DE CONFIRMATION A UNE FORMATION POUR ETUDIANT

VIII. MICROSERVICES 3 GESTION DES EVENEMENTS

1. ANALYSE FONCTIONNEL DE MICROSERVICES DE GESTION DES EVENEMENTS :

❖ Objectifs de la Fonctionnalité :

- Objectif principal: Développer une fonctionnalité de gestion des événements qui permet aux administrateurs et aux étudiants de créer, consulter, modifier et supprimer des événements.
- Rôles des Utilisateurs :

Administrateur :

- Créer des événements: Ajouter de nouveaux événements avec des détails spécifiques (titre, description, date, heure, lieu, etc.).
- Modifier des événements: Mettre à jour les informations des événements existants.

- Supprimer des événements: Supprimer des événements qui ne sont plus nécessaires.
- Consulter les événements: Voir la liste complète des événements.

Étudiant :

- Consulter les événements: Voir les événements disponibles et leurs détails.
- S'inscrire ou se désinscrire à des événements (si applicable).

2. CONCEPTION DE GESTION DES EVENEMENTS

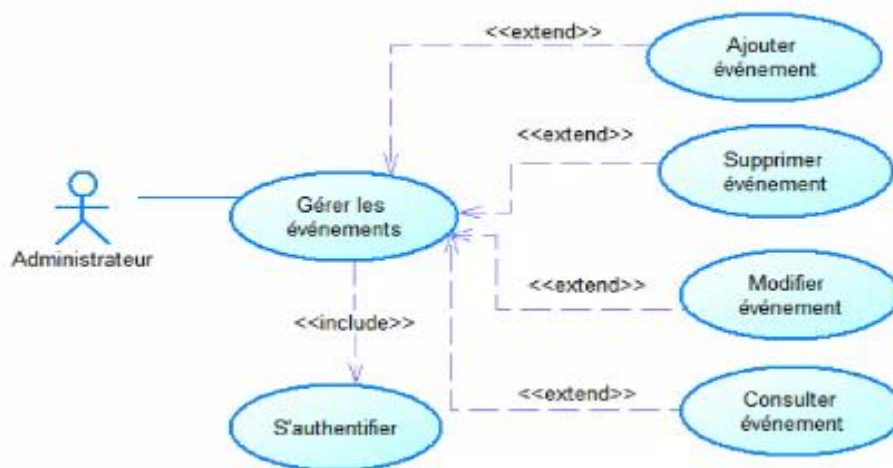


Figure 75:DIAGRAMME DE CAS UTILISATION GERER DES EVENEMENTS

3. LES ETAPES DE CREATION DU PIPELINE POUR MICROSERVICES GESTION DES EVENEMENTS :

a. Planification :

Pour planifier les étapes de développement, nous utilisons Jira. Jira permet de créer, suivre et gérer les tickets de développement. Chaque fonctionnalité ou correction

de bug est planifiée sous forme de tickets, avec des estimations de temps et des dépendances claires.

b. Gestion du Code Source :

GitHub Le code source des microservices est géré dans des dépôts GitHub. GitHub offre des fonctionnalités de gestion de versions, de branches et de collaboration, facilitant le suivi des modifications et des contributions.

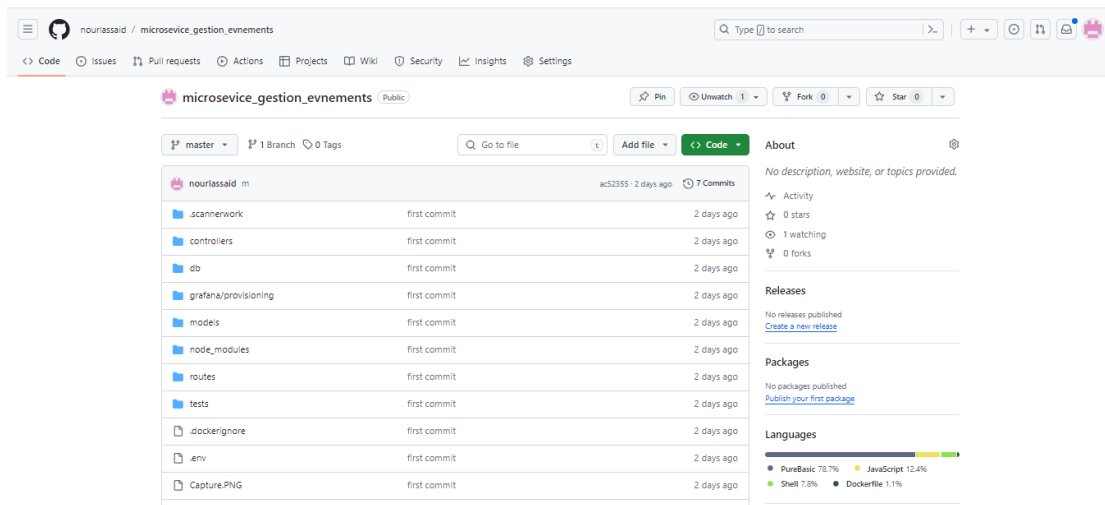


Figure 76: GITHUB GERER DES EVENEMENTS

c. Tests Unitaires et d'Intégration :

JUnit Les tests unitaires et d'intégration sont effectués à l'aide de JUnit. JUnit permet de s'assurer que chaque unité de code fonctionne comme prévu et que l'intégration entre les différentes parties du système est correcte.

```
C:\Users\MSAR\gestion-evenements-nodejs>npm test

> gestion-evenements-nodejs@1.0.0 test
> jest

  console.log
    Executing (default): SELECT `id`, `title`, `description`, `date`, `time`, `location`, `category`, `createdAt`, `updatedAt` FROM `Events` AS `Event`;
    at Sequelize.log (node_modules/sequelize/src/sequelize.js:1281:15)

  console.log
    Executing (default): INSERT INTO `Events` (`id`,`title`,`description`,`date`,`time`,`location`,`category`,`createdAt`,`updatedAt`) VALUES (DEFAULT,?,?,?,?,?,?,?);
    at Sequelize.log (node_modules/sequelize/src/sequelize.js:1281:15)

PASS tests/events.test.js
Event Routes
  ✓ should get all events (485 ms)
  ✓ should create a new event (146 ms)

Test Suites: 1 passed, 1 total
Tests: 2 passed, 2 total
Snapshots: 0 total
Time: 4.168 s
Ran all test suites.
Jest did not exit one second after the test run has completed.

'This usually means that there are asynchronous operations that weren't stopped in your tests. Consider running Jest with `--detectOpenHandles` to troubleshoot this issue.'

C:\Users\MSAR\gestion-evenements-nodejs>
```

Figure 77: CAPTURE DE JTEST

d. Analyse de la Qualité du Code :

SonarQube SonarQube est utilisé pour analyser la qualité du code. Il vérifie la présence de bugs, de vulnérabilités, et de problèmes de maintenabilité. SonarQube fournit également des métriques de couverture de code.

```

INFO: Sensor TextAndSecretsSensor [text]
INFO: Available processors: 4
INFO: Using 4 threads for analysis.
INFO: 572 source files to be analyzed
INFO: 571/572 files analyzed, current file: node_modules/moment-timezone/data/packed/latest.json
INFO: 572/572 source files have been analyzed
INFO: Sensor TextAndSecretsSensor [text] (done) | time=12961ms
INFO: ----- Run sensors on project
INFO: Sensor Zero Coverage Sensor
INFO: Sensor Zero Coverage Sensor (done) | time=249ms
INFO: SCM Publisher No SCM system was detected. You can use the 'sonar.scm.provider' property to explicitly
ify it.
INFO: CPD Executor 2 files had no CPD blocks
INFO: CPD Executor Calculating CPD for 8 files
INFO: CPD Executor CPD calculation finished (done) | time=37ms
INFO: Analysis report generated in 242ms, dir size=322.6 kB
INFO: Analysis report compressed in 311ms, zip size=85.4 kB
INFO: Analysis report uploaded in 101ms
INFO: ANALYSIS SUCCESSFUL, you can find the results at: http://localhost:9000/dashboard?id=microservice_e
INFO: Note that you will be able to access the updated dashboard once the server has processed the submit

```

Figure 78: CAPTURE 1 DE TEST SONARQUBE

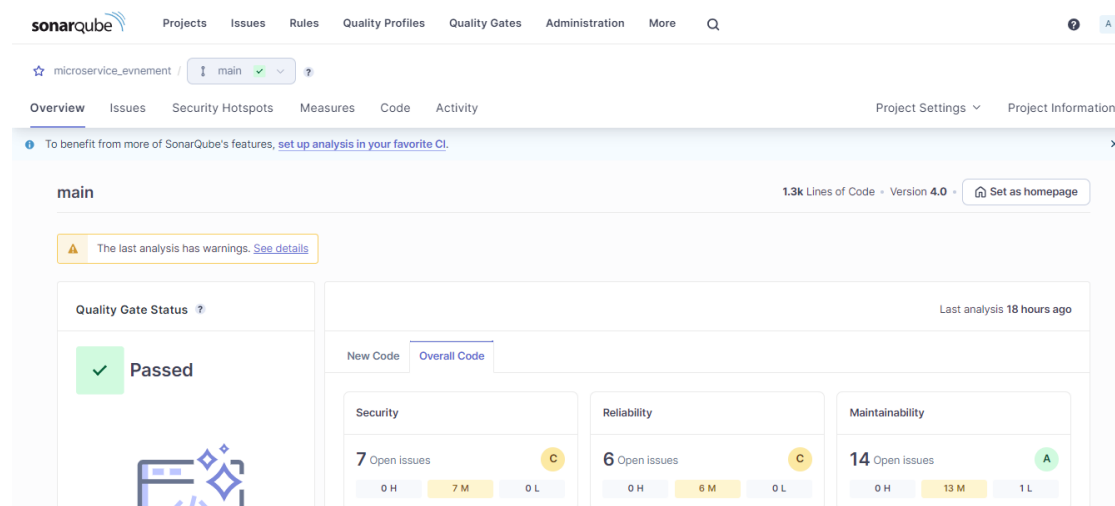


Figure 79: CAPTURE 2 DE TEST SONARQUBE

e. Création de l'Image Docker

L'image Docker des microservices est créée à partir des artefacts générés. Le Dockerfile spécifie les instructions pour construire l'image, incluant l'installation des dépendances et la configuration de l'environnement

f. Déploiement sur Kubernetes

Les images Docker sont déployées sur un cluster Kubernetes. Kubernetes gère l'orchestration des conteneurs, assurant leur disponibilité et leur scalabilité.

Les configurations de déploiement sont définies dans des fichiers YAML.

```
C:\Users\MSAR\gestion-evenements-nodejs>kubectl get deployments -n events
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
backend-deployment  1/1     1             1           27h
mysql-deployment    1/1     1             1           27h

C:\Users\MSAR\gestion-evenements-nodejs>
```

Figure 80:capture execution deployments

```
C:\Users\MSAR\gestion-evenements-nodejs\db>kubectl get pods -n events
NAME                                READY   STATUS    RESTARTS      AGE
backend-deployment-7cd6564bdc-px6m5  1/1     Running   50 (4h30m ago)  17h
mysql-deployment-7d9d857cb6-vv197    1/1     Running   1 (4h30m ago)  17h

C:\Users\MSAR\gestion-evenements-nodejs\db>kubectl logs backend-deployment-7cd6564bdc-px6m5
```

Figure 81:CAPTURE EXECUTION PODS

```
C:\Users\MSAR\gestion-evenements-nodejs\db>kubectl logs backend-deployment-7cd6564bdc-px6m5 -n events
Serveur en cours d'exécution sur le port 3001
Connexion à la base de données MySQL réussie
```

Figure 82:CAPTURE EXECUTION LOGS

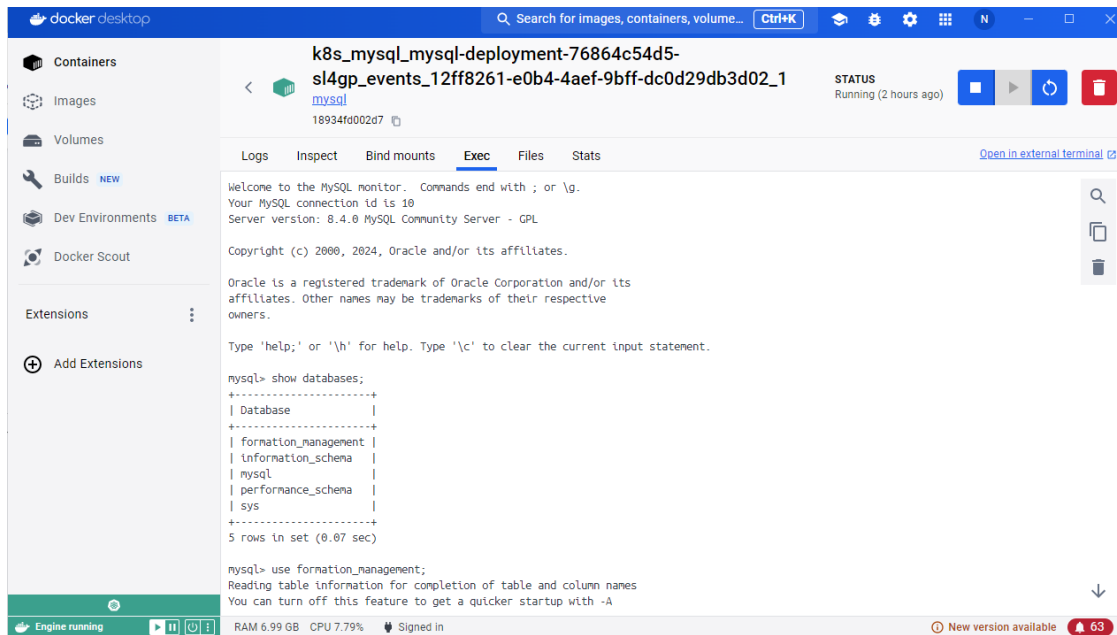


Figure 83:INTERFACE DE CONTENEUR MYSQL

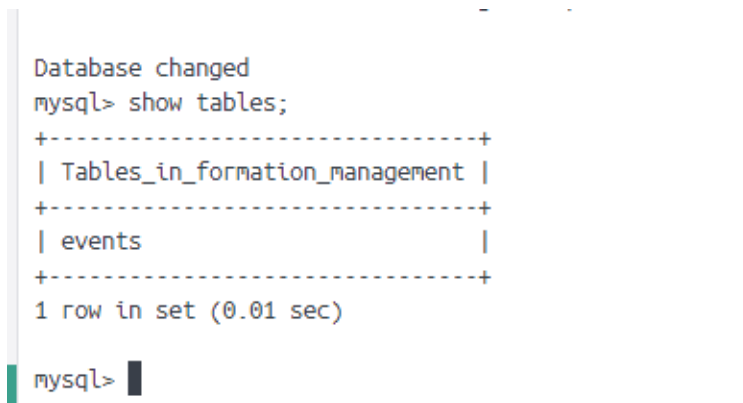


Figure 84:INTERFACE DE CONTENEUR MYSQL

g. Monitoring : Prometheus & Grafana

Le monitoring des microservices est effectué avec Prometheus et Grafana. Prometheus collecte les métriques des services et Grafana les visualise, permettant de surveiller les performances et la santé des applications.

```
localhost:5001/metrics

nodejs_gc_duration_seconds_bucket{le="0.1",kind="incremental"} 2
nodejs_gc_duration_seconds_bucket{le="1",kind="incremental"} 2
nodejs_gc_duration_seconds_bucket{le="2",kind="incremental"} 2
nodejs_gc_duration_seconds_bucket{le="5",kind="incremental"} 2
nodejs_gc_duration_seconds_bucket{le="10",kind="incremental"} 2
nodejs_gc_duration_seconds_sum{kind="incremental"} 0.001788399998867513
nodejs_gc_duration_seconds_count{kind="incremental"} 2
nodejs_gc_duration_seconds_bucket{le="0.001",kind="major"} 0
nodejs_gc_duration_seconds_bucket{le="0.01",kind="major"} 0
nodejs_gc_duration_seconds_bucket{le="0.1",kind="major"} 1
nodejs_gc_duration_seconds_bucket{le="1",kind="major"} 1
nodejs_gc_duration_seconds_bucket{le="2",kind="major"} 1
nodejs_gc_duration_seconds_bucket{le="5",kind="major"} 1
nodejs_gc_duration_seconds_bucket{le="10",kind="major"} 1
nodejs_gc_duration_seconds_sum{kind="major"} 0.0193383000000676187
nodejs_gc_duration_seconds_count{kind="major"} 1

# HELP eventement_request_duration_seconds Duration of eventement-ms service HTTP requests in microseconds
# TYPE eventement_request_duration_seconds histogram
eventement_request_duration_seconds_bucket{le="0.1",method="GET",route="/metrics",code="200"} 1
eventement_request_duration_seconds_bucket{le="0.3",method="GET",route="/metrics",code="200"} 1
eventement_request_duration_seconds_bucket{le="0.5",method="GET",route="/metrics",code="200"} 1
eventement_request_duration_seconds_bucket{le="0.7",method="GET",route="/metrics",code="200"} 1
eventement_request_duration_seconds_bucket{le="1",method="GET",route="/metrics",code="200"} 1
eventement_request_duration_seconds_bucket{le="3",method="GET",route="/metrics",code="200"} 1
eventement_request_duration_seconds_bucket{le="5",method="GET",route="/metrics",code="200"} 1
eventement_request_duration_seconds_bucket{le="7",method="GET",route="/metrics",code="200"} 1
eventement_request_duration_seconds_bucket{le="10",method="GET",route="/metrics",code="200"} 1
eventement_request_duration_seconds_bucket{le="Inf",method="GET",route="/metrics",code="200"} 1
eventement_request_duration_seconds_sum{method="GET",route="/metrics",code="200"} 0.0308568
eventement_request_duration_seconds_count{method="GET",route="/metrics",code="200"} 1
eventement_request_duration_seconds_bucket{le="0.1",method="GET",route="/favicon.ico",code="404"} 1
eventement_request_duration_seconds_bucket{le="0.3",method="GET",route="/favicon.ico",code="404"} 1
eventement_request_duration_seconds_bucket{le="0.5",method="GET",route="/favicon.ico",code="404"} 1
eventement_request_duration_seconds_bucket{le="0.7",method="GET",route="/favicon.ico",code="404"} 1
eventement_request_duration_seconds_bucket{le="1",method="GET",route="/favicon.ico",code="404"} 1
eventement_request_duration_seconds_bucket{le="3",method="GET",route="/favicon.ico",code="404"} 1
eventement_request_duration_seconds_bucket{le="5",method="GET",route="/favicon.ico",code="404"} 1
eventement_request_duration_seconds_bucket{le="7",method="GET",route="/favicon.ico",code="404"} 1
eventement_request_duration_seconds_bucket{le="10",method="GET",route="/favicon.ico",code="404"} 1
eventement_request_duration_seconds_bucket{le="Inf",method="GET",route="/favicon.ico",code="404"} 1
eventement_request_duration_seconds_sum{method="GET",route="/favicon.ico",code="404"} 0.0136479
eventement_request_duration_seconds_count{method="GET",route="/favicon.ico",code="404"} 1
```

Figure 85:INTERFACE DE METRICS

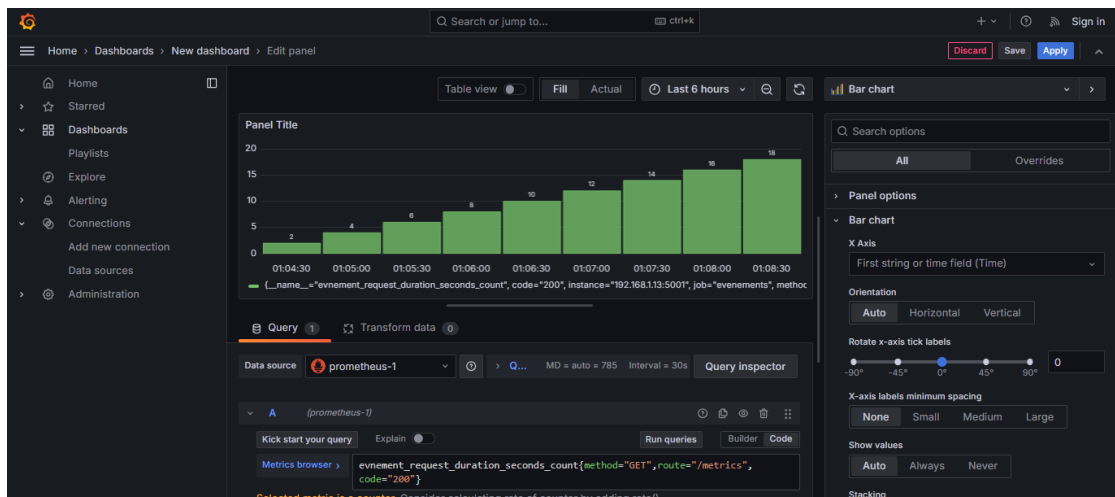


Figure 86:INTERFACE DE GARFANA

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://192.168.1.13:5001/metrics	UP	instance="192.168.1.13:5001" job="evenements"	29.164s ago	24.765ms	

Figure 87:INTERFACE DE GARFANA

h. Pipeline de CI/CD :

Jenkins Jenkins est utilisé pour orchestrer le pipeline de CI/CD. Le fichier Jenkinsfile contient les étapes définissant le processus de build, de test, de création d'image, et de déploiement. Jenkins automatise et coordonne ces étapes pour chaque commit ou pull request.

IX. REALISATION DES MICRO SERVICES DE GESTION EVENEMENTS :

1. INTEFACE DE GERER DES EVNEMENTS

Cette Figure Affiche Gestion Des Evènements Pour Admin

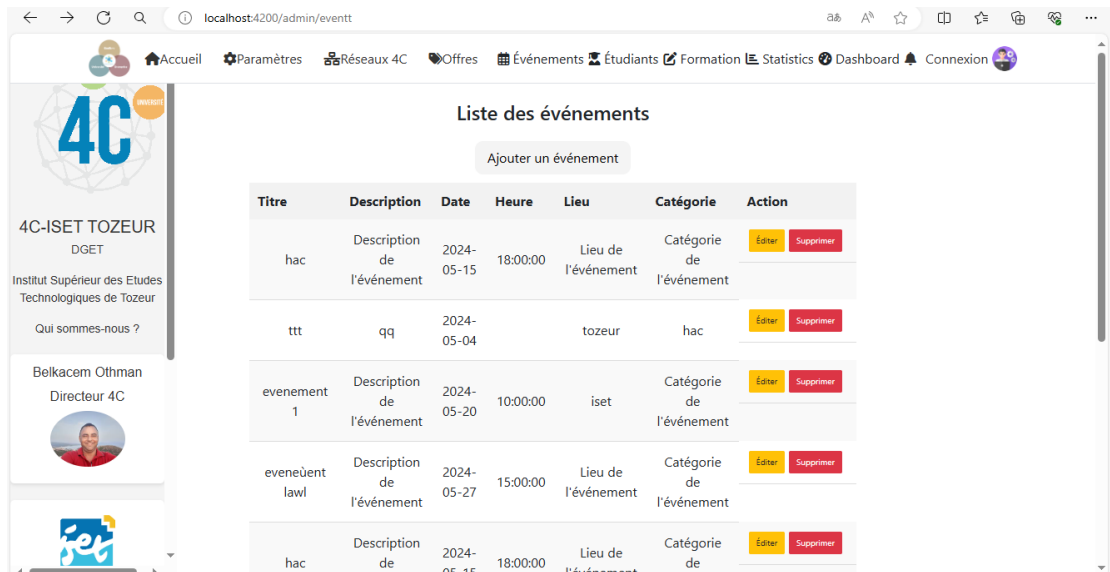


Figure 88:INTERFACE DES LISTE DES EVNEMENTS POUR ADMIN

2. INTERFACE DAJOUTER NOUVEL EVENMENTS :

Pour ajouter un événement doit admin remplir les champs nécessaires puis cliquer sur le bouton " ajouter"

Ajouter un événement

Titre

Description

Date

Heure

Lieu

Figure 89:INTERFACE AJOUTER EVNEMENTS POUR ADMIN

3. INTERFACE DE CONSULTER LES EVNEMENTS POUR ETUDIANTS



Figure 90:INTERFACE CONSULTER DES EVNEMENTS POUR ETUDIANTS

X. Conclusion :

Au cours de ce sprint, j'ai réussi à atteindre les objectifs fixés, en développant des microservices pour la gestion des formations et des événements, en intégrant des outils de test et d'assurance qualité, et en mettant en place un pipeline de CI/CD robuste. Les fonctionnalités développées permettront d'améliorer considérablement la gestion des formations et des événements, offrant une interface utilisateur intuitive et des outils d'administration efficaces.

CHAPITRE 8

SPRINT 3

I. Introduction :

Le Sprint 3 s'est concentré sur l'amélioration et le déploiement de la partie frontend du projet de gestion des formations, intégrant plusieurs étapes DevOps pour un développement et un déploiement automatisés et efficaces.

II. OBJECTIFS DU SPRINT 3 :

Dans ce sprint, j'ai focalisé nos efforts sur l'amélioration et le déploiement de la partie frontend du projet, ainsi que sur l'intégration de plusieurs étapes DevOps pour assurer un développement et un déploiement fluides et automatisés.

III. BACKLOG SPRINT 3 :

Le tableau ci-dessous présente le backlog de notre troisième sprint :

TABLEAU 8 : BACKLOG DE SPRINT 3

ID	USER STORY	PRIORITÉ	STORY POINT (j)
1	Corriger les erreurs frontend et push à GitHub	Haute	5
2	Créer Dockerfile pour frontend	Moyenne	2
3	Créer Jenkinsfile pour frontend	Moyenne	3
4	Effectuer tests Selenium et SonarQube pour frontend	Moyenne	4
5	Déployer avec Kubernetes pour frontend et push sur GitHub	Faible	5

IV. REALISATION PARTIE FRONTEND

- Correction des erreurs et mise à jour sur GitHub

▪ Description :

J'ai corrigé diverses erreurs identifiées dans la partie frontend. Ces corrections incluent des ajustements de l'interface utilisateur, la résolution de bugs fonctionnels et l'amélioration de la performance.

▪ Étapes :

- Identification des erreurs via des tests manuels et automatisés.
- Correction des erreurs dans le code source.
- Validation des corrections par des tests unitaires et d'intégration.
- Poussée des modifications sur le dépôt GitHub.

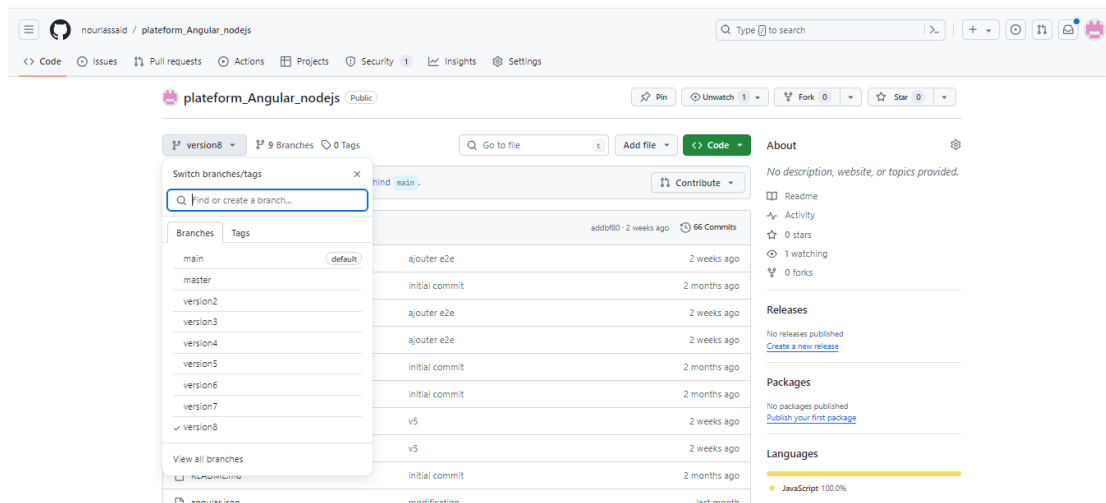


Figure 91: INTERFACE DE GESTION DES VERSION GITHUB

1. Création d'un Dockerfile

▪ Description :

J'ai créé un Dockerfile pour le frontend, permettant de construire une image Docker de l'application.

▪ Étapes :

- Écriture d'un Dockerfile définissant l'environnement et les dépendances nécessaires.
- Construction de l'image Docker localement pour les tests.
- Validation de l'image en exécutant l'application dans un conteneur Docker.
- Poussée du Dockerfile sur le dépôt GitHub.

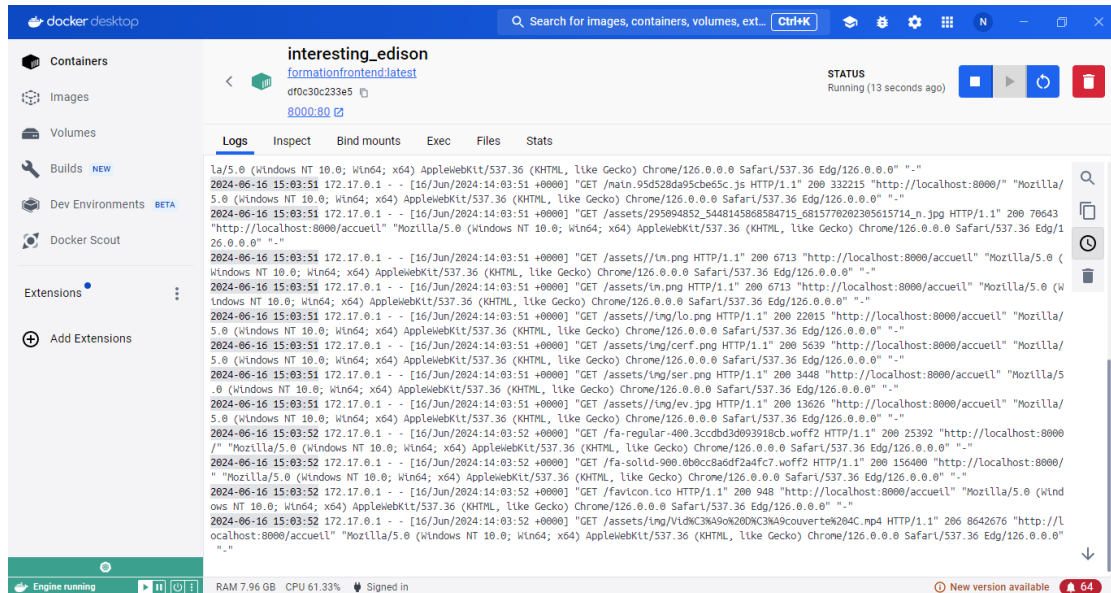


Figure 92: INTERFACE DE CREATION IMAGE POUR FRONTEND

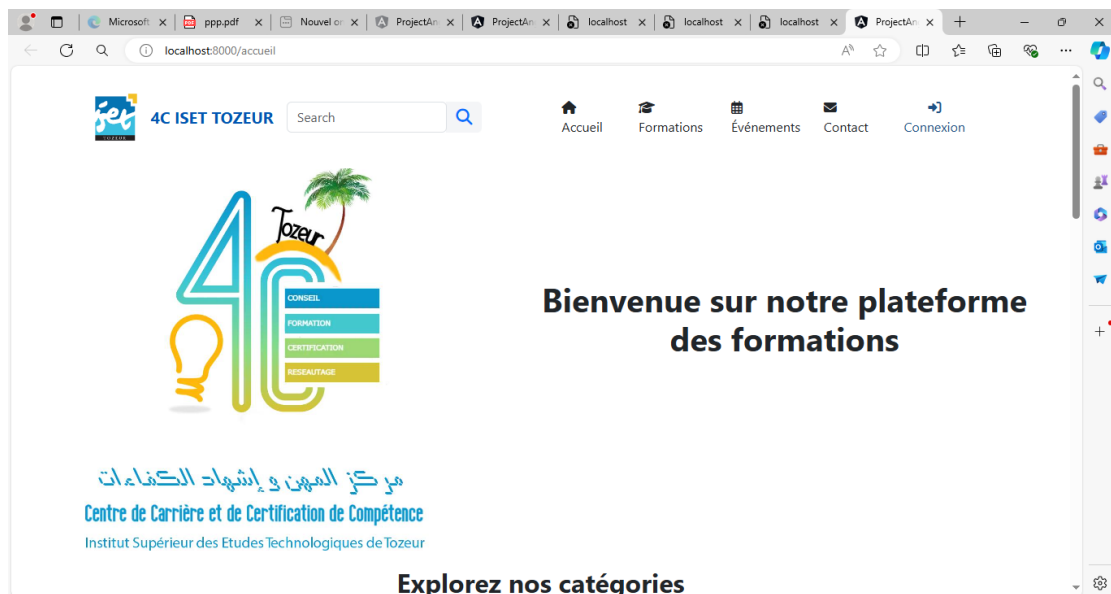


Figure 93: INTERFACE IMAGE POUR FRONTEND

2. Création d'un Jenkinsfile

- Description : j'ai créé un Jenkinsfile pour automatiser le processus de CI/CD pour le frontend.
- Étapes :
 - Écriture d'un Jenkinsfile avec les étapes de build, test et déploiement.
 - Intégration du Jenkinsfile dans le pipeline Jenkins.
 - Exécution et validation du pipeline sur Jenkins.

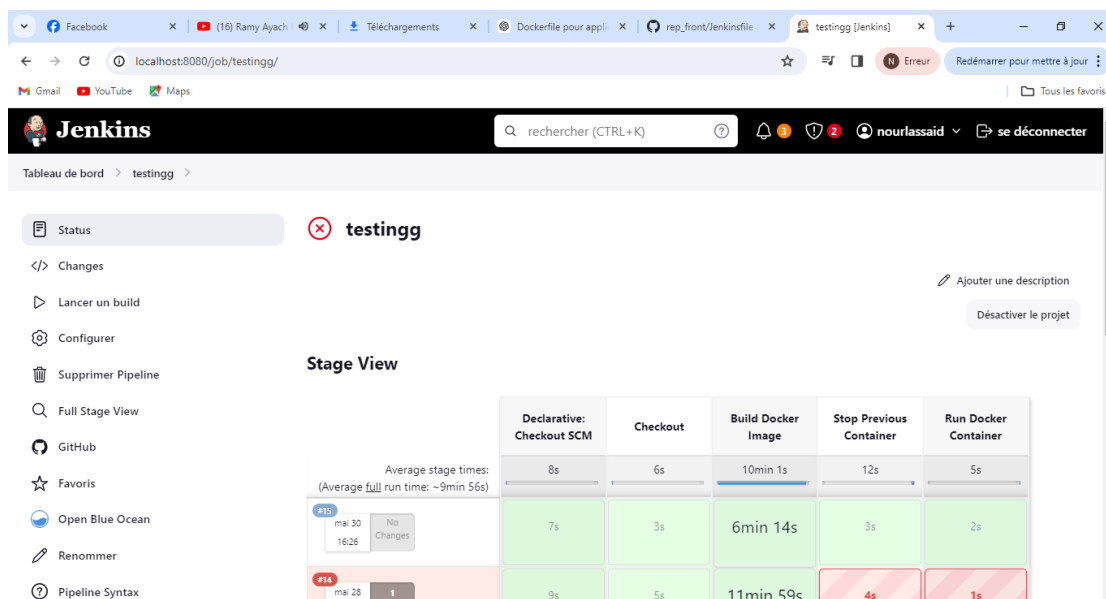


Figure 94: INTERFACE JENKINS PIPELINE POUR FRONTEND

3. Tests Selenium et SonarQube

- Description : J'ai effectué des tests de bout en bout avec Selenium et des analyses de qualité de code avec SonarQube pour le frontend.
- Étapes :
 - Écriture et exécution des scripts de tests Selenium.
 - Configuration de SonarQube pour analyser le code source du frontend.
 - Exécution des analyses et correction des problèmes identifiés.
 - Documentation des résultats des tests et des analyses.

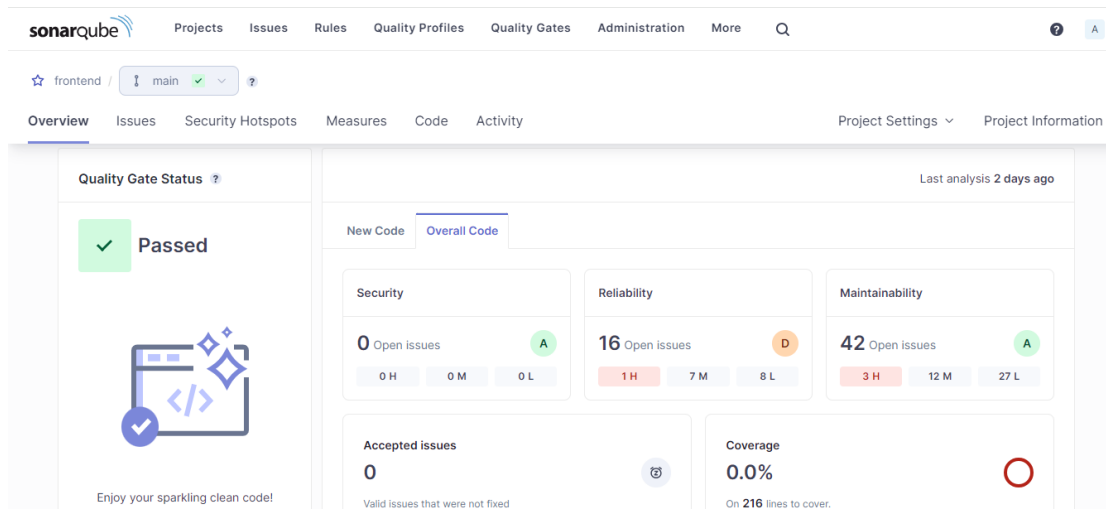


Figure 95:INTERFACE TEST SONARQUEB POUR FRONTEND

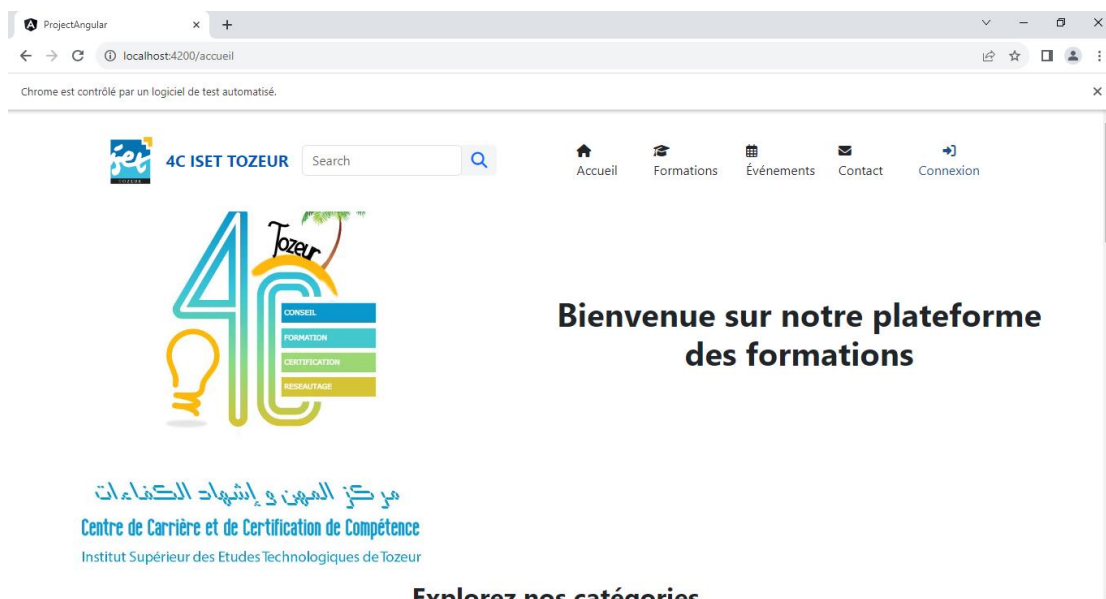


Figure 96:INTERFACE TEST SELNUM POUR FRONTEND

4. Déploiement avec Kubernetes

- Description : j'ai déployé le frontend à l'aide de Kubernetes, assurant un déploiement scalable et résilient.
- Étapes :
 - Création des fichiers de configuration Kubernetes (Deployment, Service, etc.).

- Déploiement de l'application sur un cluster Kubernetes.
- Validation du déploiement par des tests fonctionnels.
- Mise à jour du dépôt GitHub avec les fichiers de configuration Kubernetes.

```
C:\Users\MSAR\projectAngular>kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
angular-deployment-75c6d6c7bf-46vx5 1/1     Running   25 (77m ago)  10d
angular-deployment-75c6d6c7bf-5-746 1/1     Running   24 (77m ago)  10d
```

Figure 97.: INTERFACE DEPLOYMENT POUR FRONTEND PODS

```
C:\Users\MSAR\projectAngular>kubectl get svc
NAME            TYPE          CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
angular-service LoadBalancer  10.98.122.95   localhost    80:32251/TCP     19d
```

Figure 98.:INTERFACE DEPLOYMENT POUR FRONTEND SERVICES

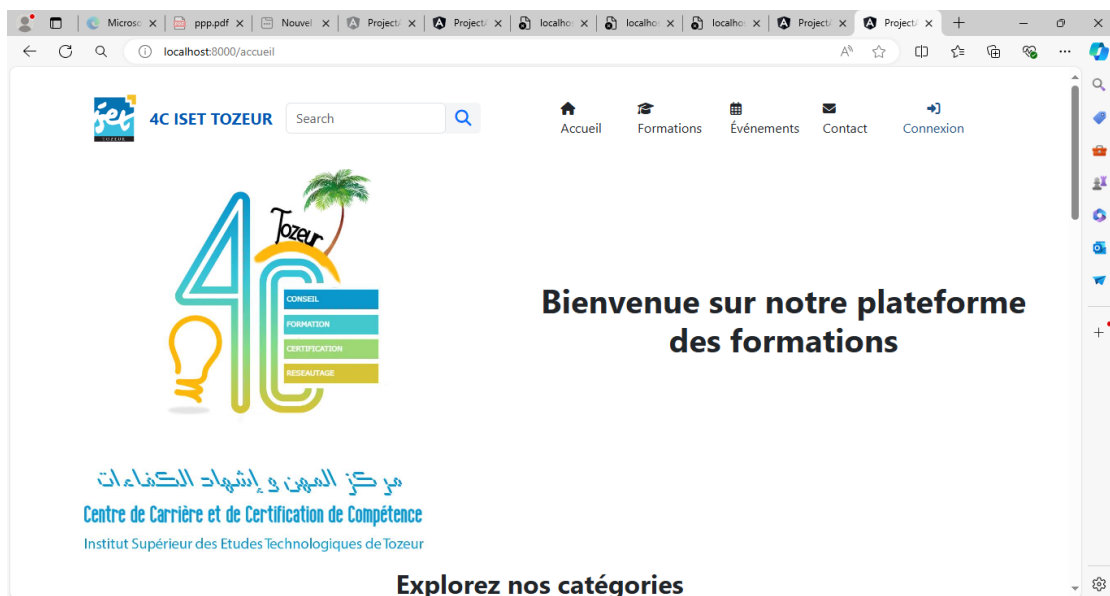


Figure 99.:INTERFACE EXECUTION DEPLOYMENT POUR FRONTEND

V. CONCLUSION :

Le Sprint 3 a permis d'améliorer la stabilité et la performance du frontend, d'automatiser le déploiement avec Docker, Jenkins et Kubernetes, et d'assurer une haute qualité de

code grâce aux tests Selenium et SonarQube. Ces avancées ont renforcé la fiabilité et la maintenabilité du projet, posant des bases solides pour les développements futurs.

CONCLUSION GENERALE

La réalisation de la plateforme de gestion des inscriptions pour le centre 4C ISET Tozeur a été un exemple probant de l'efficacité d'une approche agile et d'une architecture microservices dans le domaine du développement logiciel moderne. En adoptant Scrum comme méthodologie de gestion de projet, l'équipe a pu bénéficier d'une flexibilité et d'une adaptation continues, essentielles dans un environnement où les exigences peuvent évoluer rapidement. La méthode Scrum, avec ses cycles itératifs de développement (sprints), ses réunions quotidiennes de synchronisation (daily stand-ups) et ses revues régulières (sprint reviews), a favorisé une communication ouverte et une collaboration étroite au sein de l'équipe et avec les parties prenantes.

Parallèlement, l'architecture basée sur des microservices a offert plusieurs avantages significatifs. Chaque service, spécialisé dans une fonctionnalité spécifique comme l'authentification des utilisateurs, la gestion des formations ou les notifications, a permis une modularité accrue et une mise à l'échelle indépendante des composants du système. L'utilisation de Docker pour la containerisation et Kubernetes pour l'orchestration des conteneurs a simplifié le déploiement, la gestion et la mise à jour des services, assurant ainsi une haute disponibilité et une gestion efficace des ressources.

En intégrant des pratiques DevOps tout au long du cycle de vie du développement, l'équipe a renforcé la qualité, la sécurité et la fiabilité de la plateforme. Les outils et techniques DevOps, tels que l'intégration continue (CI), le déploiement continu (CD), ainsi que le monitoring et la gestion des logs en temps réel, ont permis de détecter rapidement les erreurs, d'améliorer les performances et de garantir une expérience utilisateur optimale.

En conclusion, ce projet illustre comment une combinaison synergique d'approches agiles, d'architectures modernes comme les microservices, et de pratiques DevOps peut conduire à la création de solutions logicielles robustes, évolutives et alignées sur les besoins spécifiques des utilisateurs finaux. Cette réussite témoigne également de l'importance croissante de l'agilité et de la technologie dans la réalisation de projets informatiques complexes, offrant ainsi un modèle pour les futurs développements dans le domaine.

NETOGRAPHIE

¹ <https://geekflare.com/fr/source-code-management-tools/>

¹ https://www.padok.fr/blog/outils-devops#Automatisation_et_gestion_de_configuration

¹ <https://jch.blog4ever.com/contruire-un-pipeline-devops>

¹ Cours monsieur haithem hafsi Culture devops

Perspective et Intégrations Technologiques

La réalisation de la plateforme de gestion des inscriptions pour le centre 4C ISET Tozeur a été une démonstration remarquable de l'efficacité d'une approche agile et d'une architecture microservices, enrichie par l'utilisation de technologies modernes et de pratiques DevOps avancées.

- Application Mobile Flutter

Pour répondre aux besoins de mobilité et d'accessibilité des utilisateurs, une application mobile basée sur Flutter a été intégrée au système. Cette application offre une interface conviviale et réactive, permettant aux étudiants de parcourir le catalogue des formations disponibles, de s'inscrire à celles de leur choix, et de suivre leur historique de formations directement depuis leurs appareils mobiles. L'intégration de Flutter a facilité une expérience utilisateur harmonieuse, tout en permettant une gestion efficace des processus d'inscription et de participation aux formations.

- Notifications par Email via Gmail

Chaque étape du processus d'inscription, de l'acceptation à la confirmation, est notifiée aux étudiants via des notifications par email. Grâce à l'intégration avec Gmail, les étudiants reçoivent instantanément des notifications personnalisées lorsqu'ils sont acceptés ou refusés pour une formation particulière. Cette fonctionnalité non seulement assure une communication transparente, mais aussi renforce l'engagement des étudiants en leur fournissant des informations cruciales en temps opportun.

- Statistiques et Dashboard Personnalisé

Pour améliorer la transparence et permettre une auto-évaluation efficace, chaque étudiant dispose d'un tableau de bord personnalisé et d'une section de statistiques. Ces outils offrent une vue d'ensemble claire du nombre de formations auxquelles ils ont participé, de leurs progrès, et des certifications obtenues. Cette fonctionnalité aide les étudiants à gérer leur parcours de formation de manière autonome, en visualisant leur historique et en identifiant les domaines où des améliorations peuvent être apportées.

Conclusion

L'intégration de Flutter pour une application mobile intuitive, l'utilisation de notifications par email via Gmail pour une communication instantanée, et la mise en place de tableaux de bord personnalisés pour les statistiques individuelles illustrent l'engagement du projet à fournir une expérience utilisateur optimale et centrée sur les besoins spécifiques des étudiants. Ces innovations, combinées à une gestion agile et à une architecture microservices robuste, positionnent la plateforme de gestion des inscriptions comme une solution moderne et adaptable, prête à soutenir efficacement les besoins croissants du centre 4C ISET Tozeur dans le domaine de la formation continue.

GLOSSAIRE

CI : (Continuous Integration)

CD : (Continuous Deployment/Delivery)

SDLC : (Software Development Life Cycle)

RESUME

L'implantation de DevOps au centre 4C de l'ISET Tozeur a été une véritable réussite, transformant la plateforme de gestion des inscriptions aux événements et aux cours en un outil plus efficace et évolutif. En adoptant les principes DevOps, tels que l'architecture microservices, la conteneurisation et les pratiques CI/CD, le centre a considérablement amélioré la vitesse de déploiement des logiciels, leur fiabilité et la satisfaction des utilisateurs. Les méthodologies agiles appliquées tout au long du cycle de vie du projet ont permis des améliorations progressives et une réactivité accrue face aux changements de besoins. Ce rapport démontre clairement les avantages concrets de DevOps dans les contextes de gestion éducative et événementielle, et met en lumière son potentiel pour rationaliser les opérations et stimuler l'innovation continue.

SUMMARY

The implementation of DevOps at the 4C Center of ISET Tozeur has been a remarkable success, transforming the platform for managing event and course registrations into a more efficient and scalable tool. By adopting DevOps principles such as microservices architecture, containerization, and CI/CD practices, the center has significantly improved software deployment speed, reliability, and user satisfaction. Agile methodologies applied throughout the project lifecycle have enabled iterative improvements and increased responsiveness to changing needs. This report clearly demonstrates the tangible benefits of DevOps in educational and event management contexts, highlighting its potential to streamline operations and drive continuous innovation.

ملخص

نجاحًا ملحوظًا، حيث حوّل منصة إدارة تسجيل الفعاليات ISET Tozeur التابع لـ C في مركز 4 DevOps كان تنفيذ مثل تقنية الخدمات المصغرة DevOps والدورات إلى أداة أكثر كفاءة وقابلية للتطوير. من خلال اعتماد مبادئ ، نجح المركز في تحسين سرعة نشر CI/CD وممارسات (Containerization) والتحميل (Microservices) البرمجيات، وموثوقيتها، ورضا المستخدمين بشكل كبير. استخدمت المنهجيات السريعة طوال دورة حياة المشروع لتمكين DevOps التحسينات التدريجية وزيادة الاستجابة للتغيرات في الاحتياجات. يبرز هذا التقرير بوضوح الفوائد الملموسة لـ في سياقات إدارة التعليم والفعاليات، مشيرًا إلى إمكانيته في تبسيط العمليات ودفع الابتكار المستمر.

